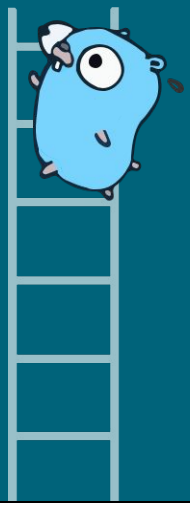


Advancing Go Garbage Collection with *Green Tea*

Michael Knyszek | Go Team | GopherCon 2025

Today we'll be talking about Go garbage collection and how we've made it better with a new technique we call Green Tea.

TL;DR: Garbage collection will be *faster* in Go 1.26.

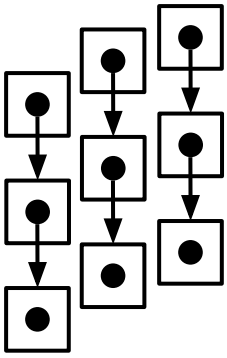
If you take nothing else from this talk, we made Go garbage collection faster, and we plan to make it generally available in Go 1.26.

You could leave now, but I'm not going to tell you why it's called Green Tea until the very end. So stick around if you'd like to find out. ;)

Intro to garbage collection

Before we can talk about Green Tea let's get us all on the same page.

Garbage collection



To **reclaim memory,** concerns itself with **objects** and **pointers**.

Objects: one or more Go values heap-allocated together

```
var x = make([]*int, 10) // global
```

Pointers: memory address of a Go value

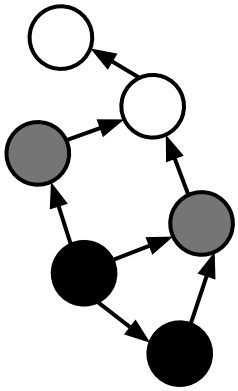
```
&x[0] // 0xc000104000
```

The purpose of garbage collection is to automatically reclaim and reuse memory no longer used by the program.

To this end, the Go garbage collector concerns itself with objects and pointers. Objects are Go values whose underlying memory is allocated from the heap. Objects are created when the compiler can't figure out how else to allocate memory for a value.

To figure out whether objects are still being used, the garbage collector looks at the pointers that refer to these objects. Pointers of course are the memory addresses of Go values.

Tracing garbage collection



Mark-sweep algorithm. ***Much*** simpler than it sounds.

Mark: **walk the object graph**.

Objects are *nodes*, pointers are *edges*.

Marking objects means they've been *visited*.

Sweep: **iterate** over all nodes and free *unvisited* ones.

Go's garbage collector follows a strategy broadly referred to as tracing garbage collection, which just means that the garbage collector follows, or traces, the pointers in the program to identify which objects the program is still using.

More specifically, the Go garbage collector implements the mark-sweep algorithm. This is much simpler than it sounds. Imagine objects and pointers as a sort of graph, in the computer science sense. Objects are nodes, pointers are edges.

The mark-sweep algorithm proceeds in two phases. First, we walk the object graph from well-defined source edges called roots. Think global and local variables. Then, we mark everything we find along the way as visited to avoid going in circles. This is your typical graph flood algorithm, like a depth-first or breadth-first search.

Once we're done with that, we reach the second phase: the sweep phase. Whatever nodes were not visited in our graph walk are unused, or unreachable, by the program. We call this state unreachable because it is impossible with normal safe Go code to access that memory, simply through the semantics of the language. So, we iterate through all the unvisited nodes and mark that memory as free to the memory allocator.

Don't sweat the details!

It's more **complex** than that, but **it's all details**.

Marking is concurrent with user code.

Marking is parallelized.

Sweeping is incremental.

When do we run a collection?

...

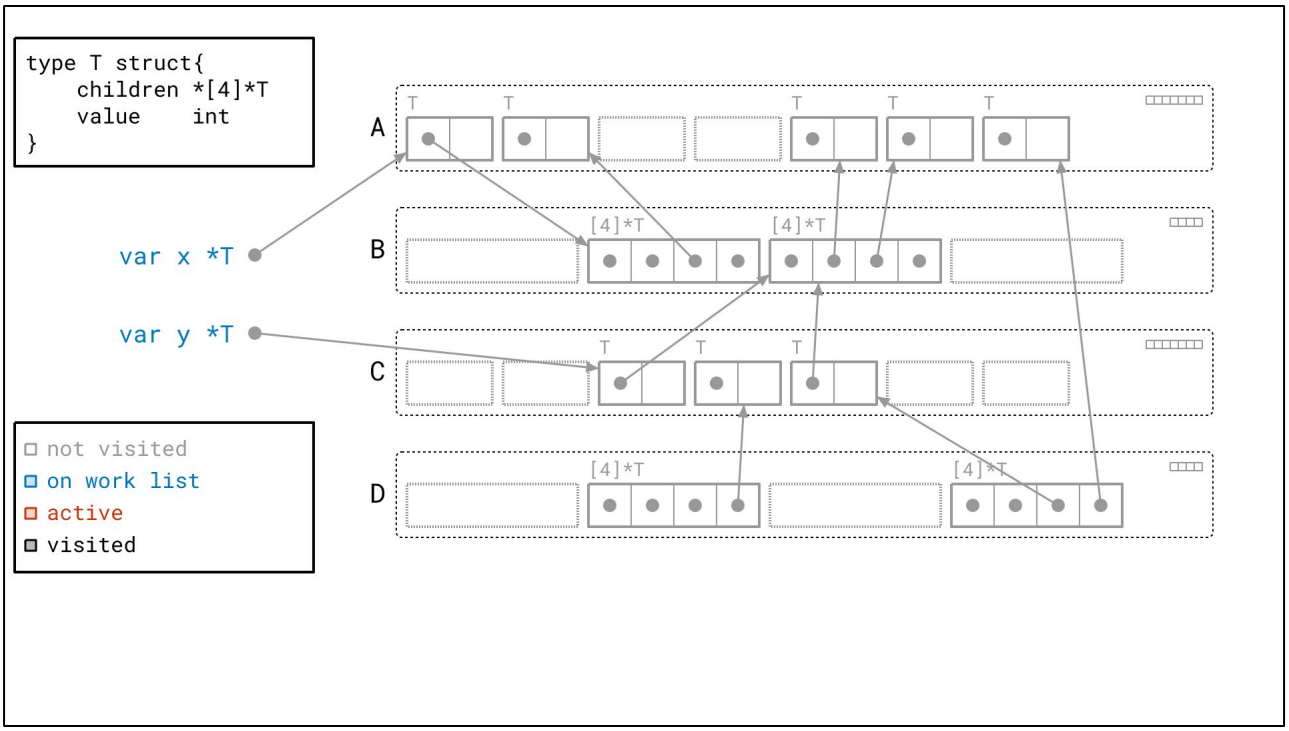
You may think I'm oversimplifying a bit here. Garbage collectors are frequently referred to as *magic*, and *black boxes*.

Don't get me wrong, there are more complexities. For example, this algorithm is, in general, executed concurrently with your regular Go program. Walking a graph that's mutating underneath you brings challenges. We also are able to parallelize this algorithm, which is a detail that'll come up again later.

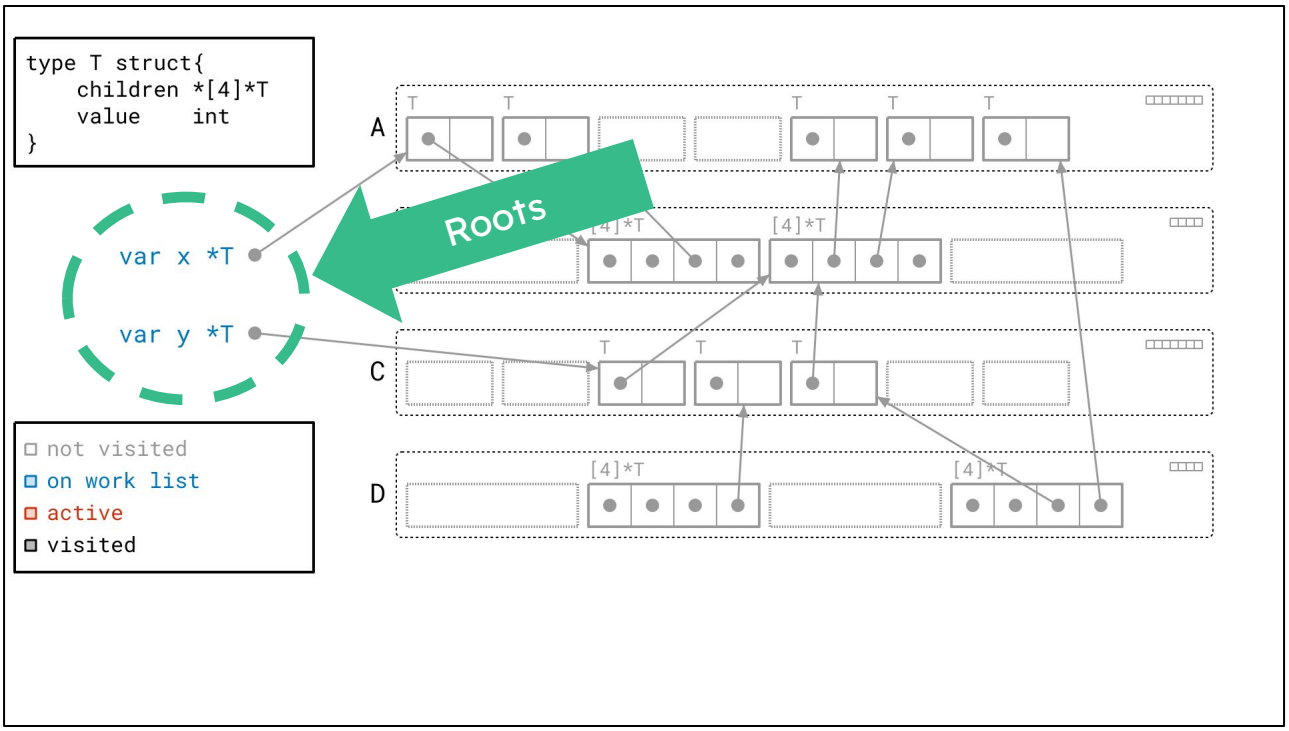
But trust me when I tell you that these details are mostly separate from the core algorithm. It really is just a simple graph flood at the center.

Graph flood example

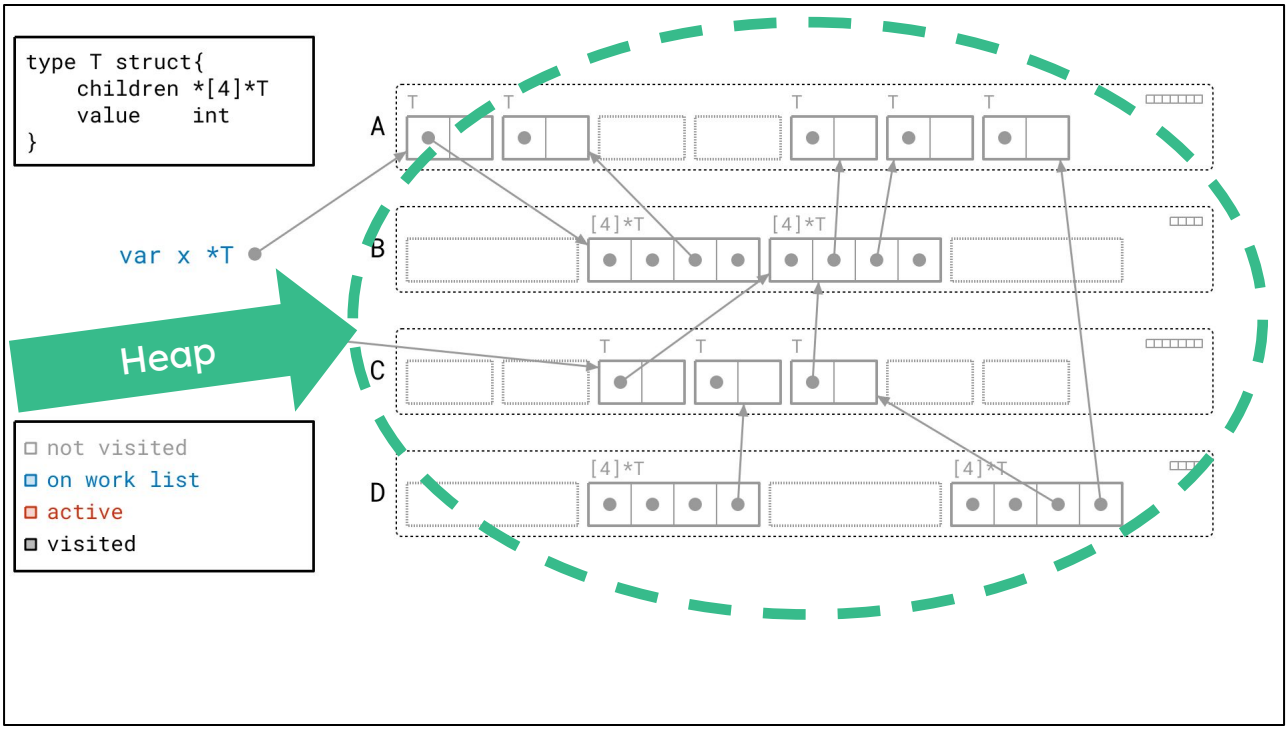
Let's go over a simple example.



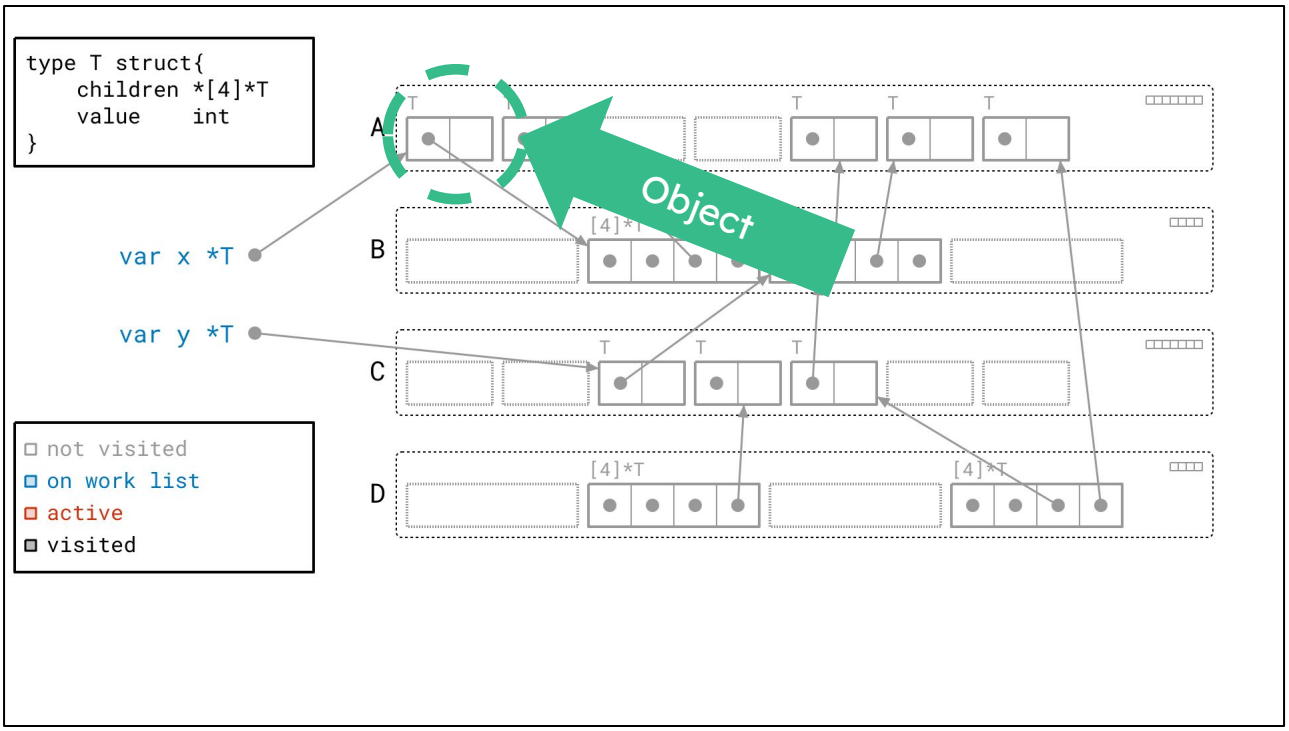
Here we have a diagram of some global variables and Go heap. There's a lot going on here, so bear with me for a moment. This is important because we'll be using this example throughout the talk. Let's break it down, piece by piece.



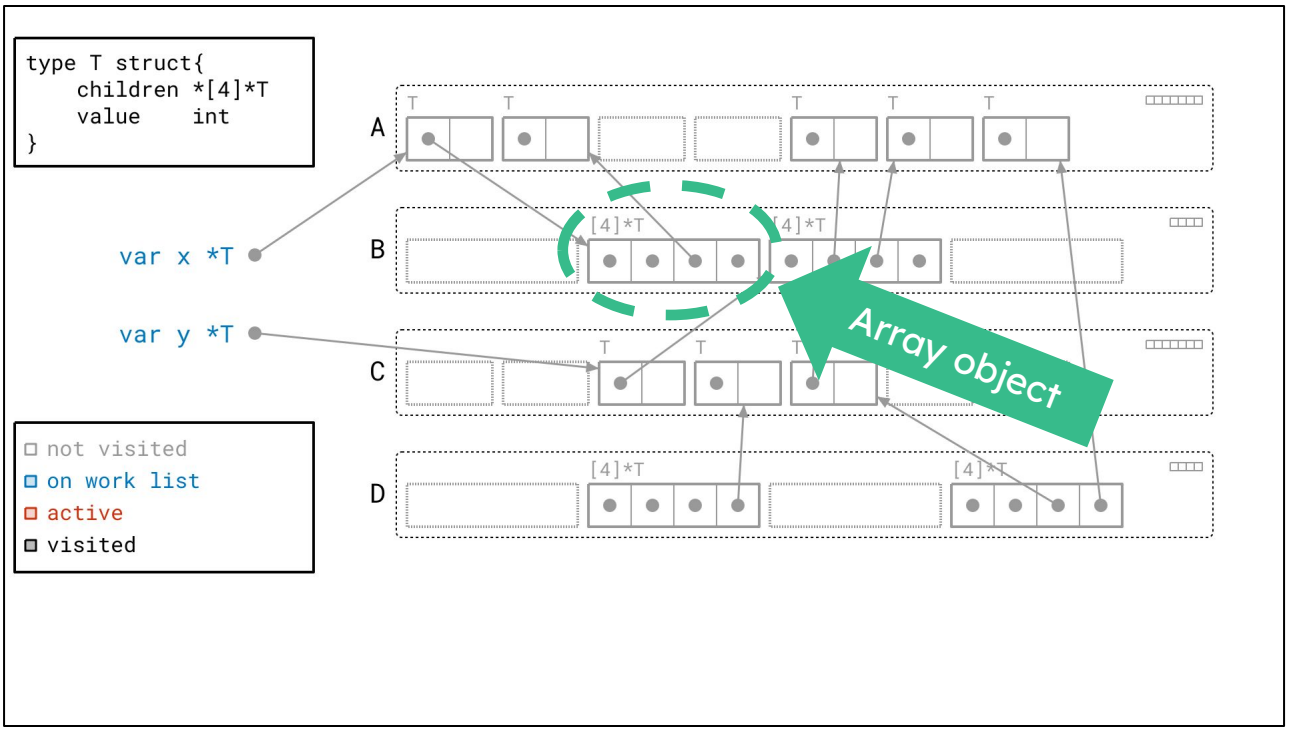
On the left here we have our roots. These are global variables x and y. They will be the starting point of our graph walk. Since they're marked blue, according to our handy legend in the bottom left, they're currently on our work list.



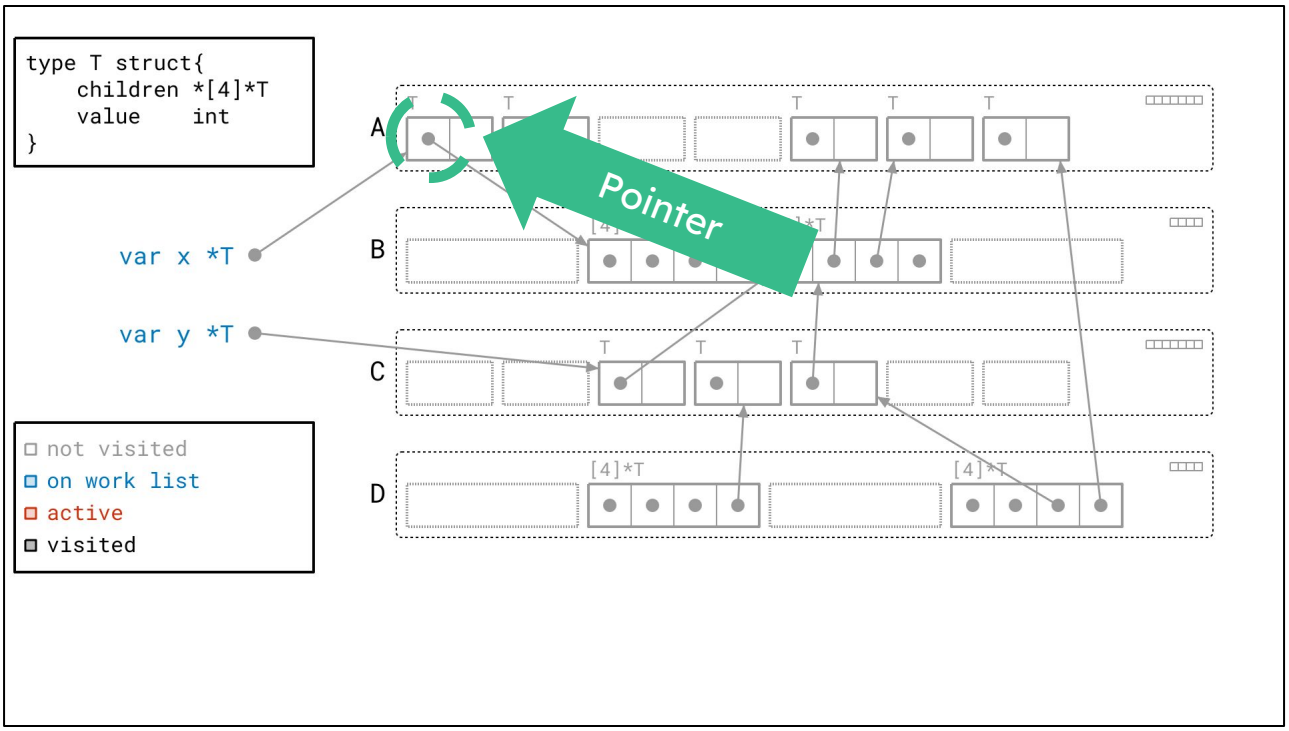
On the right side, we have our heap. Currently, everything in our heap is grayed out because we haven't visited any of it yet. Again, according to the legend in the bottom left.



Each one of these rectangles represents an object. Each object is labeled with its type. This object in particular is an object of type T, whose type definition is on the top left. It's got a pointer to an array of children, and some value. We can surmise that this is some kind of recursive tree data structure.

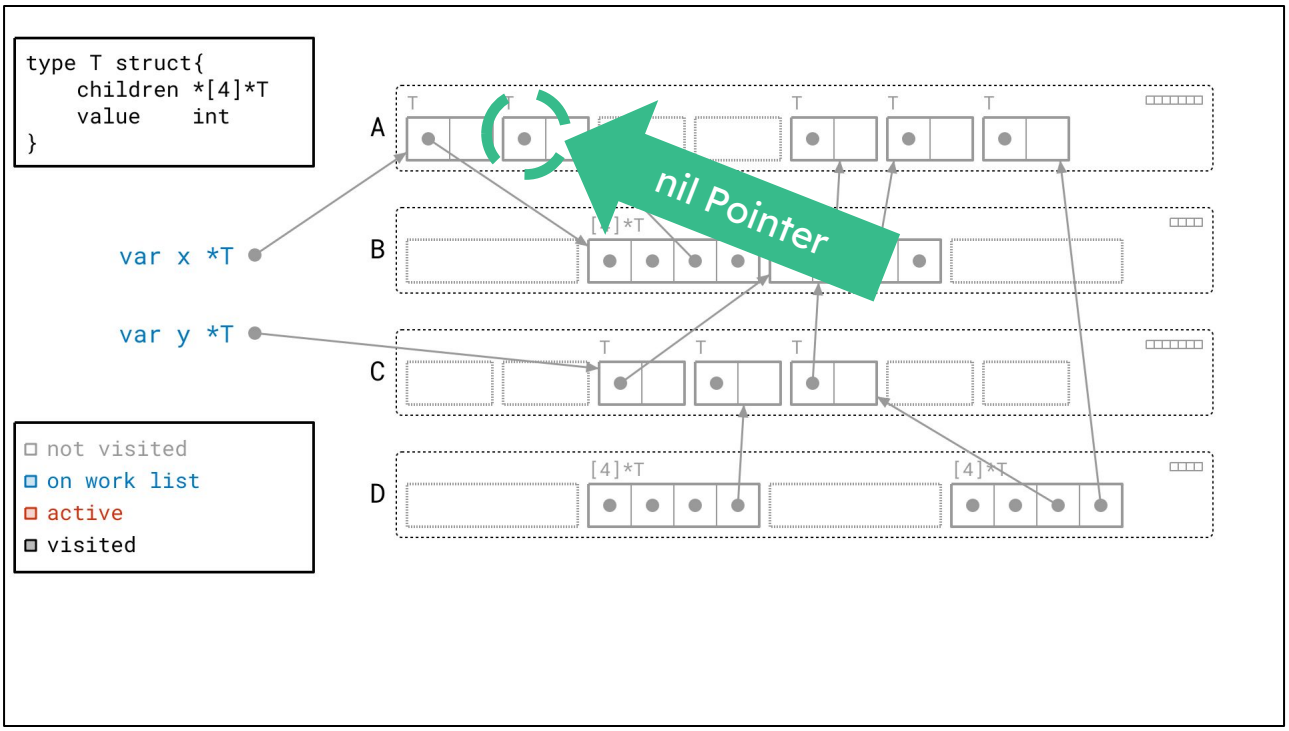


In addition to the objects of type T, you'll also notice that we have array objects containing star-Ts. These are pointed to by the "children" field of objects of type T.

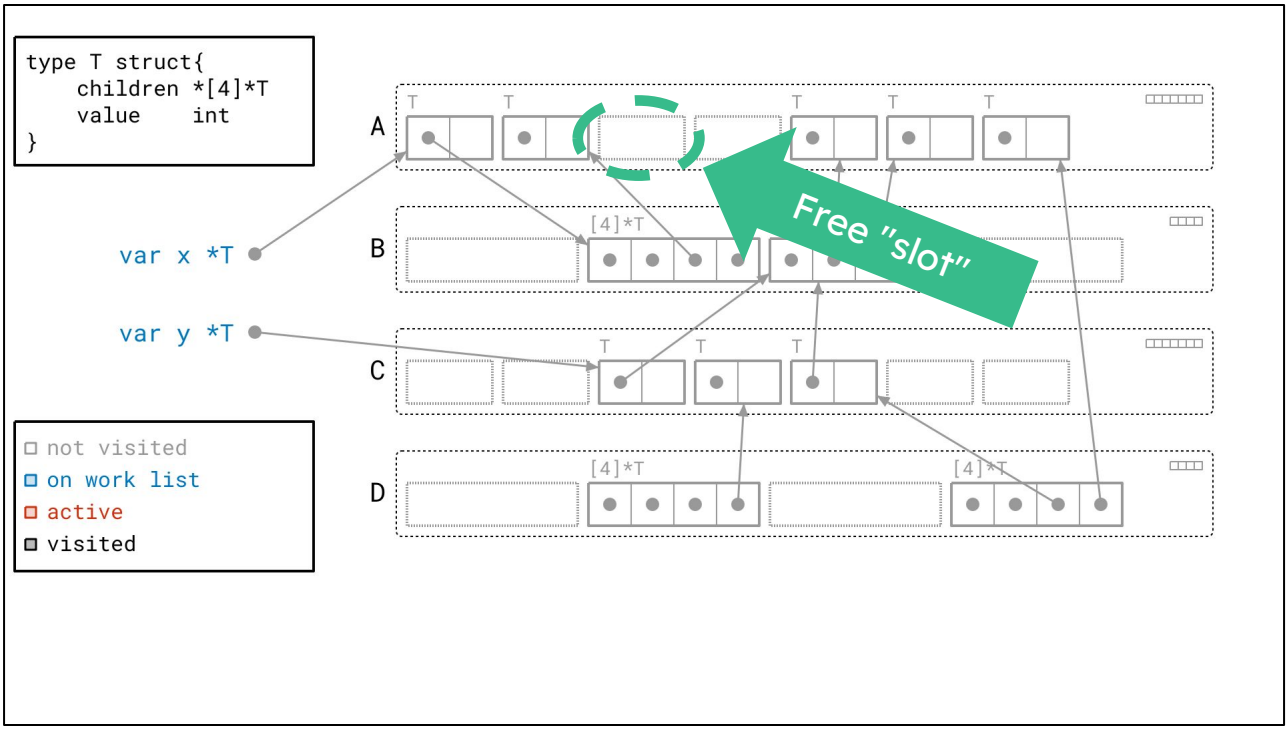


Each square inside of the rectangle represents 8 bytes of memory.

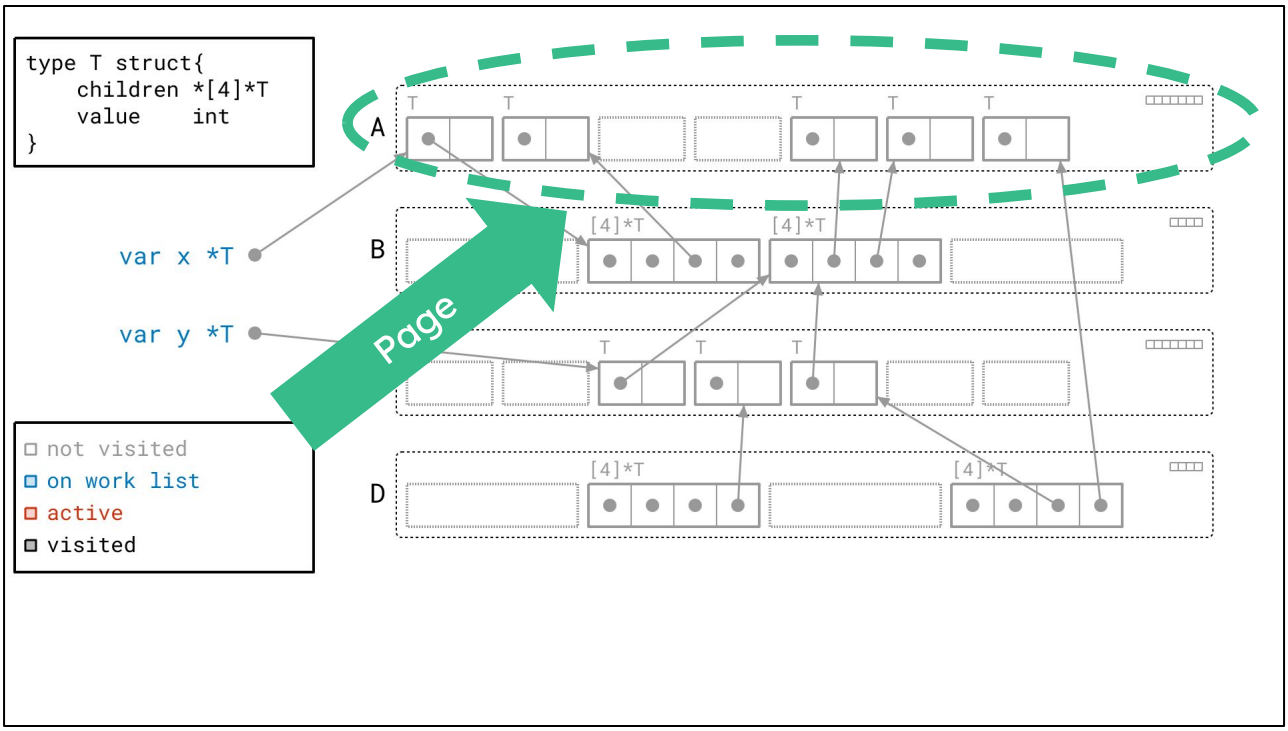
A square with a dot is a pointer. If it has an arrow, it is a non-nil pointer pointing to some other object.



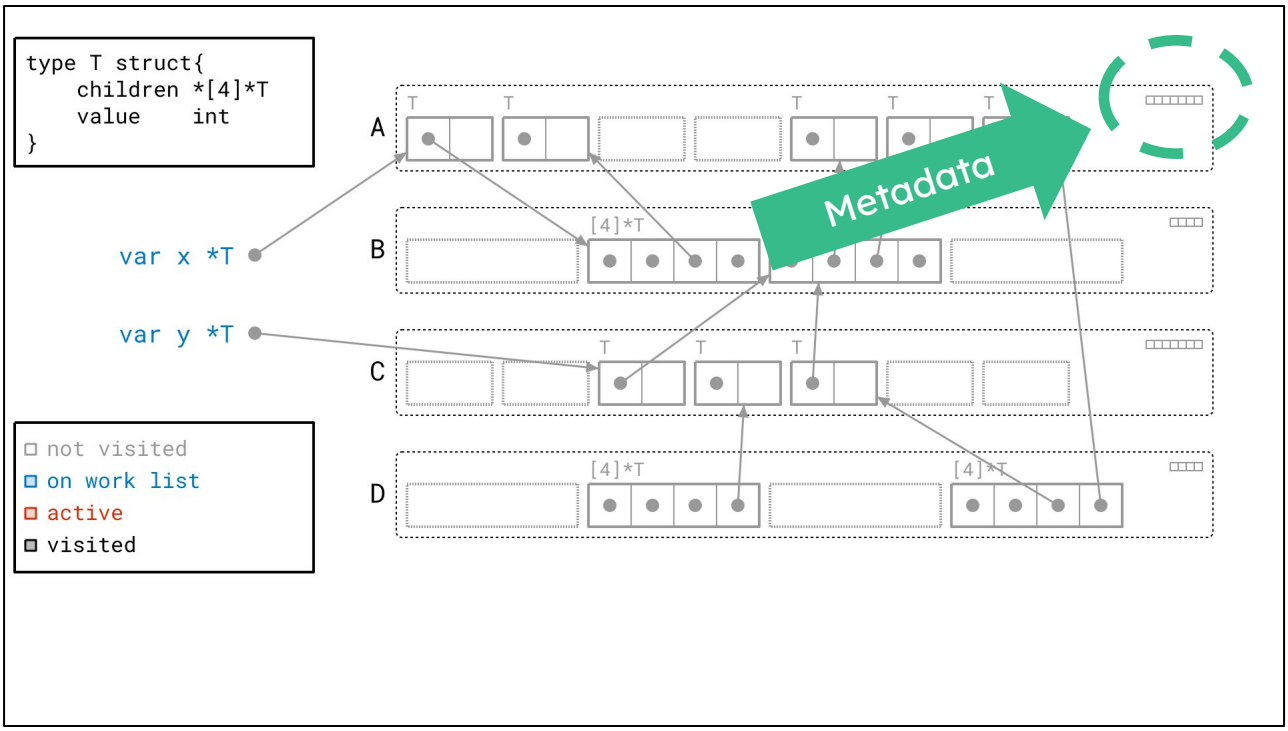
And if it doesn't have a corresponding arrow, then it's a nil pointer.



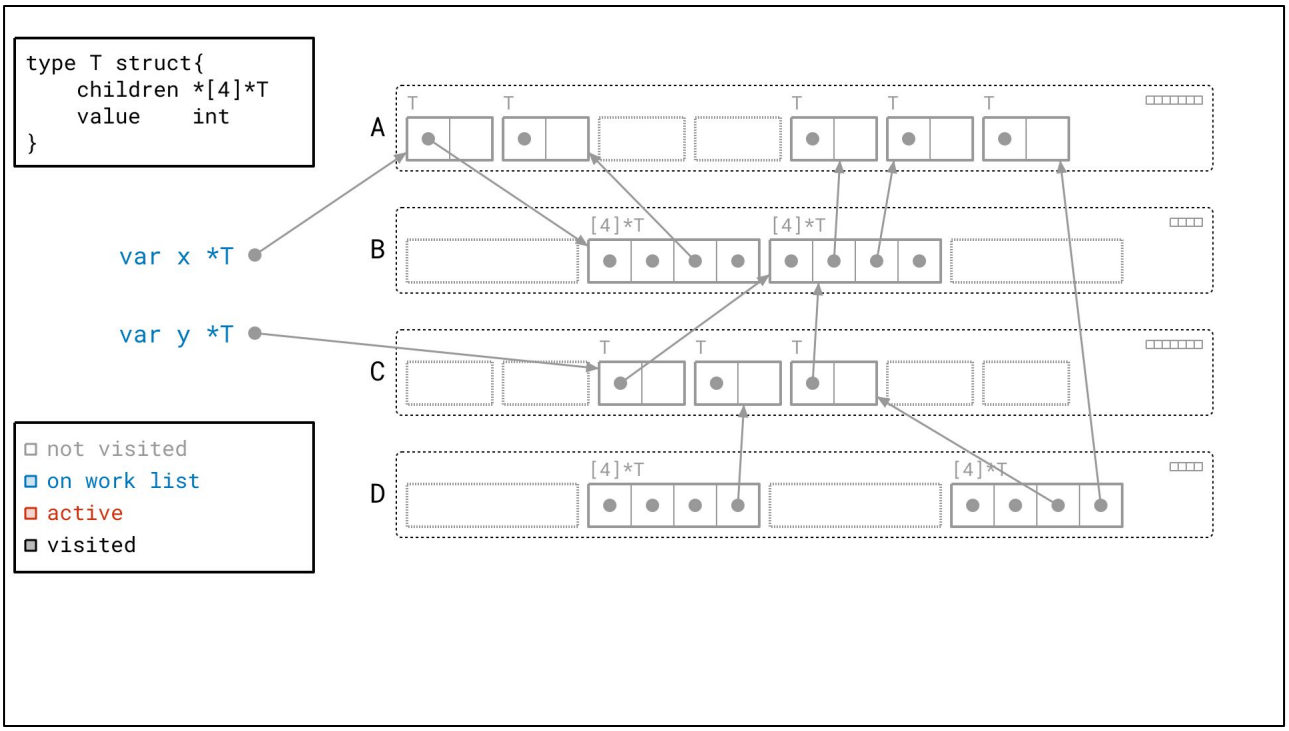
Next, these dotted rectangles represents free space, what I'll call a free "slot." We could put an object there, but there currently isn't one.



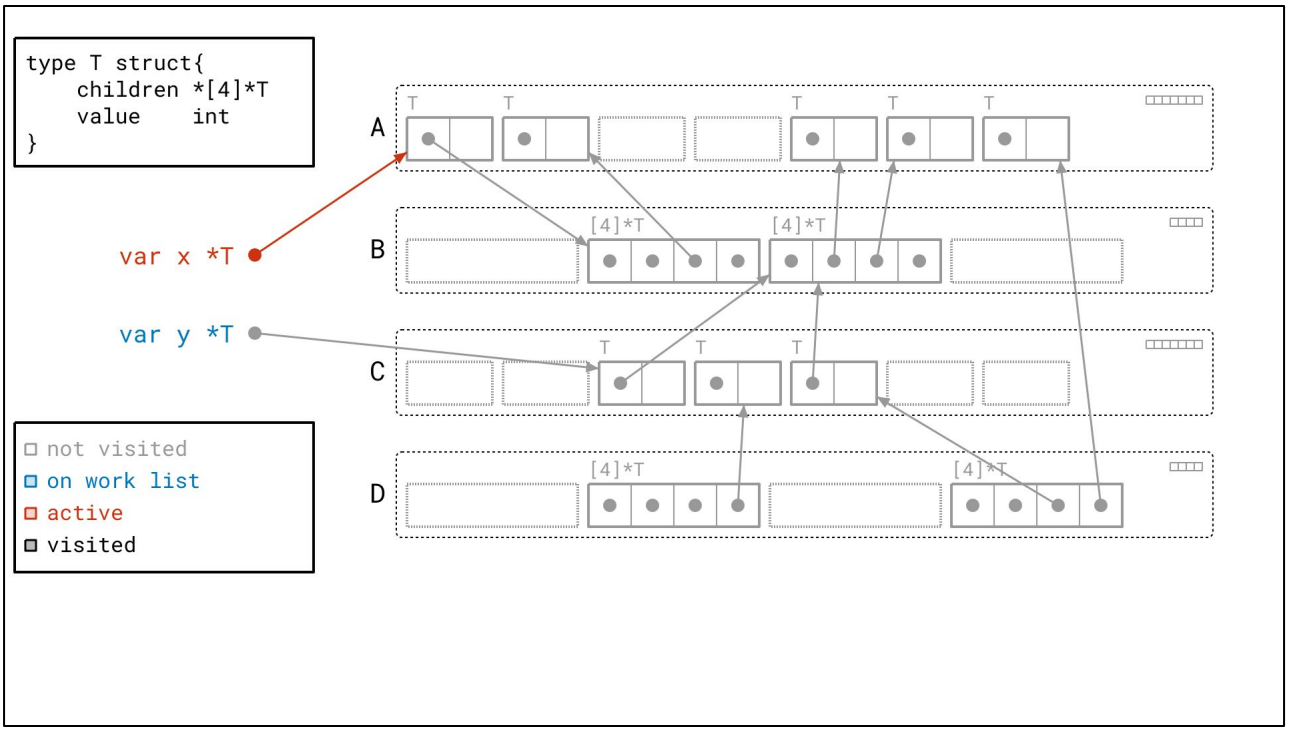
You'll also notice that objects are grouped together by these labeled, dotted rounded rectangles. Each of these represents a page: a contiguous block of memory. In this diagram, we see that each object is allocated as part of some page. Like in the real implementation, each page here only contains objects of a certain size. This is just how the Go heap is organized. Finally, these pages are labeled A, B, C, and D, and I'll refer to them that way.



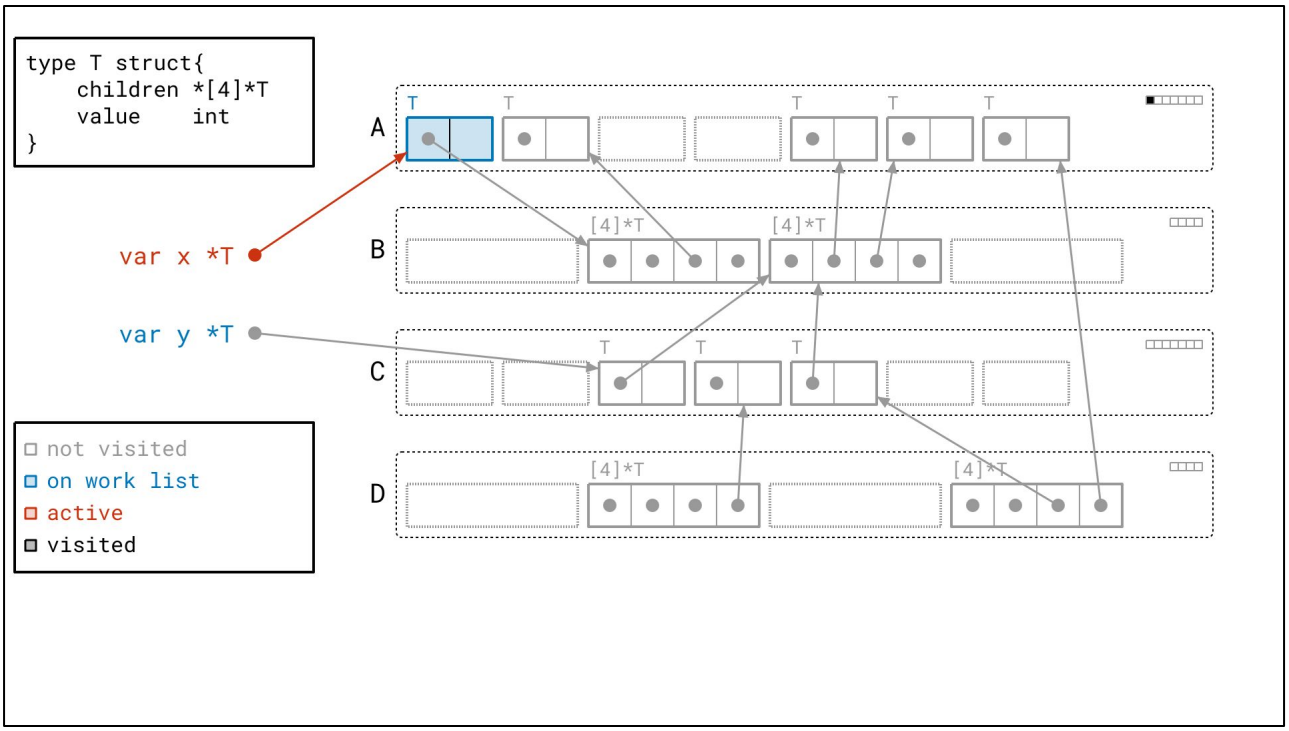
Pages are also how we organize *per-object metadata*. Here you can see seven boxes, each corresponding to one of the seven object slots in page A. Each box represents one bit of information: whether or not we have *seen* the object before. This is actually how the real runtime manages whether an object has been visited, and it'll be an important detail later.



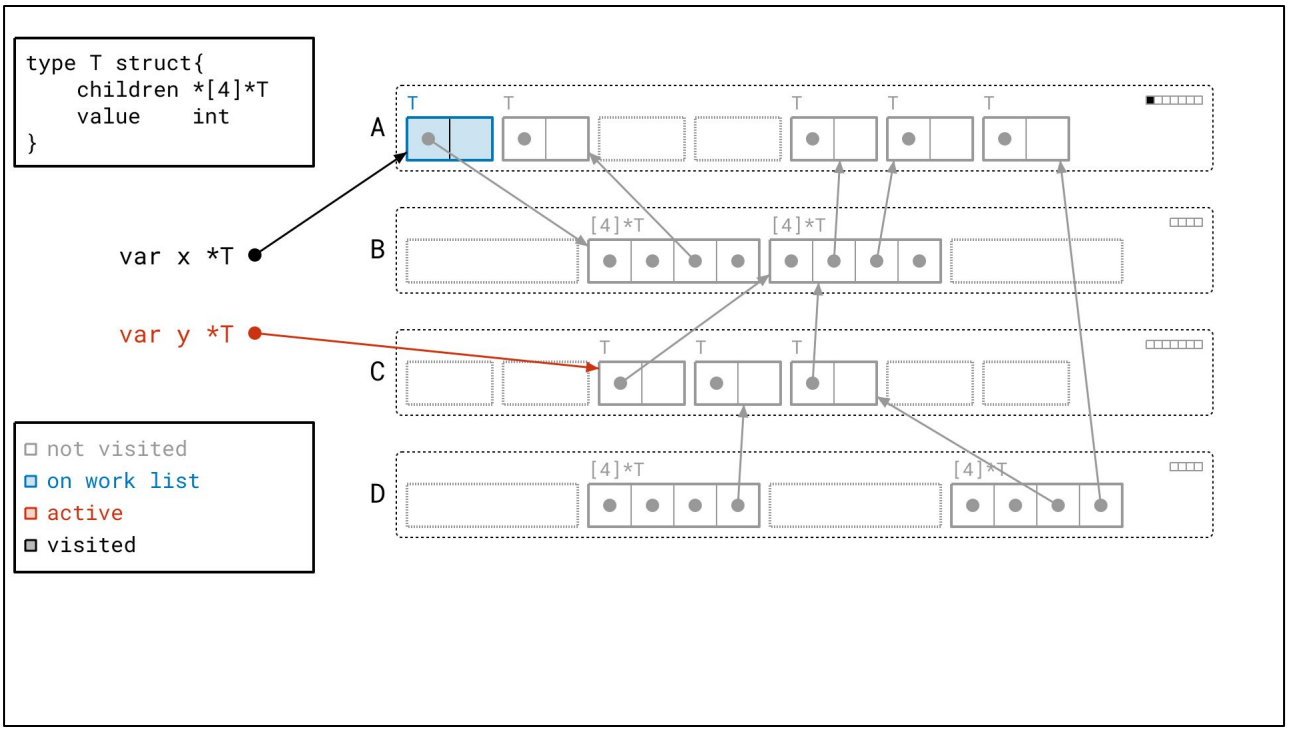
Alright, thanks for staying with me. That was a lot of detail to explain after I just told you to not sweat the details, but I promise, this will all come into play later. For now, let's just see how our graph flood applies to this picture.



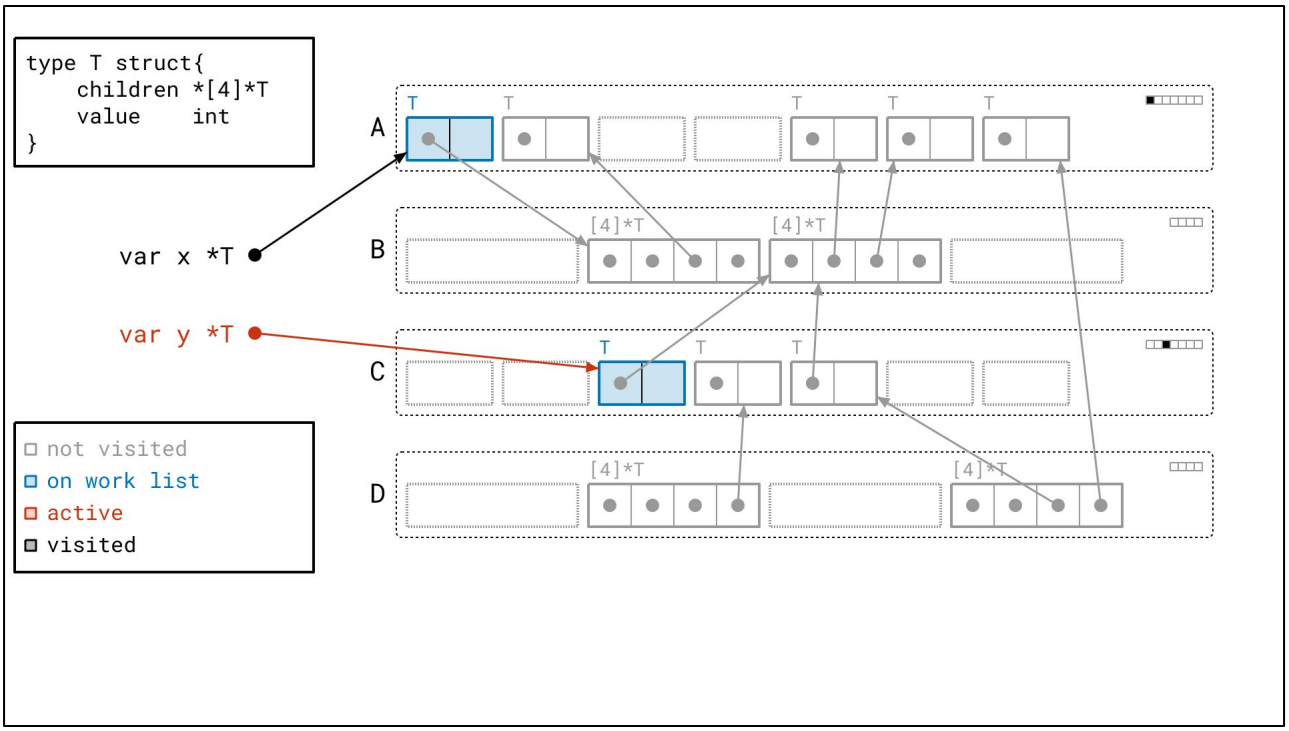
We start by taking a root off of the work list. We mark it red to indicate that it's now *active*.



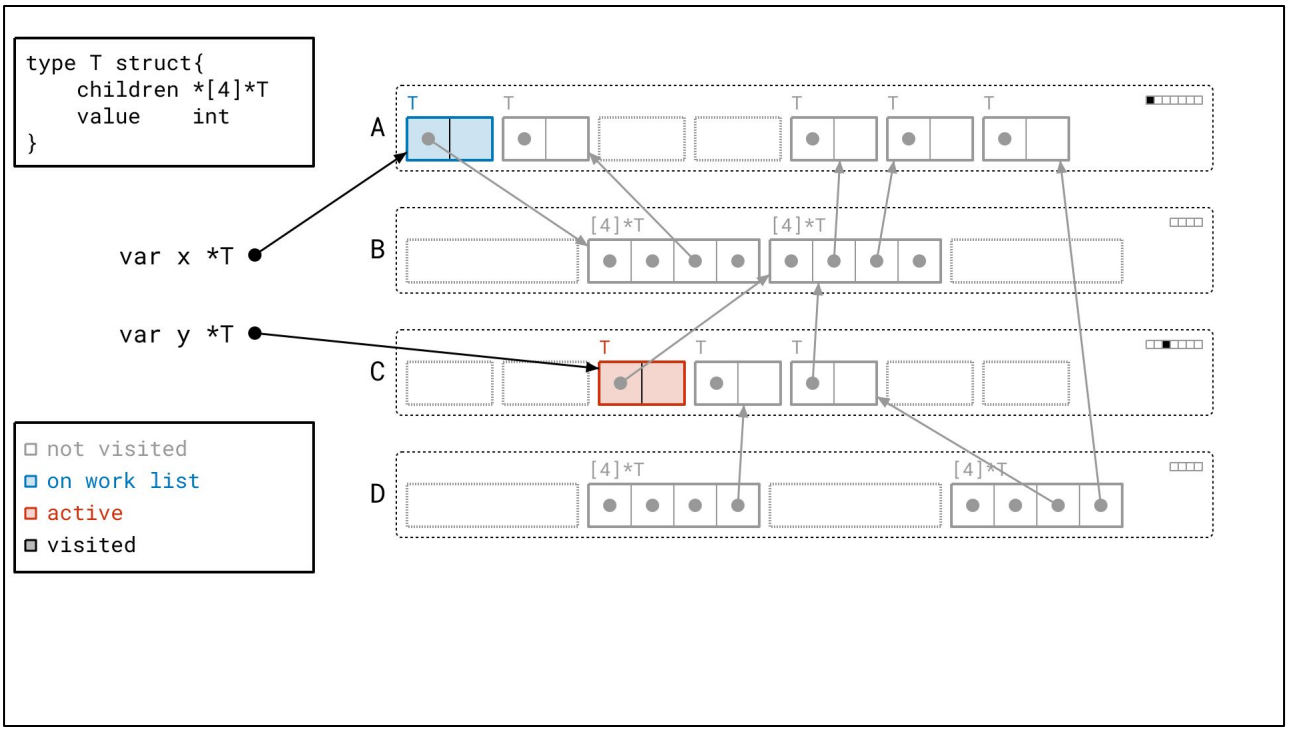
Following that root's pointer, we find an object of type T, which we add to our work list. Following our legend, we draw the object in blue to indicate that it's on our work list. Note also that we set the *seen bit* corresponding to this object in our metadata.



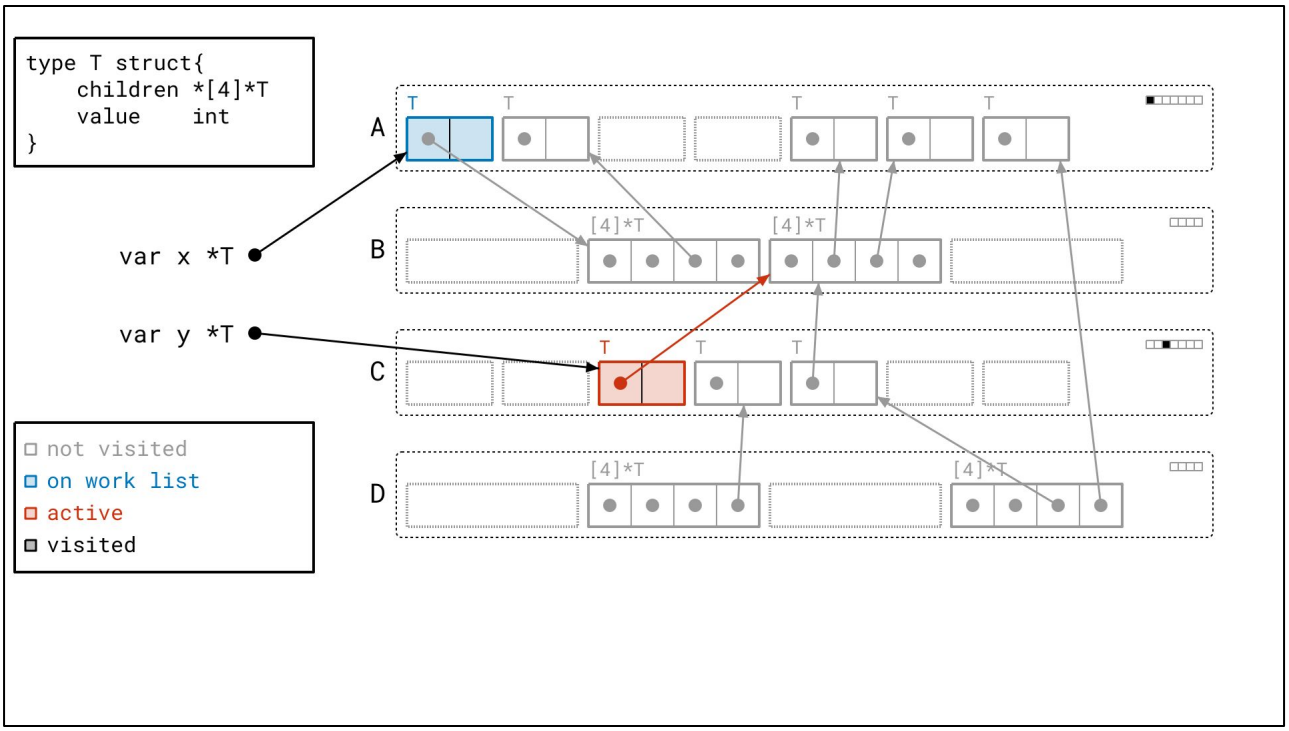
Same goes for the next root.



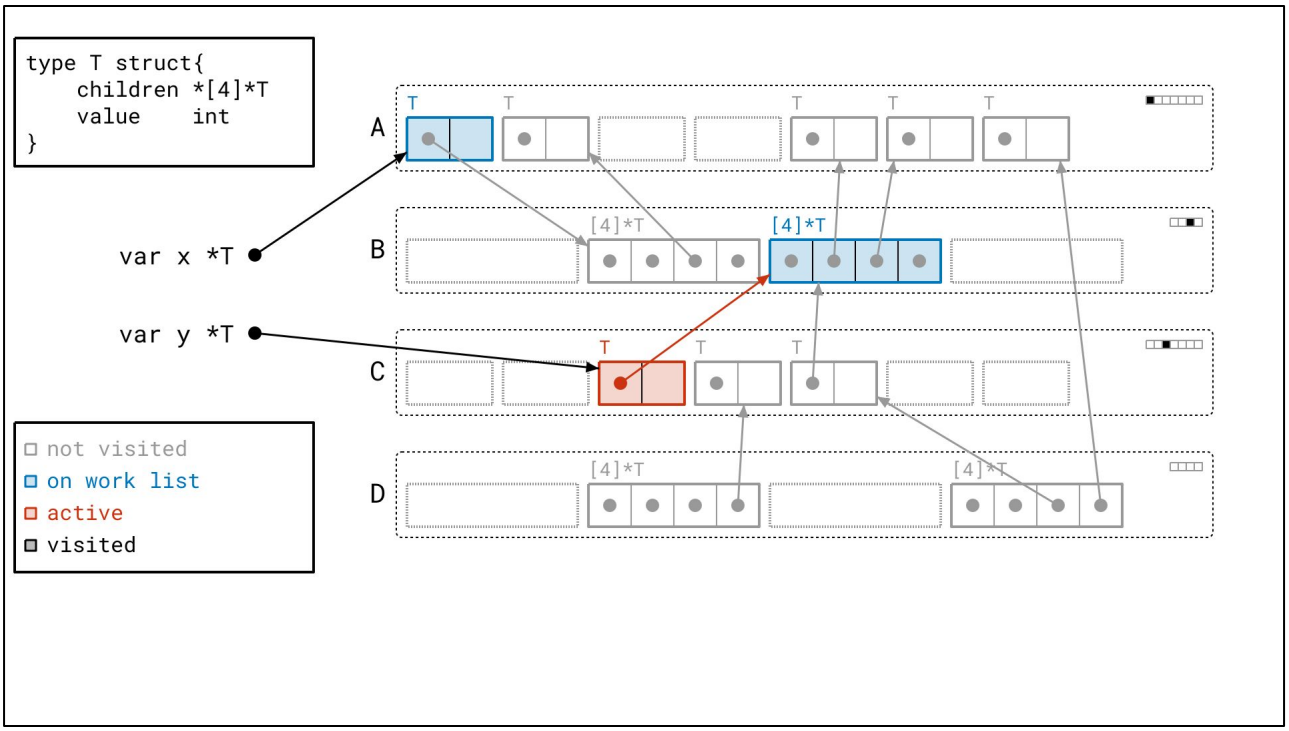
Now that we've taken care of all the roots, we're left with two objects on our work list. Let's take an object off the work list.



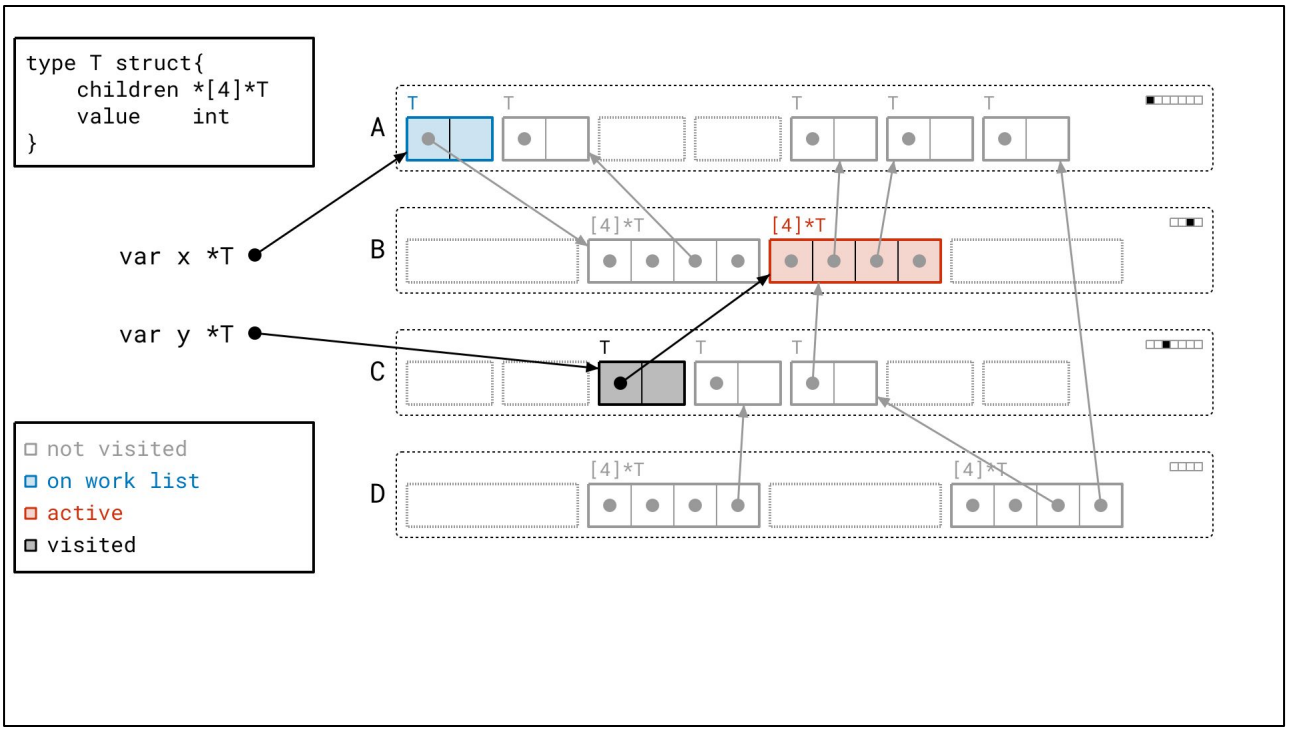
What we're going to do now is walk the pointers of the objects, to find more objects. By the way, we call walking the pointers of an object "scanning" the object.



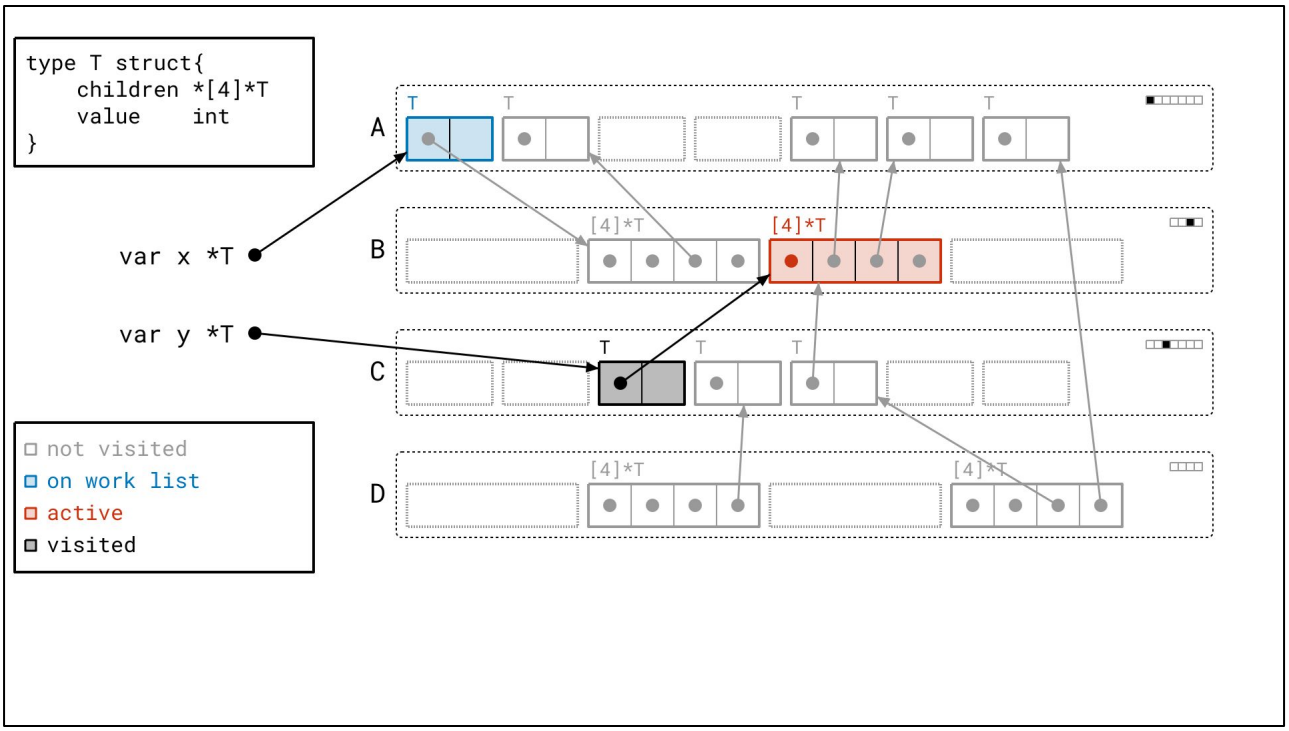
We find this valid array object…



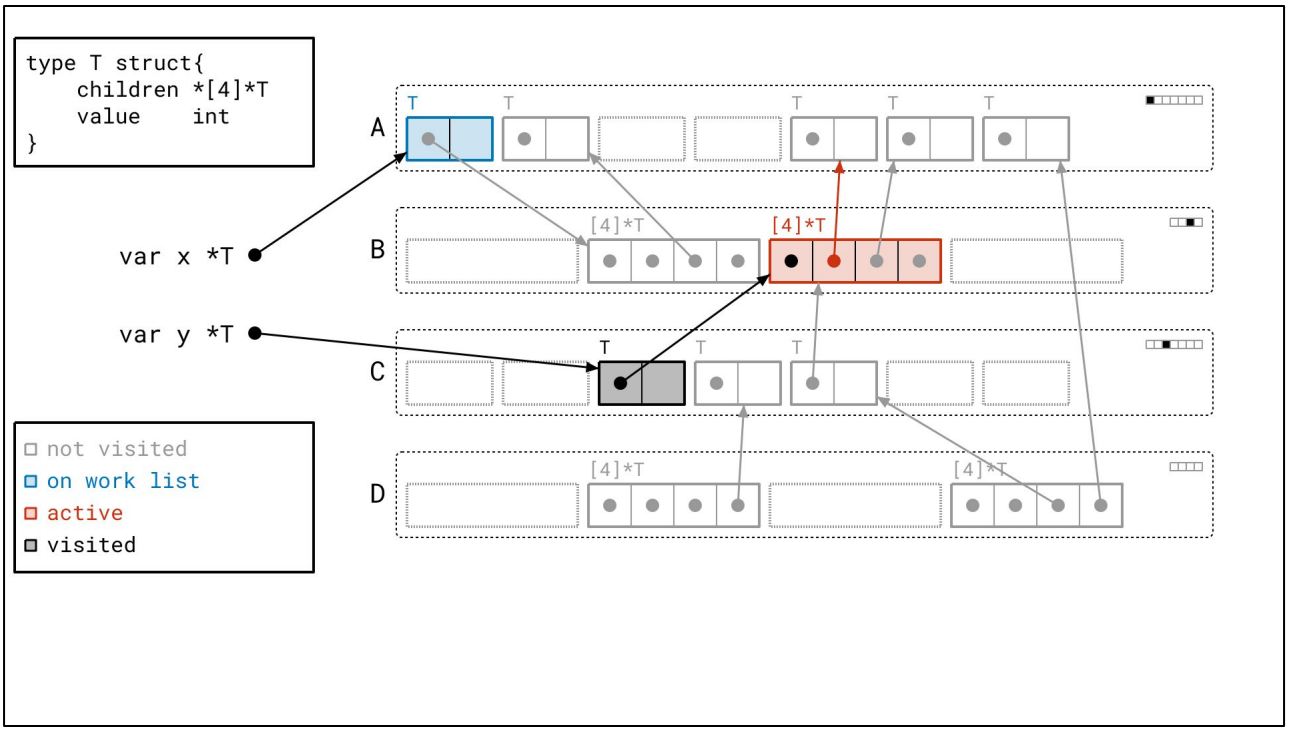
… and add it to our work list.



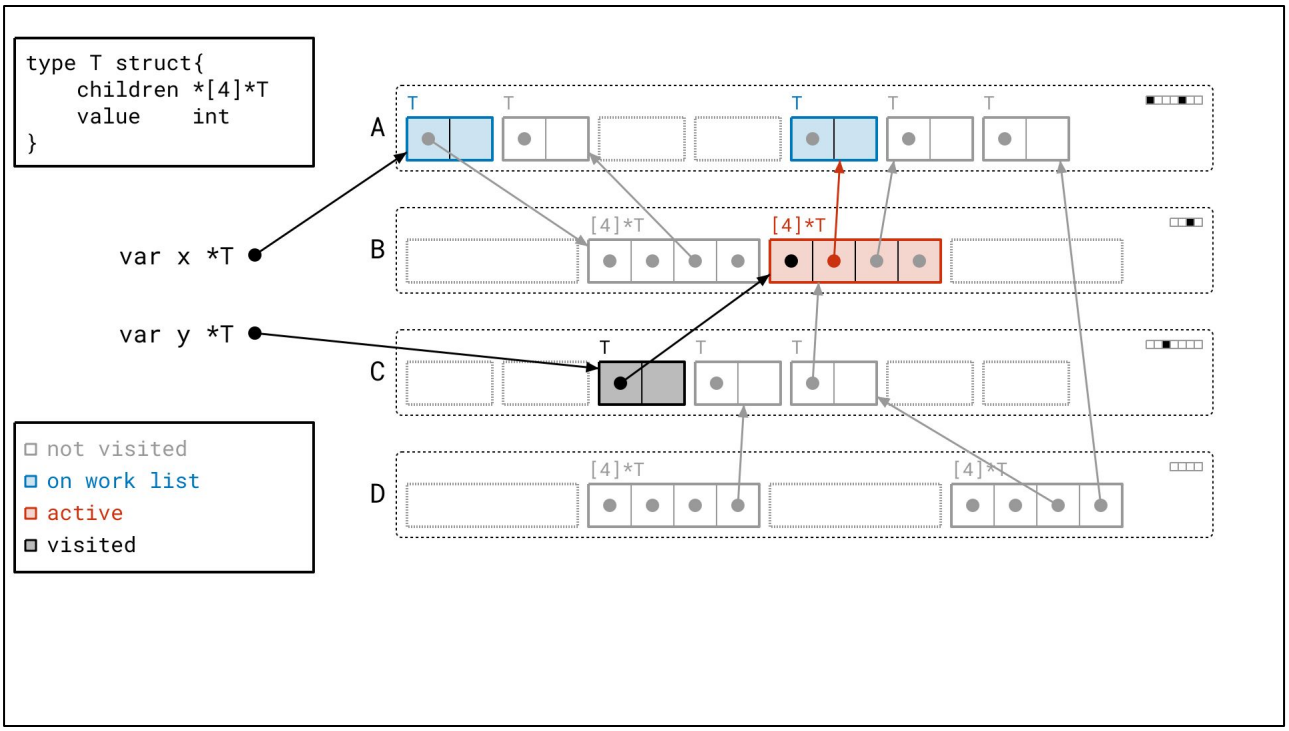
From here, we proceed recursively.



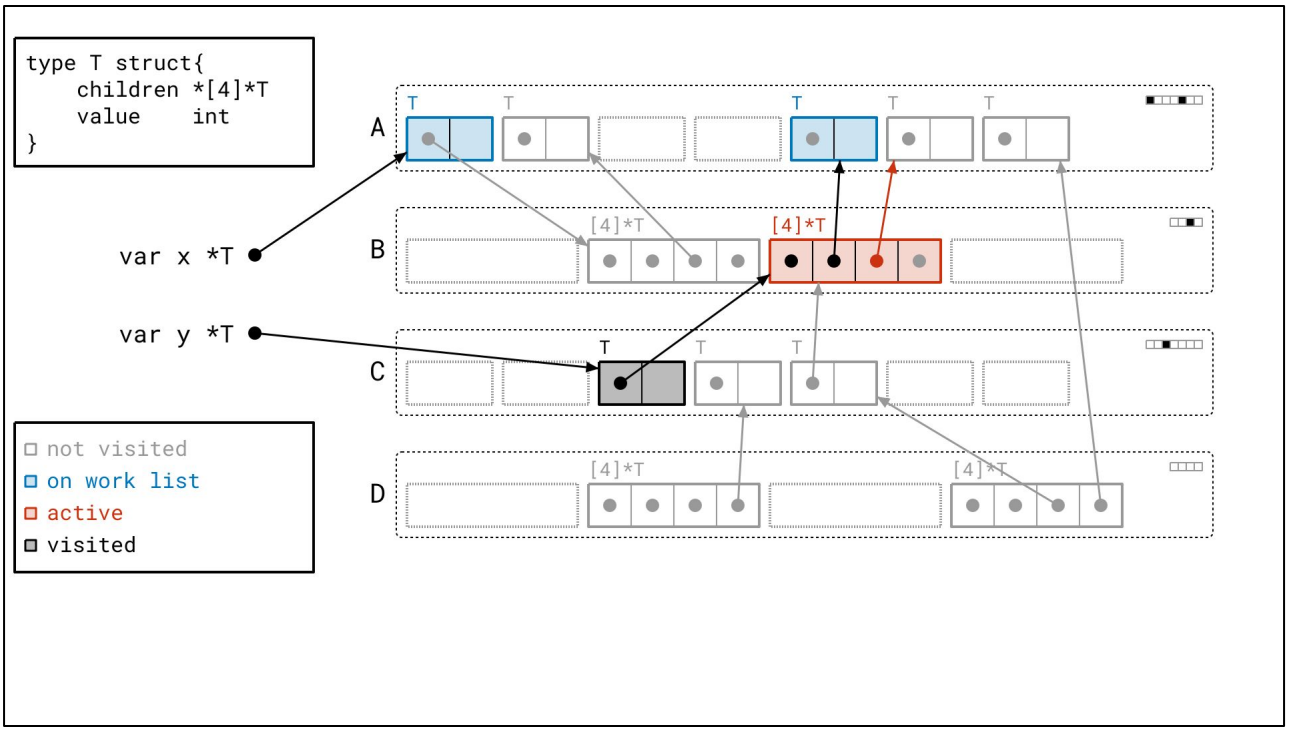
We walk the array's pointers.


type T struct{
children *[4]*T
value int
}
var x *T
var y *T
A
B
C
D
T
[4]*T
not visited
on work list
active
visited

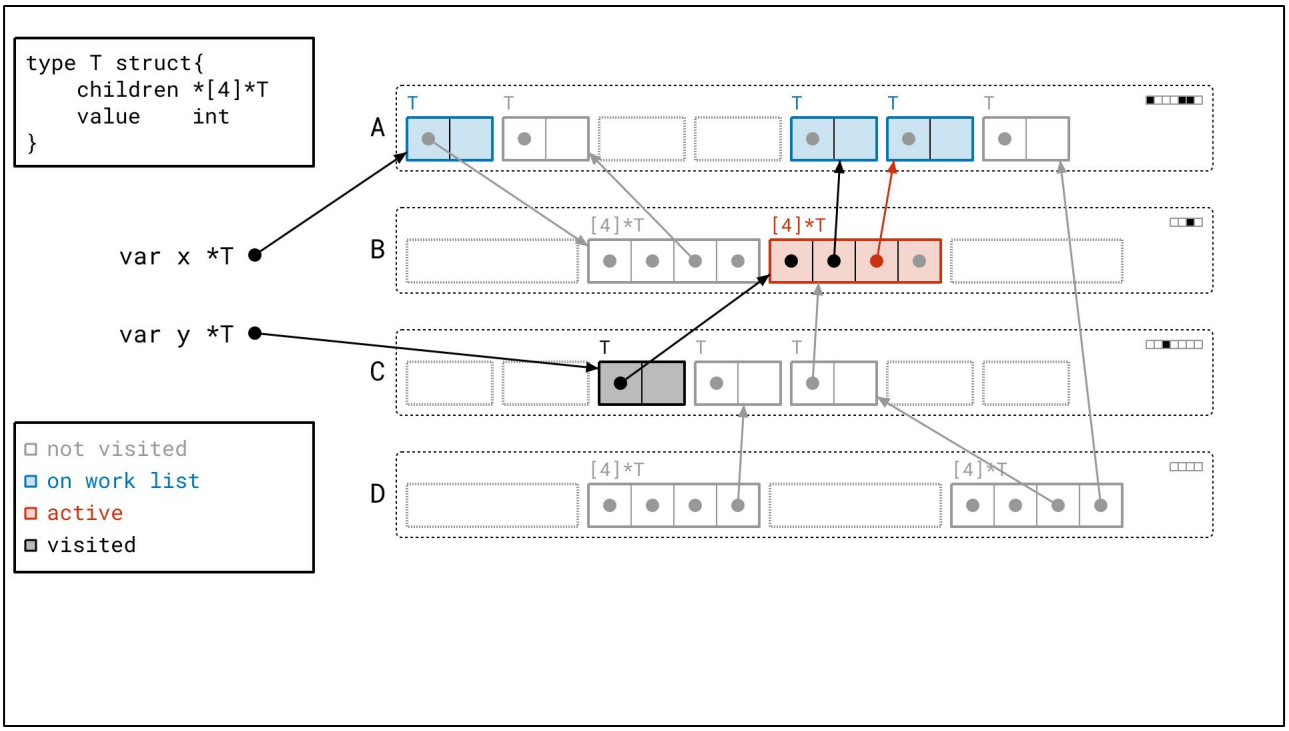
<next>


type T struct{
children *[4]*T
value int
}
var x *T
var y *T
A
B
C
D
T
[4]*T
not visited
on work list
active
visited

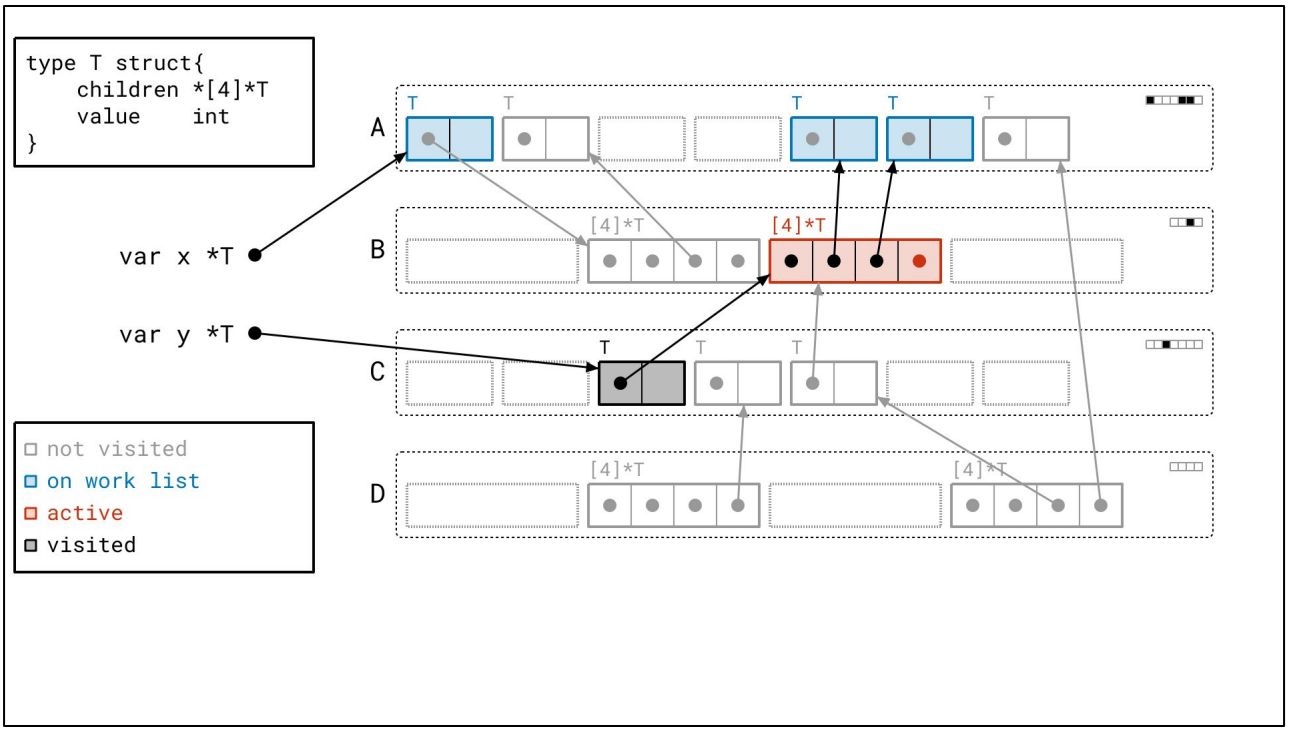
<next>



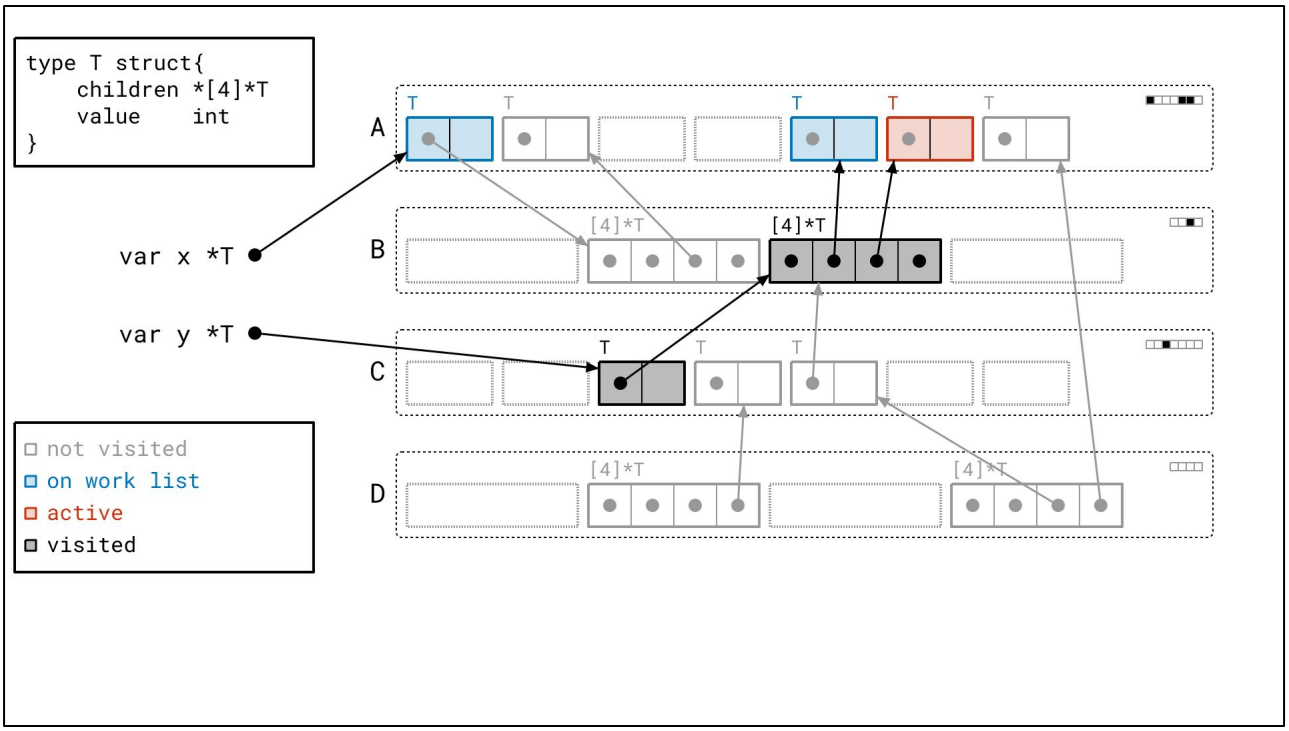
Find some more objects…


type T struct{
children *[4]*T
value int
}
var x *T
var y *T
A
B
C
D
T
[4]*T
not visited
on work list
active
visited

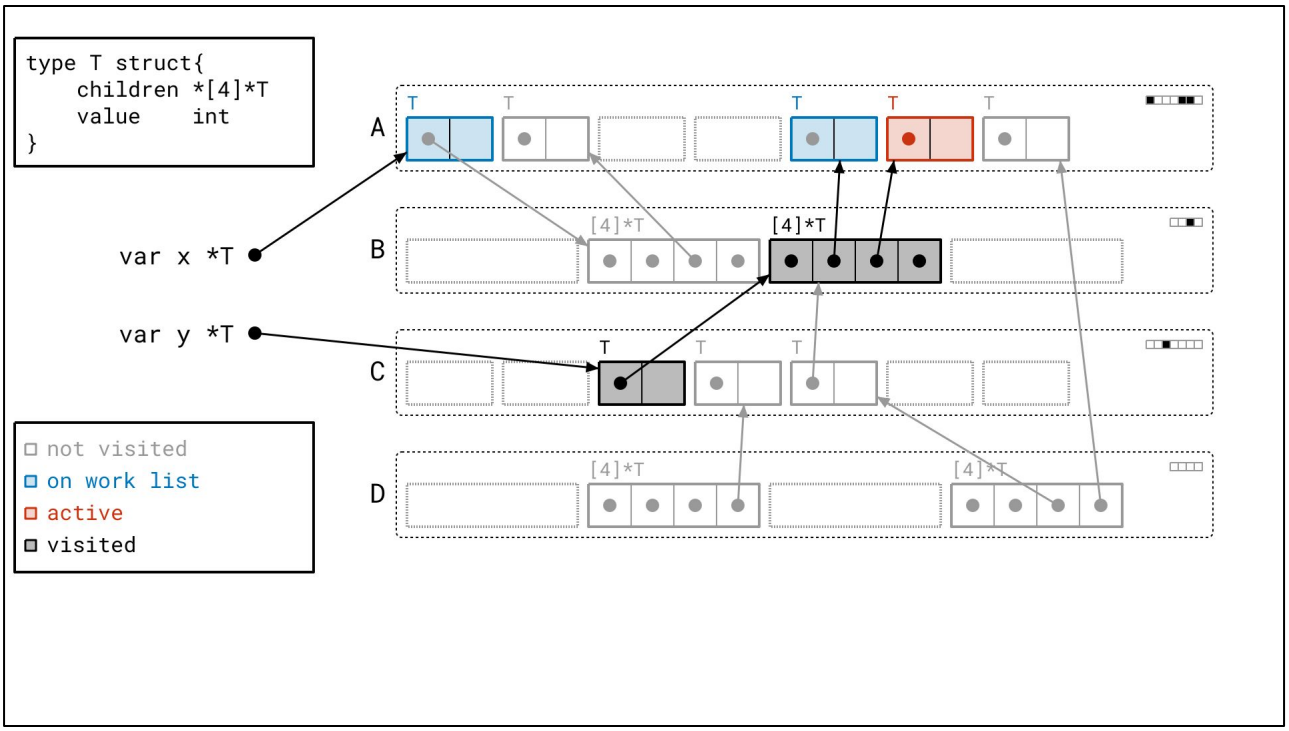
<next>


type T struct{
children *[4]*T
value int
}
var x *T
var y *T
A
B
C
D
T
[4]*T
not visited
on work list
active
visited

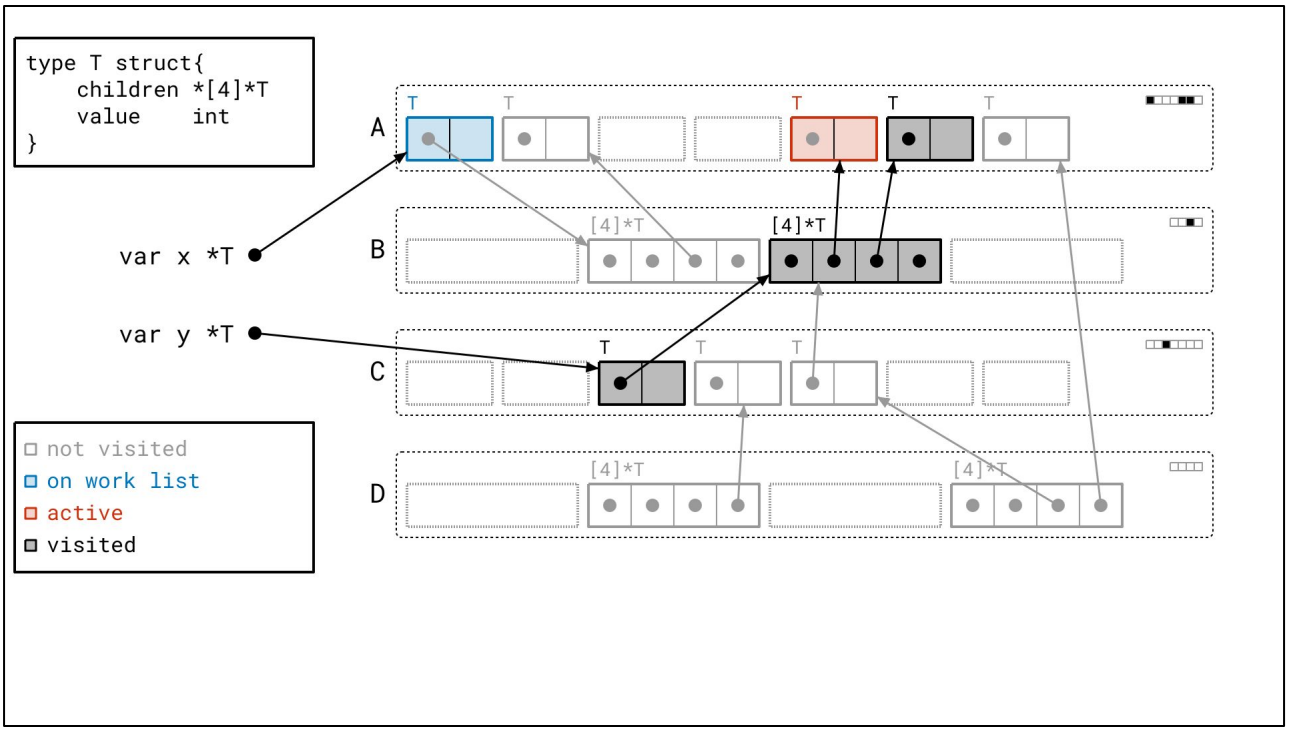
<next>



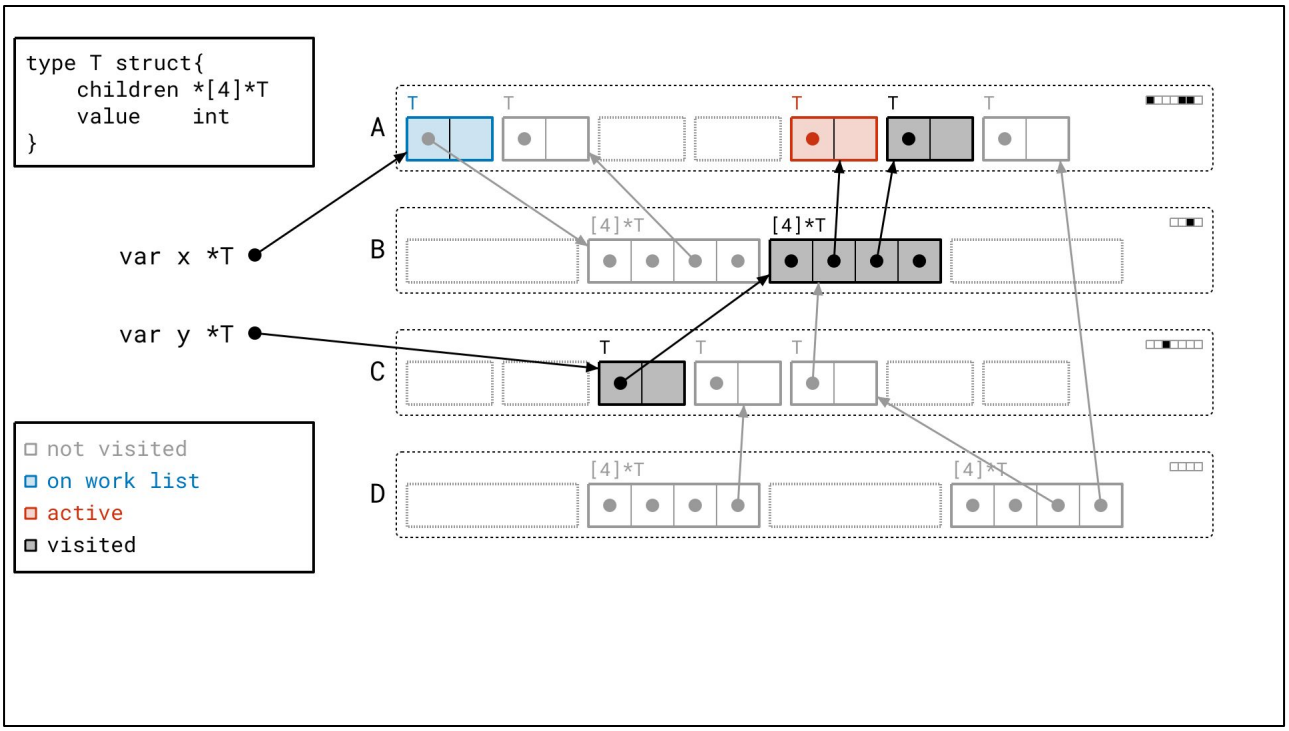
Then we walk the objects that the array object referred to!



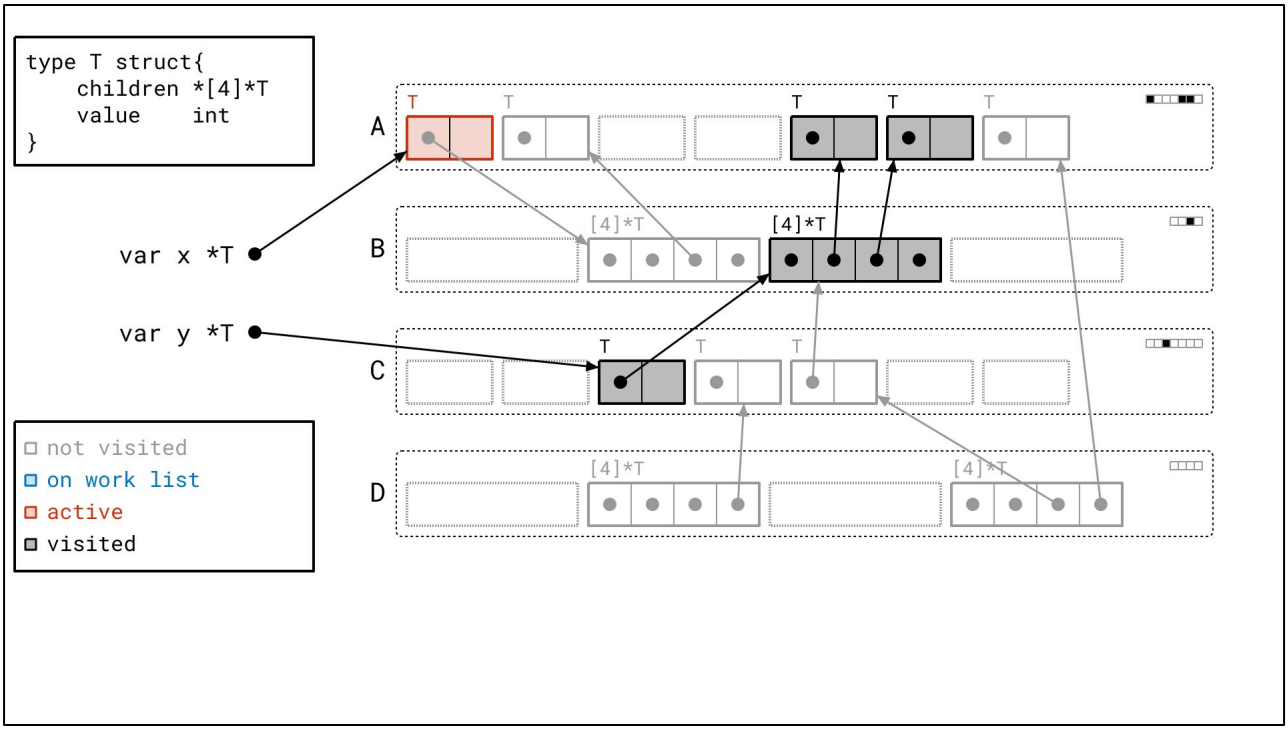
And note that we *still* have to walk over *all* pointers, even if they're nil. We don't know ahead of time if they will be.



One more object down this branch…

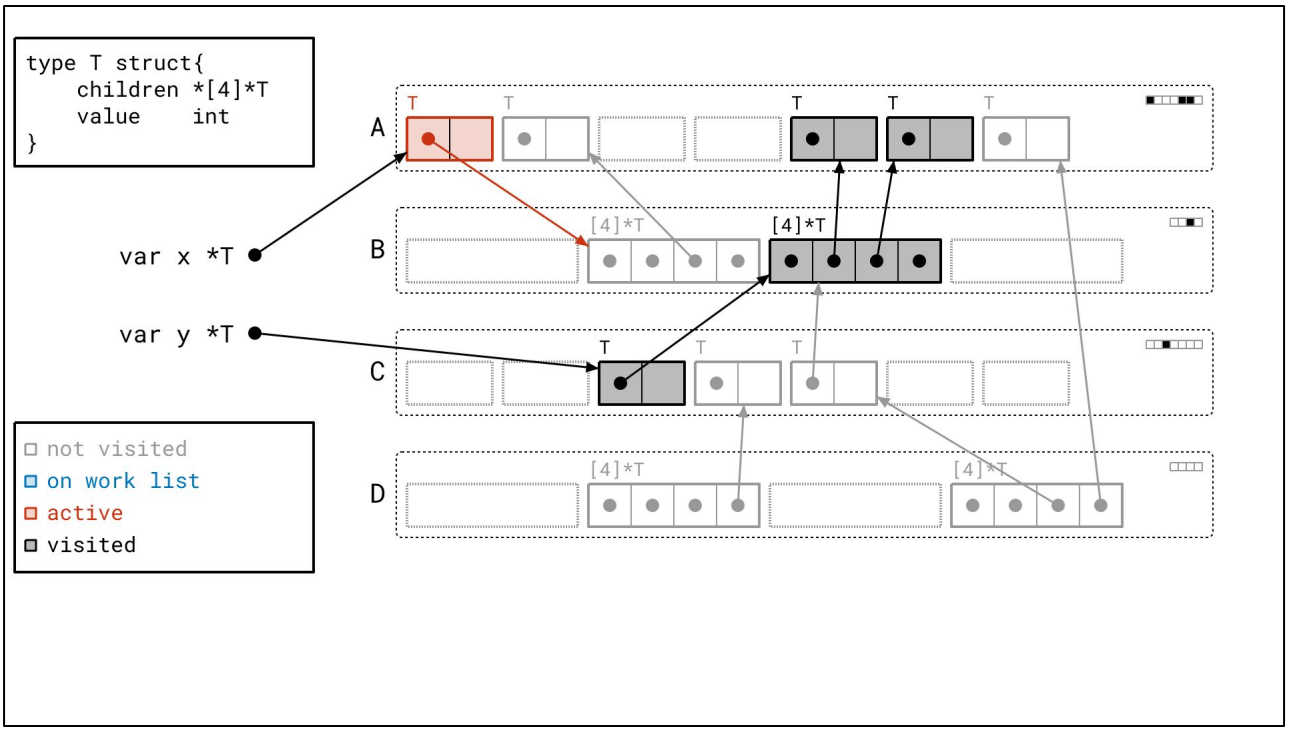

type T struct{
children *[4]*T
value int
}
var x *T
var y *T
A
B
C
D
T
[4]*T
not visited
on work list
active
visited

<next>

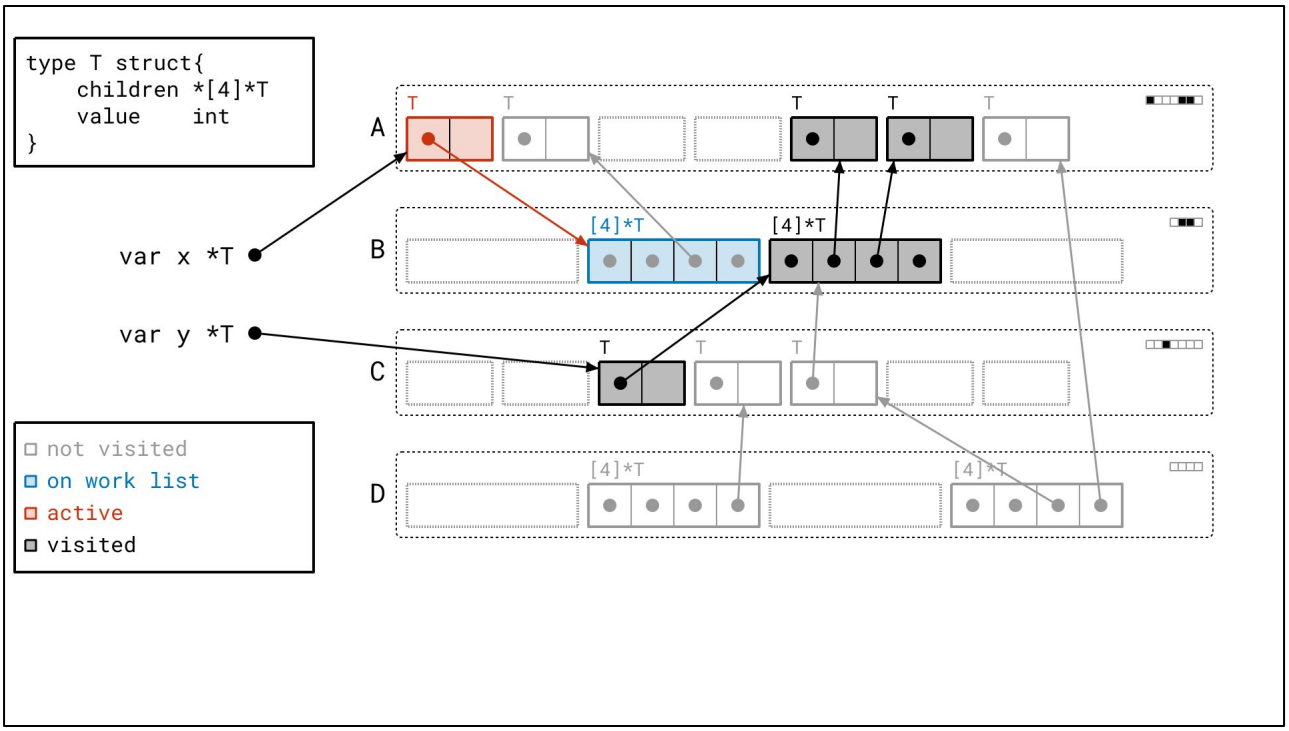


And now we've reached the other branch, starting from that object in page A we found much earlier from one of the roots.

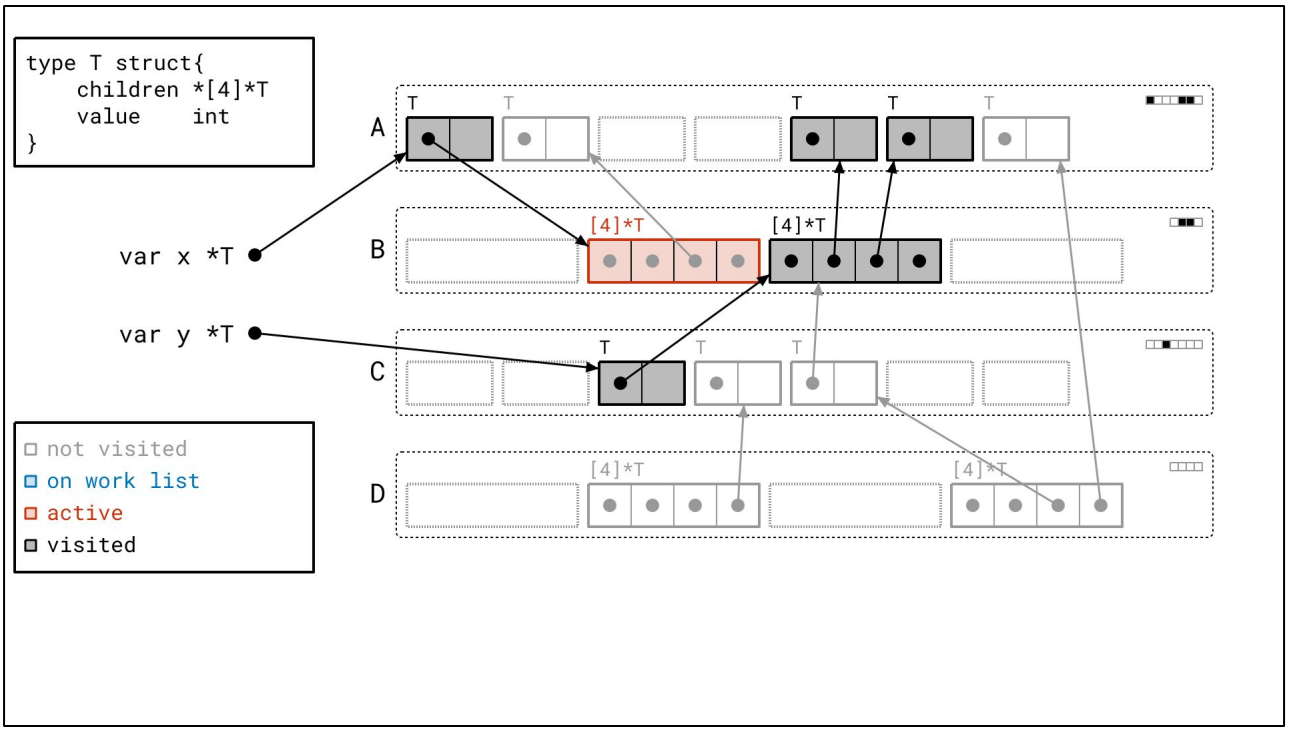
You may be noticing a last-in-first-out discipline for our work list here, indicating that our work list is a *stack*, and hence our graph flood is approximately *depth-first*. This is intentional, and reflects the actual graph flood algorithm in the Go runtime.



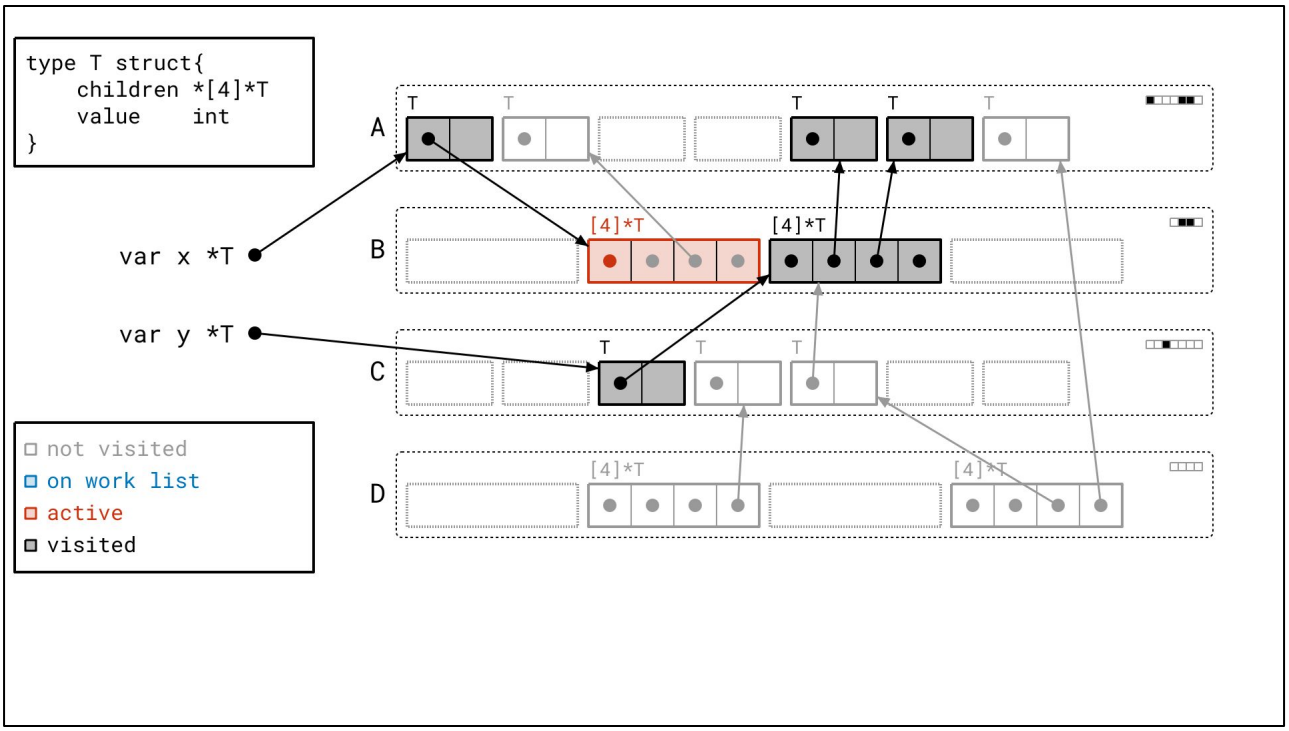
Let's keep going…



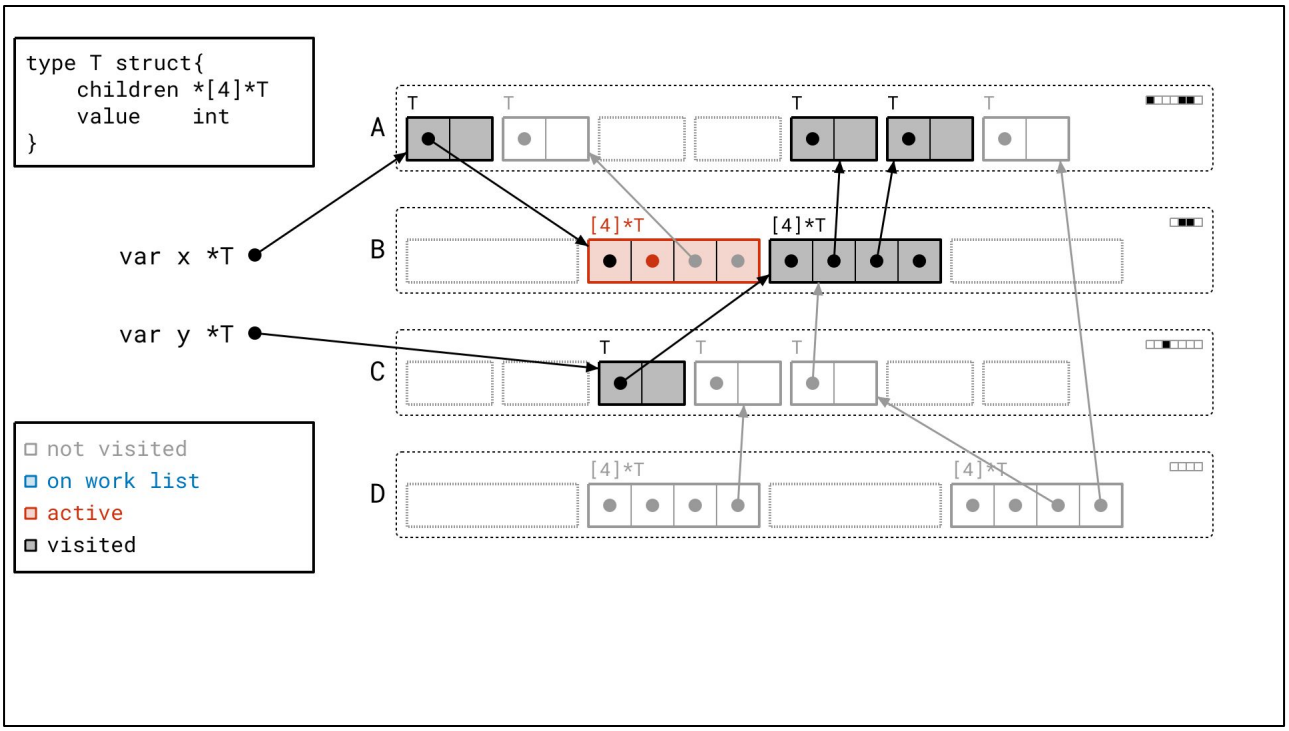
Next we find another array object…


type T struct{
children *[4]*T
value int
}
var x *T
var y *T
A
B
C
D
T
[4]*T
not visited
on work list
active
visited

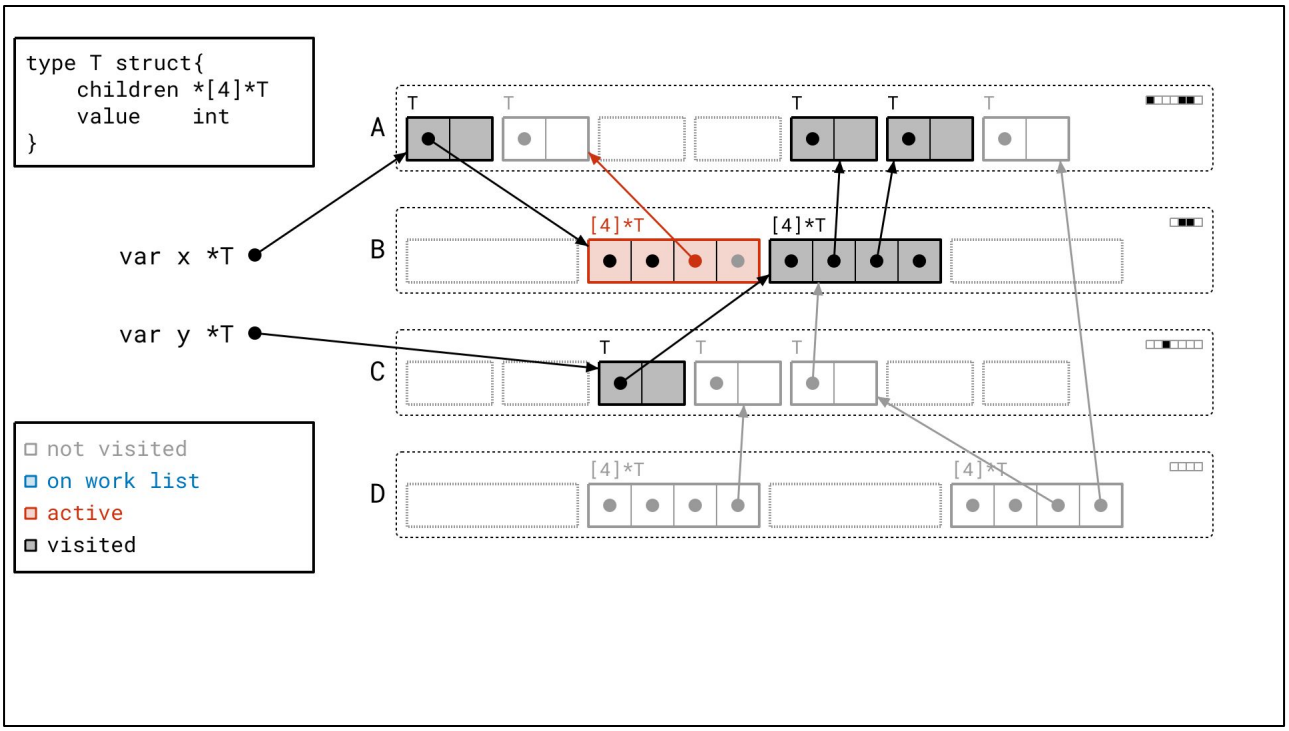
And walk it…


type T struct{
children *[4]*T
value int
}
var x *T
var y *T
A
B
C
D
T
[4]*T
not visited
on work list
active
visited

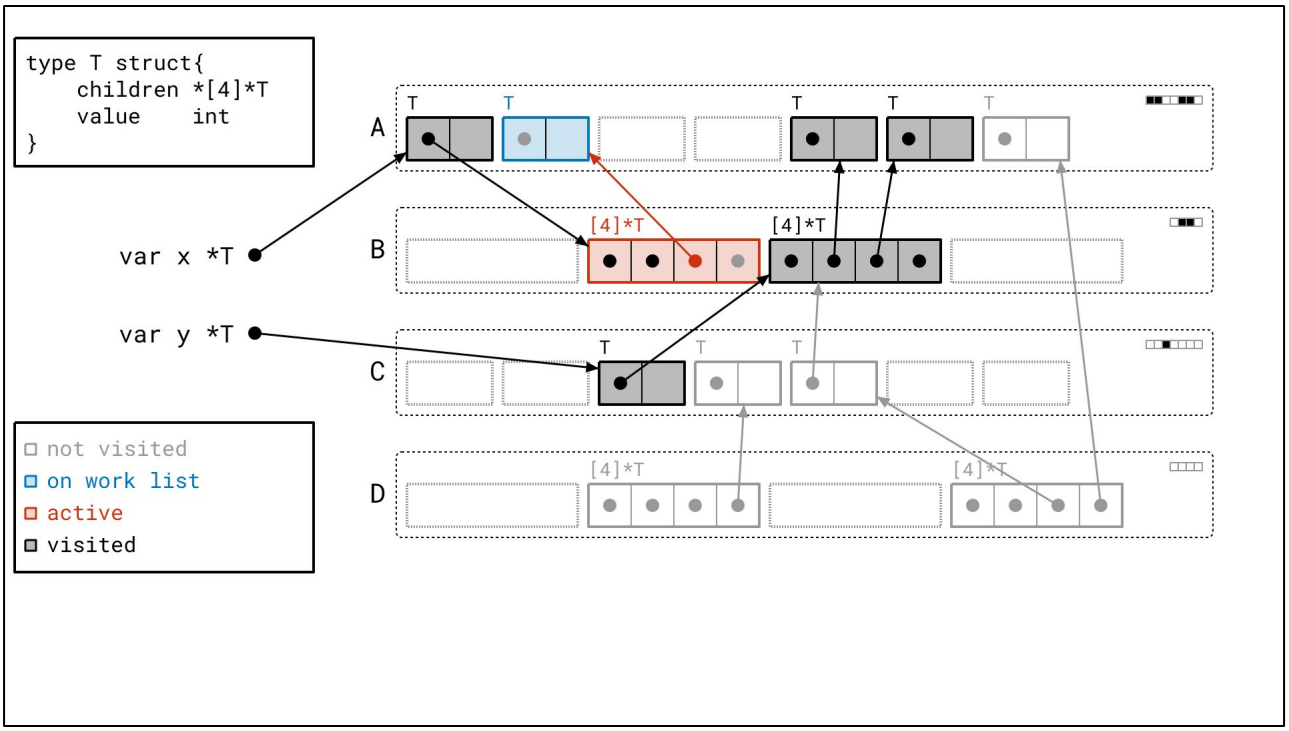
<next>


type T struct{
children *[4]*T
value int
}
var x *T
var y *T
A
B
C
D
T
[4]*T
not visited
on work list
active
visited

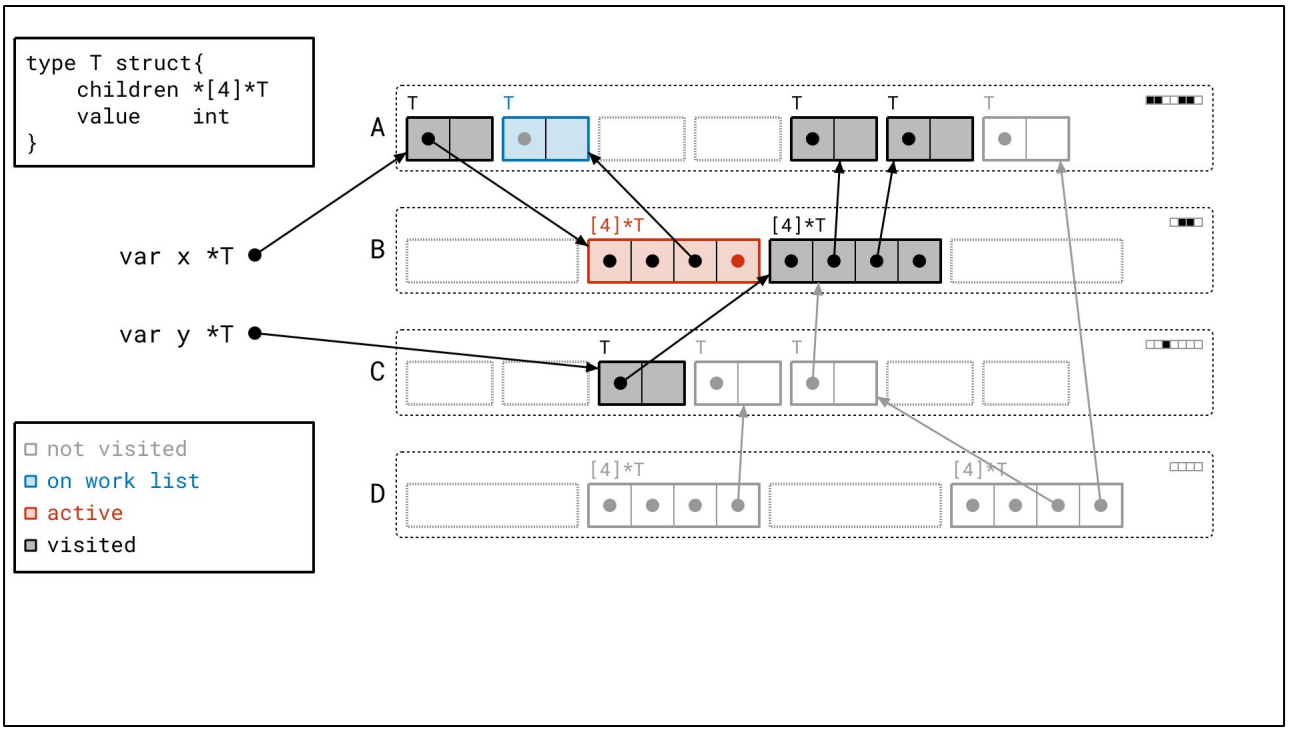
<next>


type T struct{
children *[4]*T
value int
}
var x *T
var y *T
A
B
C
D
T
[4]*T
not visited
on work list
active
visited

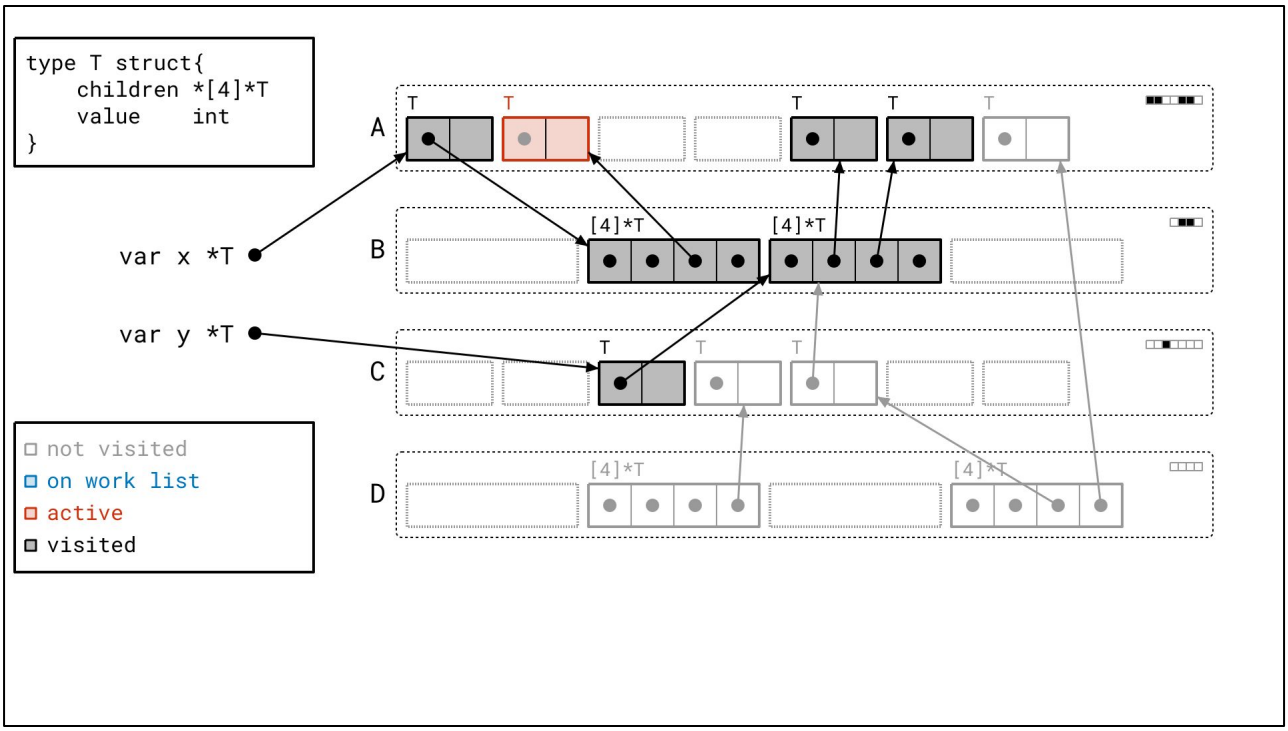
<next>


type T struct{
children *[4]*T
value int
}
var x *T
var y *T
A
B
C
D
T
[4]*T
not visited
on work list
active
visited

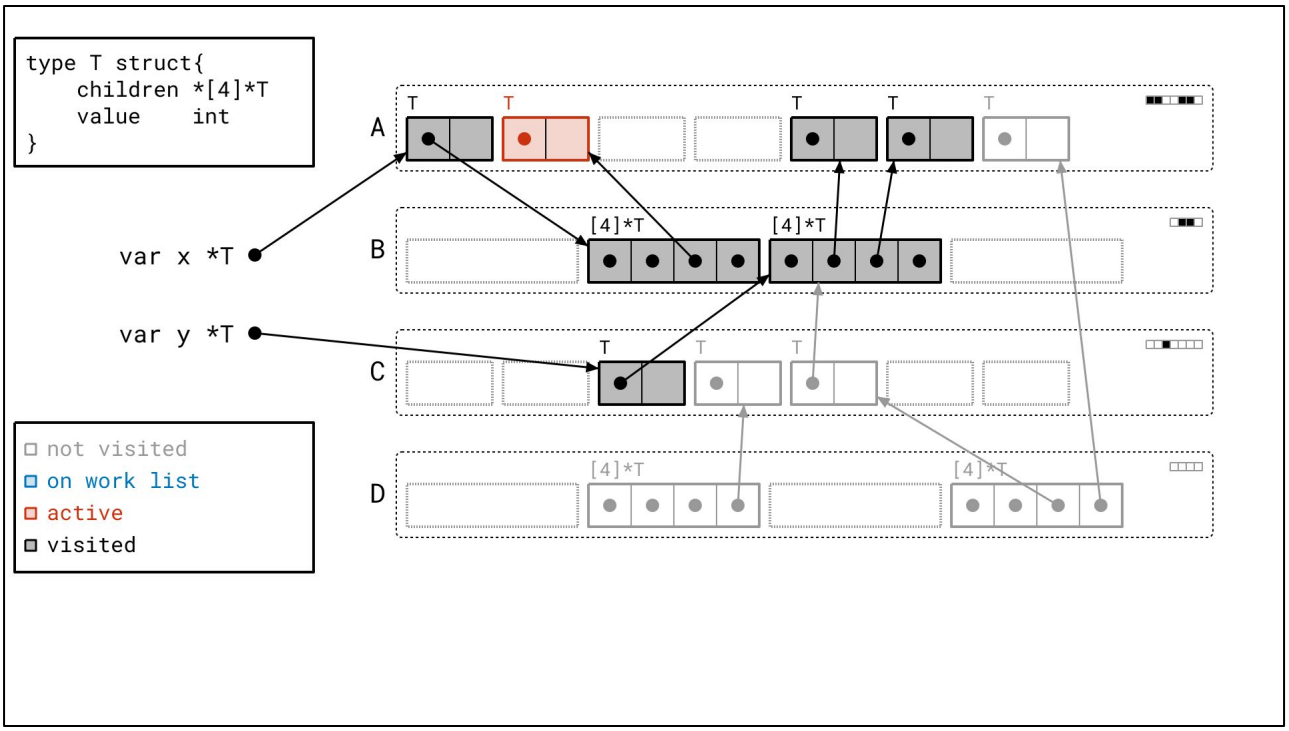
<next>



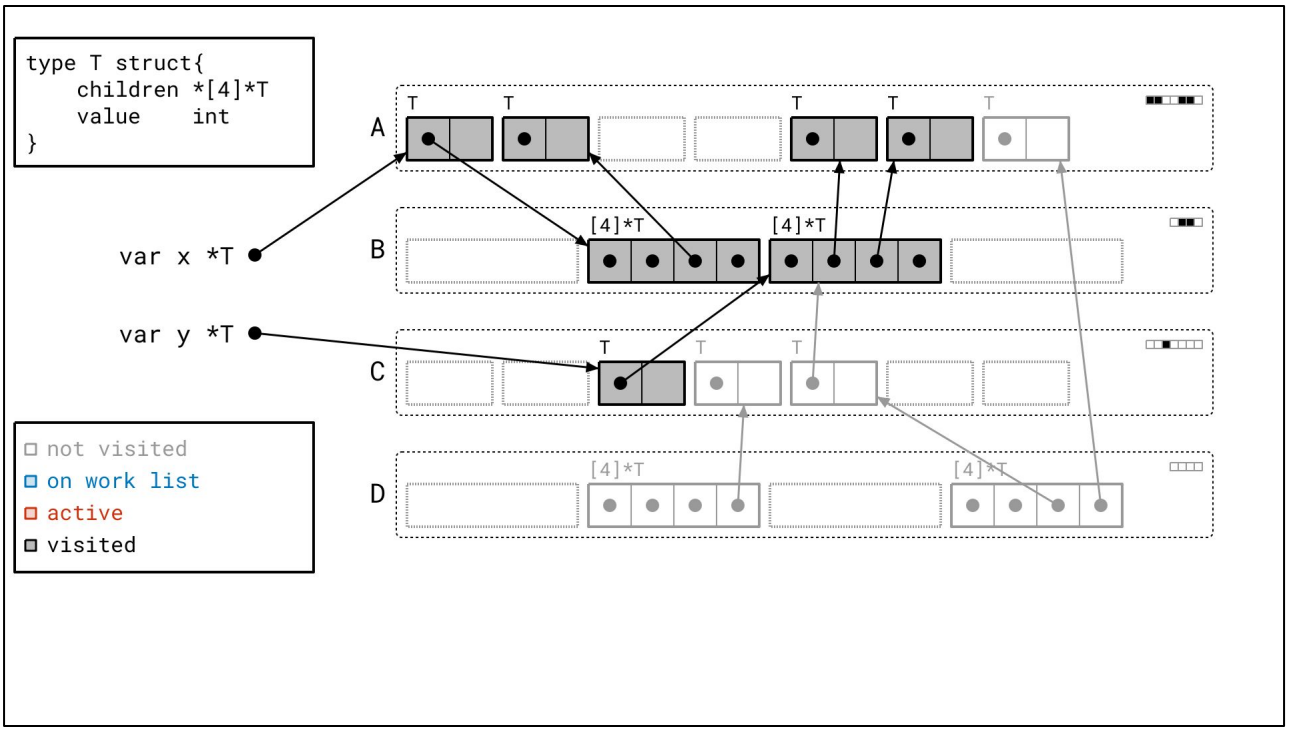
Just one more object left on our work list…


type T struct{
children *[4]*T
value int
}
var x *T
var y *T
A
B
C
D
T
[4]*T
not visited
on work list
active
visited

Let's scan it…

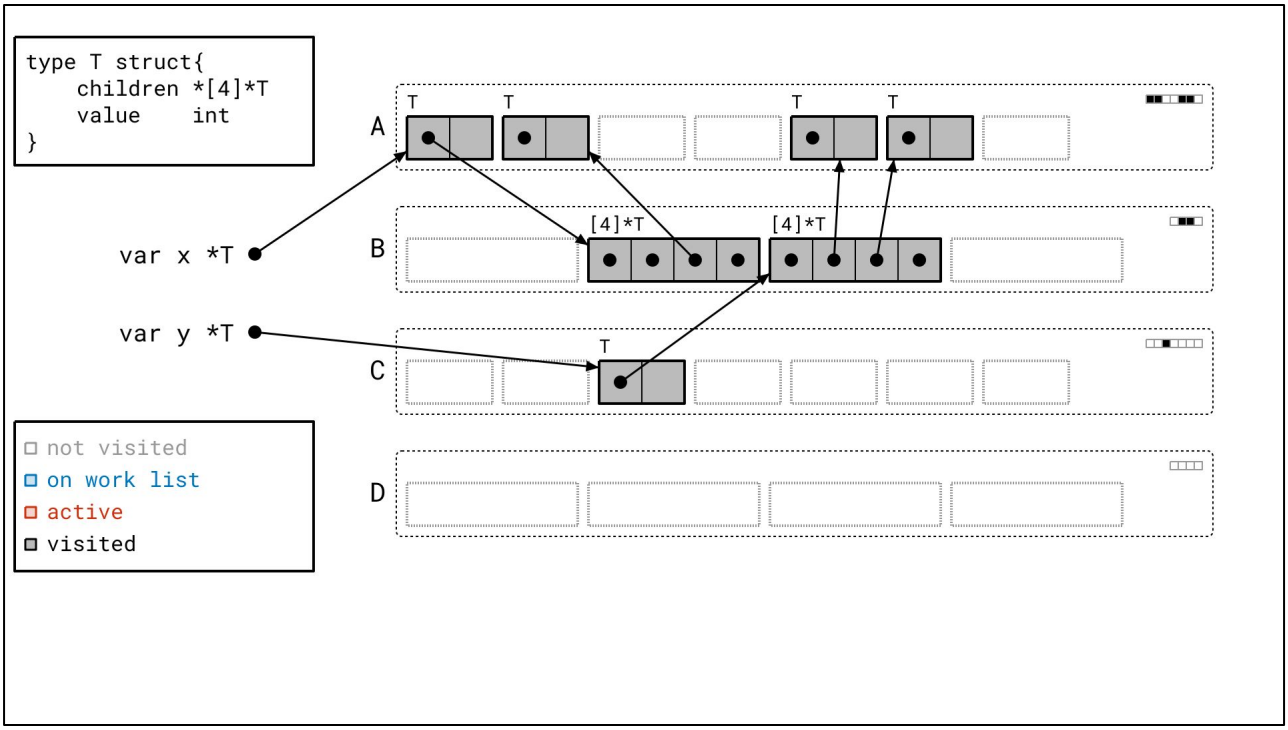

type T struct{
children *[4]*T
value int
}
var x *T
var y *T
A
B
C
D
T
[4]*T
not visited
on work list
active
visited

<next>



And we're done with the mark phase! There's nothing we're actively working on and there's nothing left on our work list. Every object drawn in black is reachable, and every object drawn in gray is unreachable.

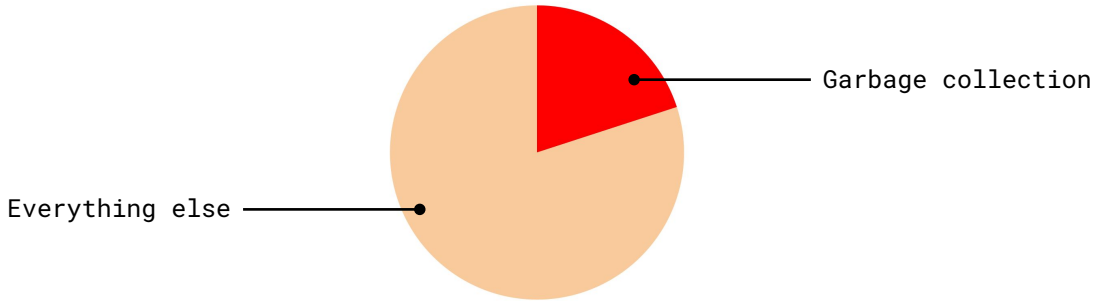
Let's sweep the unreachable objects, all in one go.



We've converted those objects into free slots, ready to hold new objects.

Problems

After all that, I think we have a handle on what the Go garbage collector is actually doing. Let's get on with the core of this talk. If you've tuned out the first part, pay attention again. Why does this need changing?

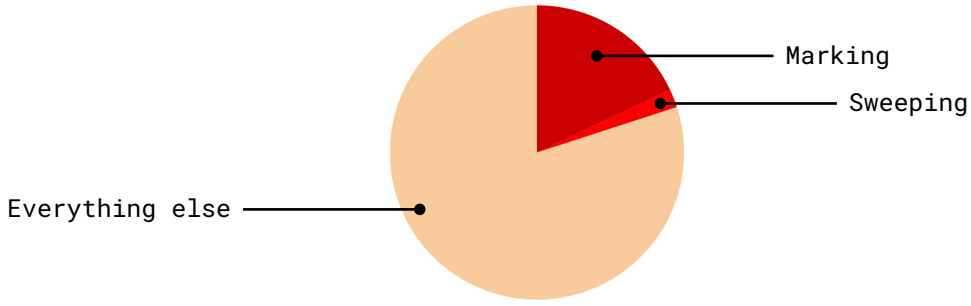


>20% CPU time in garbage collection in real programs.

Well, it turns out we can spend a lot of time executing this particular algorithm. I'm sure many of you have diagnosed programs spending a lot of time in the garbage collector.

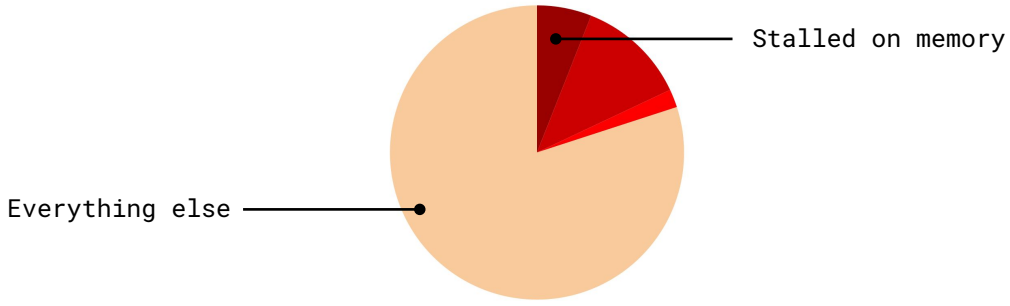
There are two parts to the cost of the garbage collector. The first is how often it runs, and the second is how much work it does each time it runs. Multiply those two together, and you get the total cost of the garbage collector.

For the first part, how often it runs, see my GopherCon EU talk from 2022 about Go memory limits. For this talk, we'll focus on the second part – what it's doing.



~90% garbage collection CPU time in marking.

And what it's doing is spending a most of its time on marking and scanning objects in the mark phase. That's why I didn't even bother explaining the sweep phase too much. In practice, sweeping is very cheap, since we designed it to be that way. (It's also much easier to design it that way!)

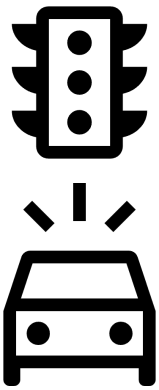


>35% marking CPU time stalled on a memory access.

And at the heart of why marking is so expensive is that it's just not a very good fit for modern CPU microarchitecture.

At least 35% of time spent marking is just waiting on the CPU to fetch memory. On it's own that's already bad, but it totally gums up the works on what makes modern CPUs actually fast.

A driving analogy



The CPU wants to go fast; it wants to be on the ***highway***.

To do so, it must ***see ahead for a long distance***.

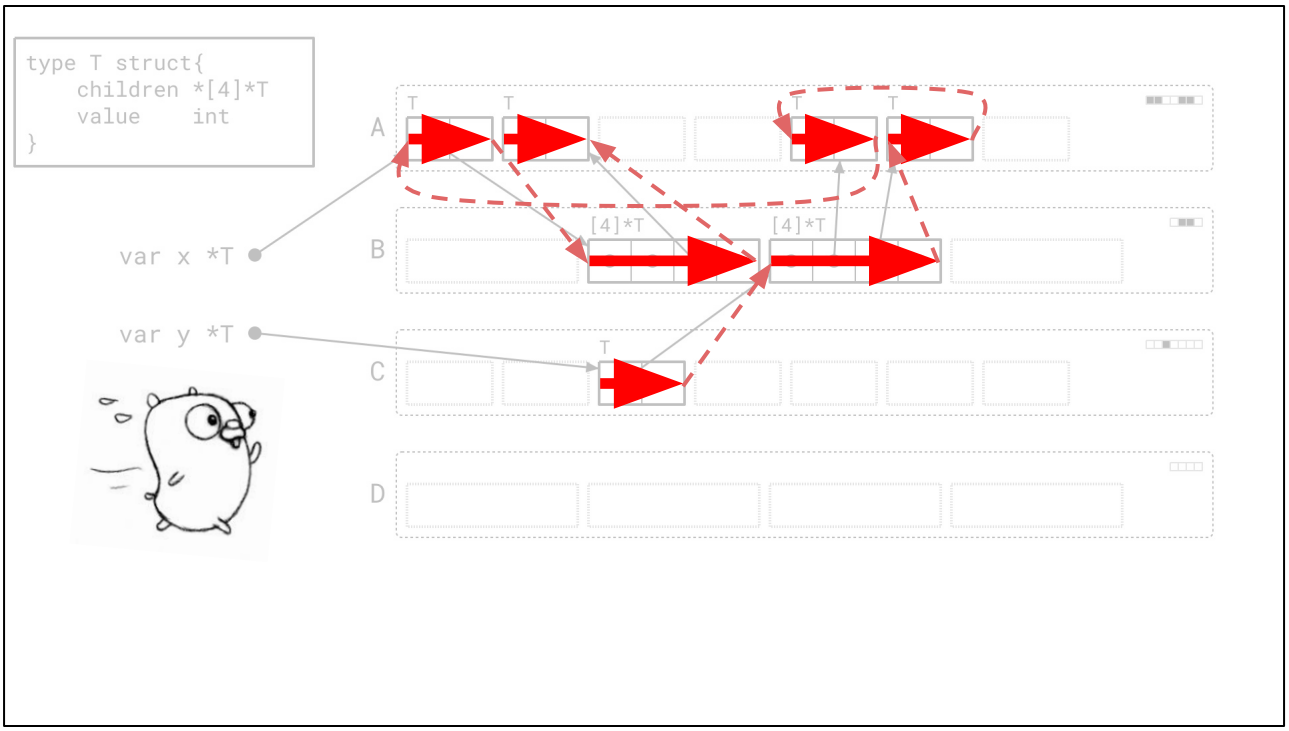
But the **graph flood** is like ***city streets***.

The CPU ***can't see far ahead*****.**

The CPU ***constantly has to make starts and stops***.

What do I mean by "gum up the works"? Well, the specifics of modern CPUs can get pretty complicated, so let's use an analogy.

Imagine the CPU driving down a road, where that road is your program. The CPU wants to ramp up to a high speed, and to do that it needs to be able to see far ahead of it, and the way needs to be clear. But the graph flood algorithm is like driving through city streets for the CPU. The CPU can't see around corners and it can't predict what's going to happen next. To make progress, it constantly has to slow down to make turns, stop at traffic lights, and avoid pedestrians. It hardly matters how fast your engine is because you never get a chance to get going.



And let's make that more concrete by looking at our favorite diagram again. I've overlaid the heap here with the path that we took. Each left-to-right arrow represents a piece of scanning work that we did.

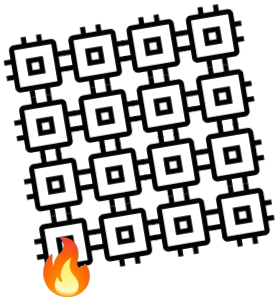
What I want you notice is that we were jumping all over memory doing tiny bits of work in each place. In particular, we're frequently jumping between pages, and between different parts of pages.

Modern CPUs to a lot of caching. Going to main memory can be up to 100x slower than accessing memory that's in our cache. CPU caches are populated with memory that's been recently accessed, and memory that's nearby to recently accessed memory.

But there's no guarantee that any two objects that point to each other will *also* be close to each other in memory. The graph flood doesn't take this into account.

Quick side note: if we were *just* doing slow memory accesses it might not be so bad. CPUs issue memory requests asynchronously, so even slow ones could overlap if the CPU could see far enough ahead. But in the graph flood, every bit of work is small and highly dependent on the last, so the CPU is forced to wait on nearly each individual memory fetch.

It's only getting worse...



Wait two years and your code will get **slower**.

> ***Non-uniform memory access*** *is becoming more common.*
>
> ***Memory bandwidth*** *is becoming more limited.*
>
> *Work list must* ***scale to more CPU cores****.*
>
> *Hard to use* ***vector hardware*** *in the garbage collector.*

And unfortunately for us, this problem is only trending worse. There's an adage in the industry of "wait two years and your code will get faster."

But Go, as a garbage collected language, relying on the mark-sweep algorithm, risks the opposite. "Wait two years and your code will get *slower*." New hardware is adding challenges to garbage collector performance.

- For one, memory now tends to be associated subsets of CPU cores. Accesses by other CPU cores to that memory are *slower than before*.
  - In other words, the cost of a main memory access now depends on which CPU core is accessing it. It's non-uniform, so we call this non-uniform memory access or NUMA. The graph flood doesn't take this into account at all.
- Memory bandwidth is becoming more constrained.
  - This just means that while we have more CPU cores, each core can only submit relatively fewer requests to main memory, forcing non-cached requests to wait longer than before.
- More CPU cores means our work list data structure needs to scale gracefully to more CPU cores.
  - Remember, this work is happening in parallel.
- New hardware has fancy features like vector instructions, which let us operate on a lot of data at once.
  - This has the potential for big speedups, it's really unclear how to make that work for a marking.

Green Tea 🍵

Finally, this brings us to Green Tea: our new approach to the mark-sweep algorithm. The key idea behind Green Tea is astonishingly simple.

Pages, not objects.

Work with pages, not objects.

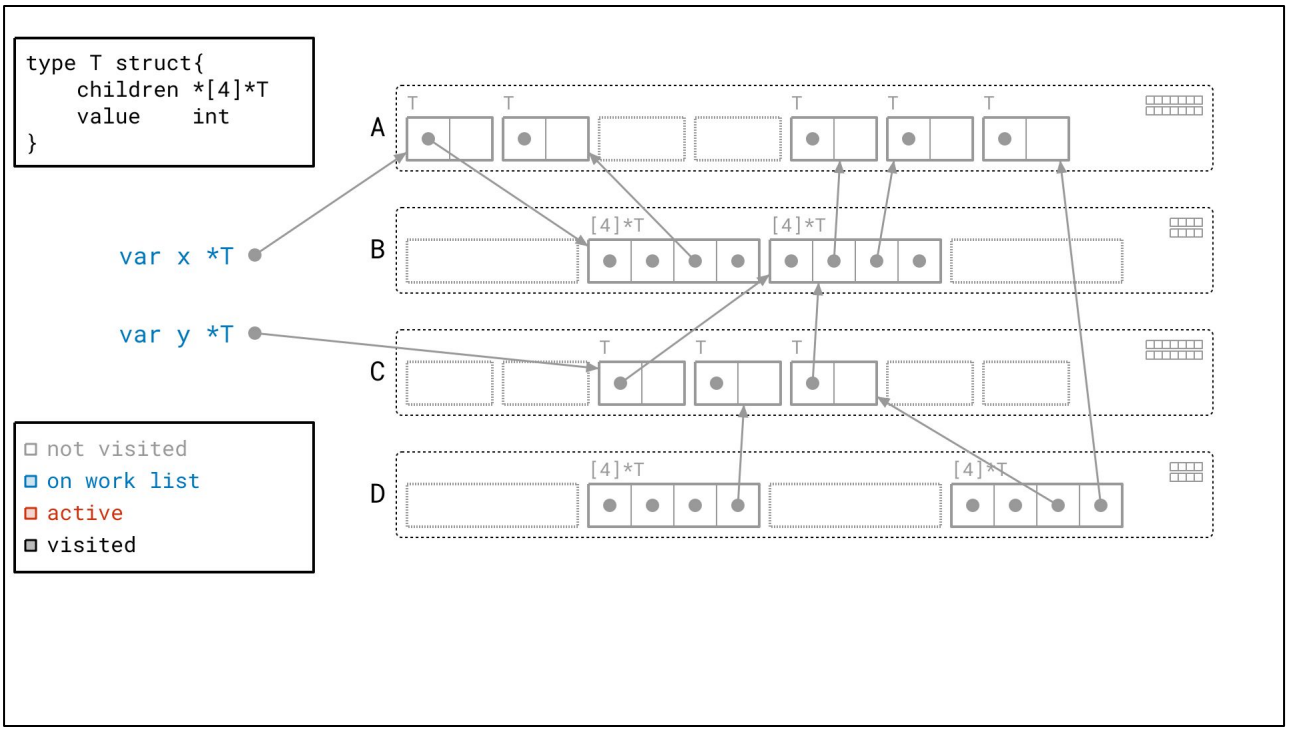
Sounds trivial, right? And yet, it took us years to figure out how to order the object graph walk and what we needed to track to make this work well in practice.

This means:

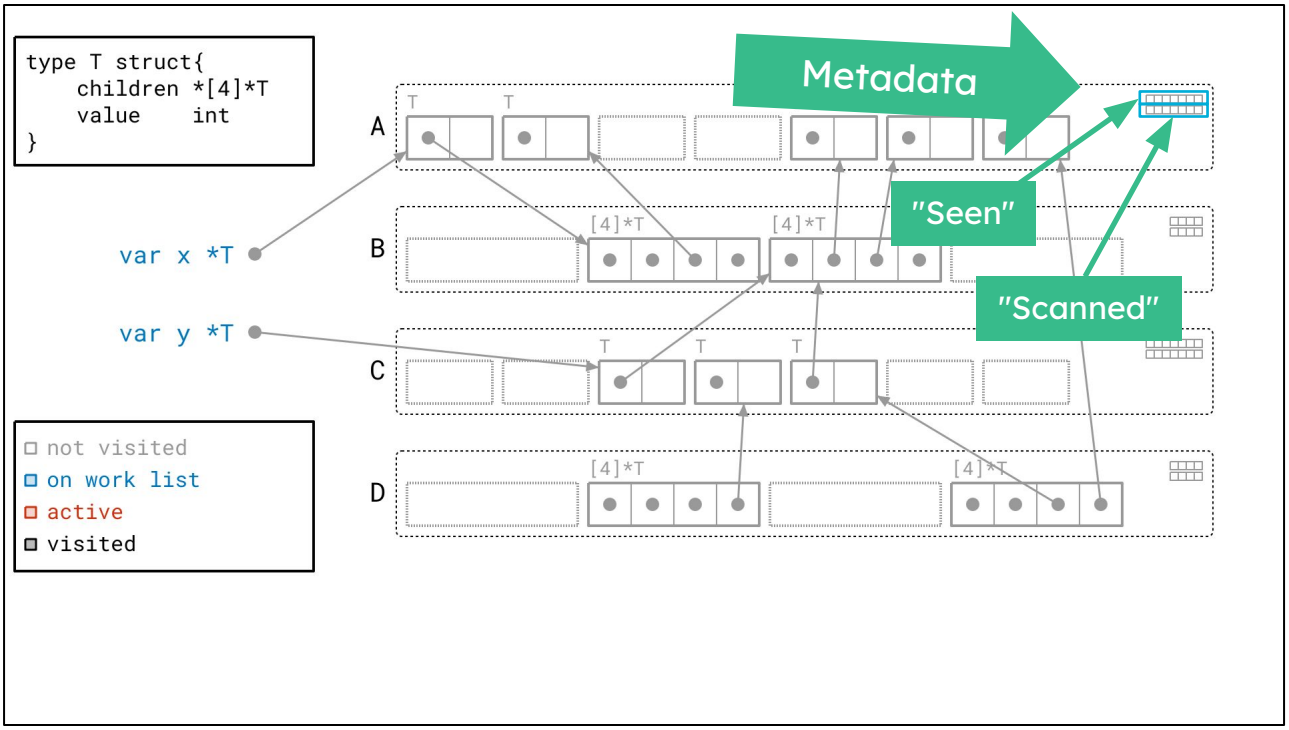
1. Instead of scanning objects we scan whole pages.
2. Instead of tracking objects on our work list, we track pages.
3. We still need to mark objects at the end of the day, but we'll track that locally to each page, rather than across the whole heap.

Green Tea example

Let me show you what I mean.

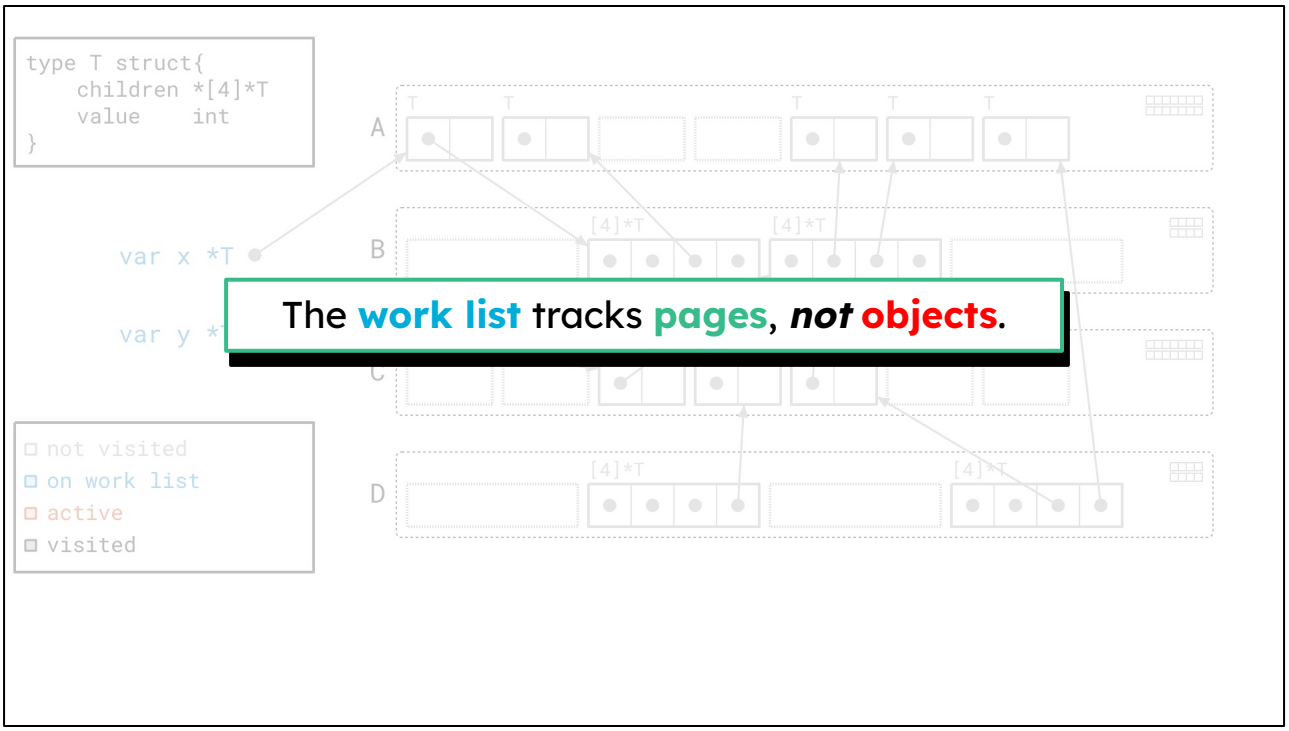


Back to our favorite diagram, we've got just one difference from before.



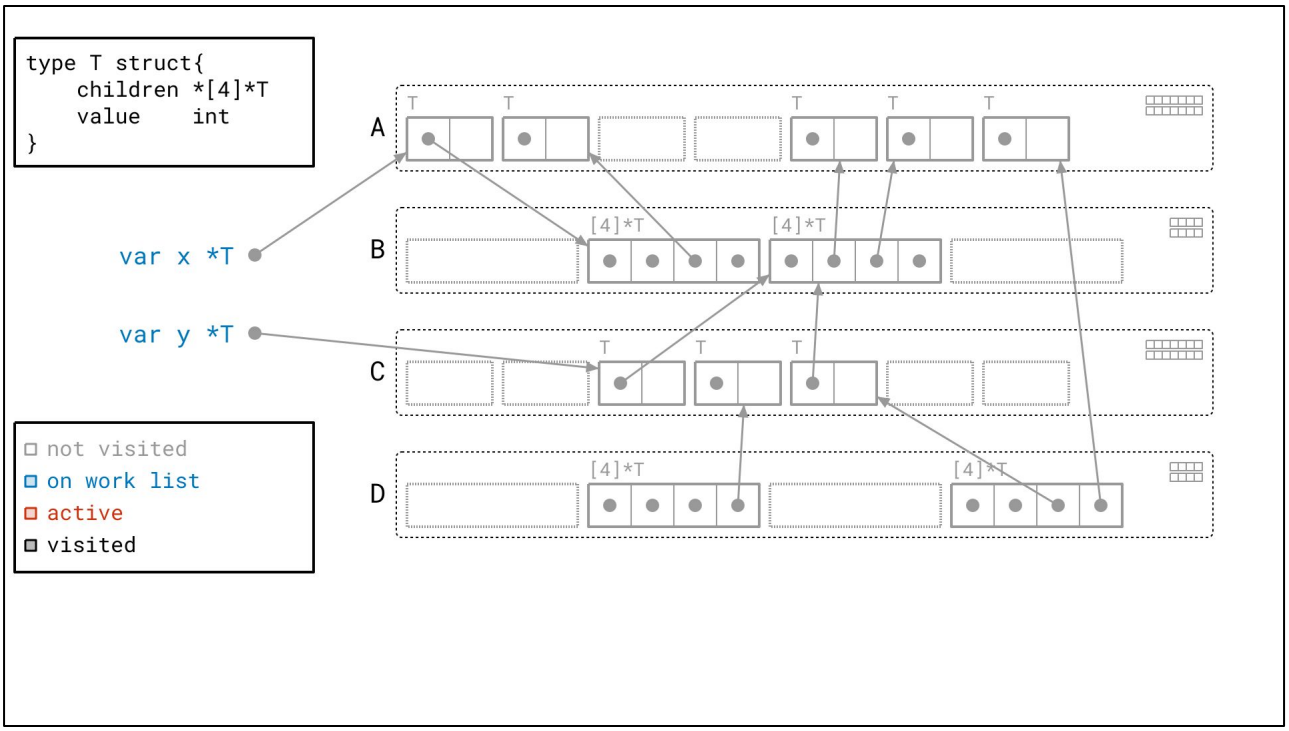
And it's that we now have *two* bits of metadata per page. Again, each bit, or box, corresponds to one of the object slots in the page. In total, we now have fourteen bits that correspond to the seven slots in page A. The top bits represent the same thing as before: whether we've *seen* the object or not. I'll call these the "seen" bits.

The bottom set of bits are new. They're going to tell us whether we've already *scanned* the object or not. This new piece of metadata is going to come in handy because…

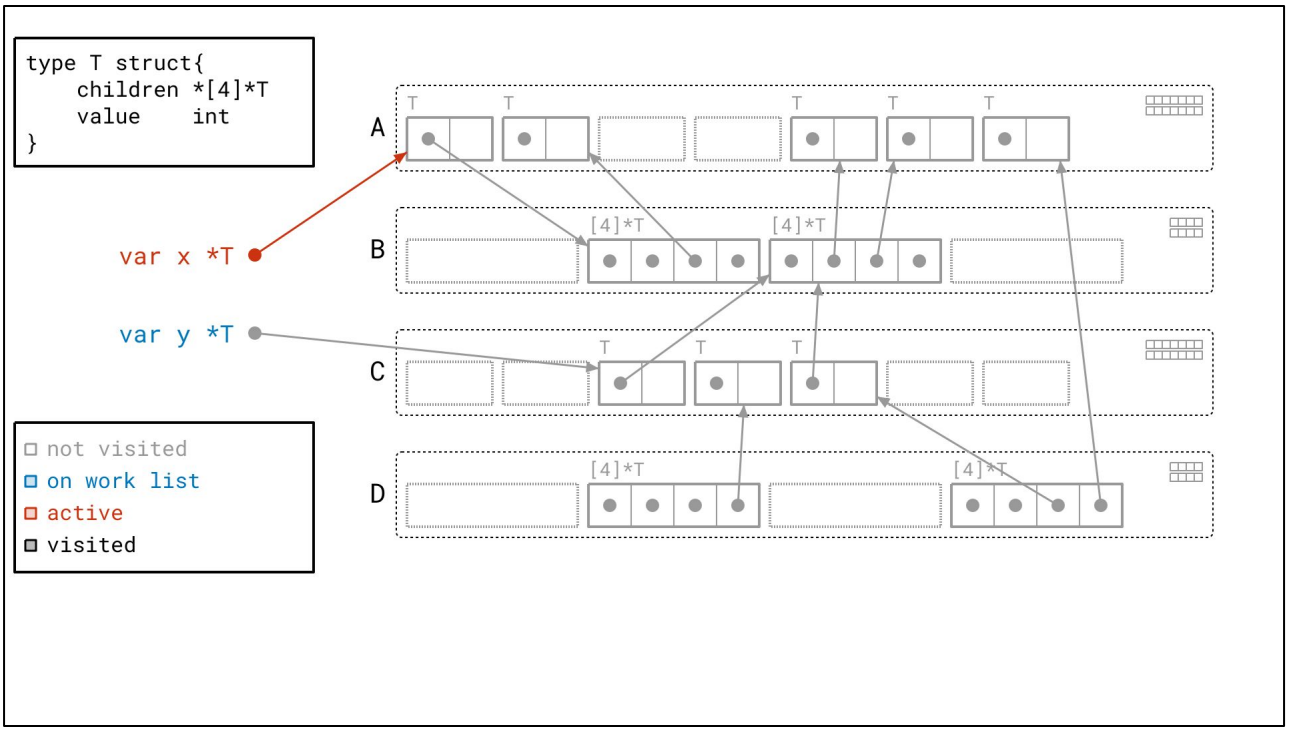


… again, we're going to run our algorithm this time by tracking *pages* on the work list, and not *objects*. That extra bit of metadata per object per page will help us identify what we actually need to look at in the page.

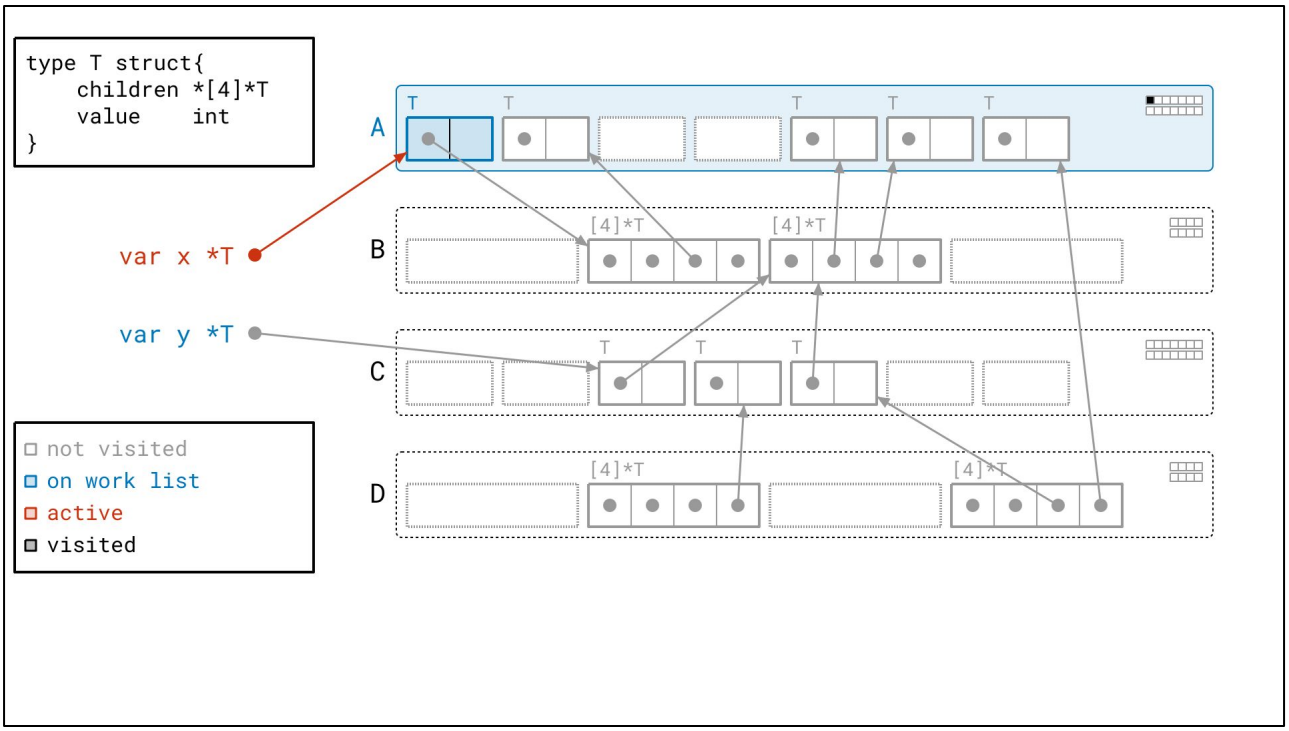
Without further adieu, let's begin the mark phase.



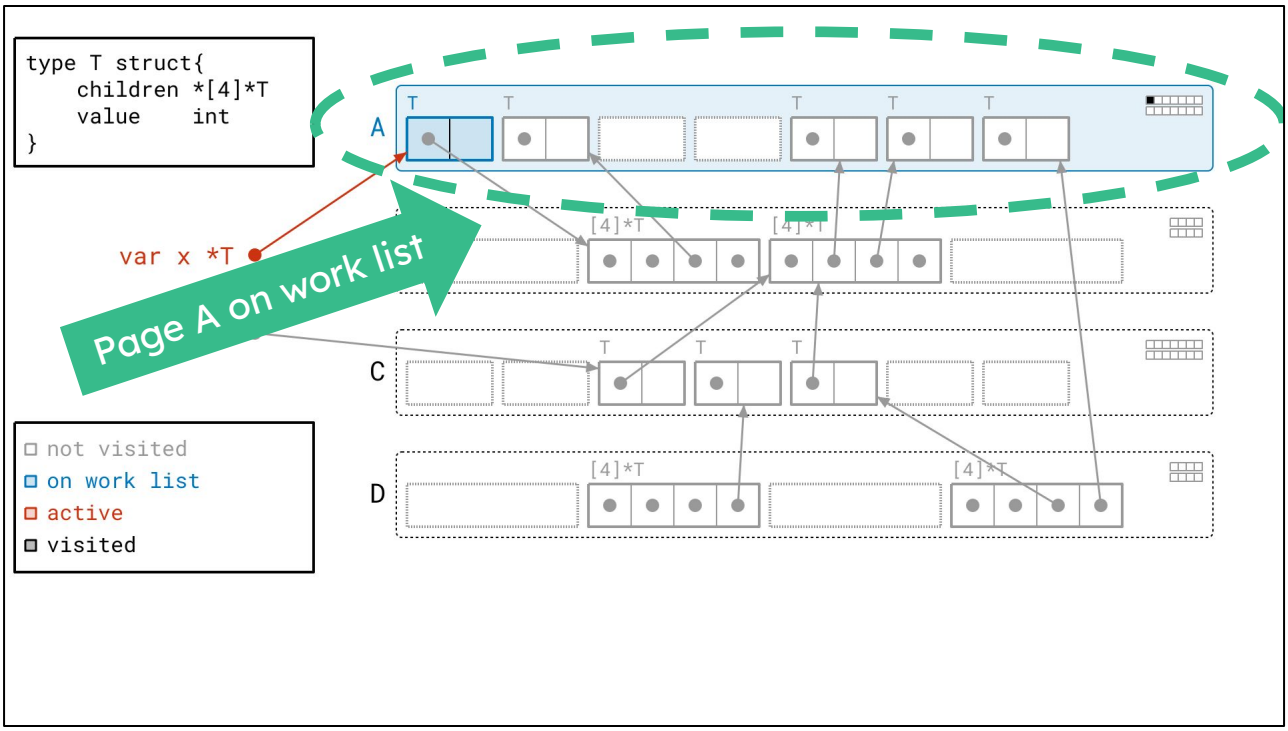
We start off the same as before, walking objects from the roots.


type T struct{
children *[4]*T
value int
}
var x *T
var y *T
not visited
on work list
active
visited
A
B
C
D
T
[4]*T

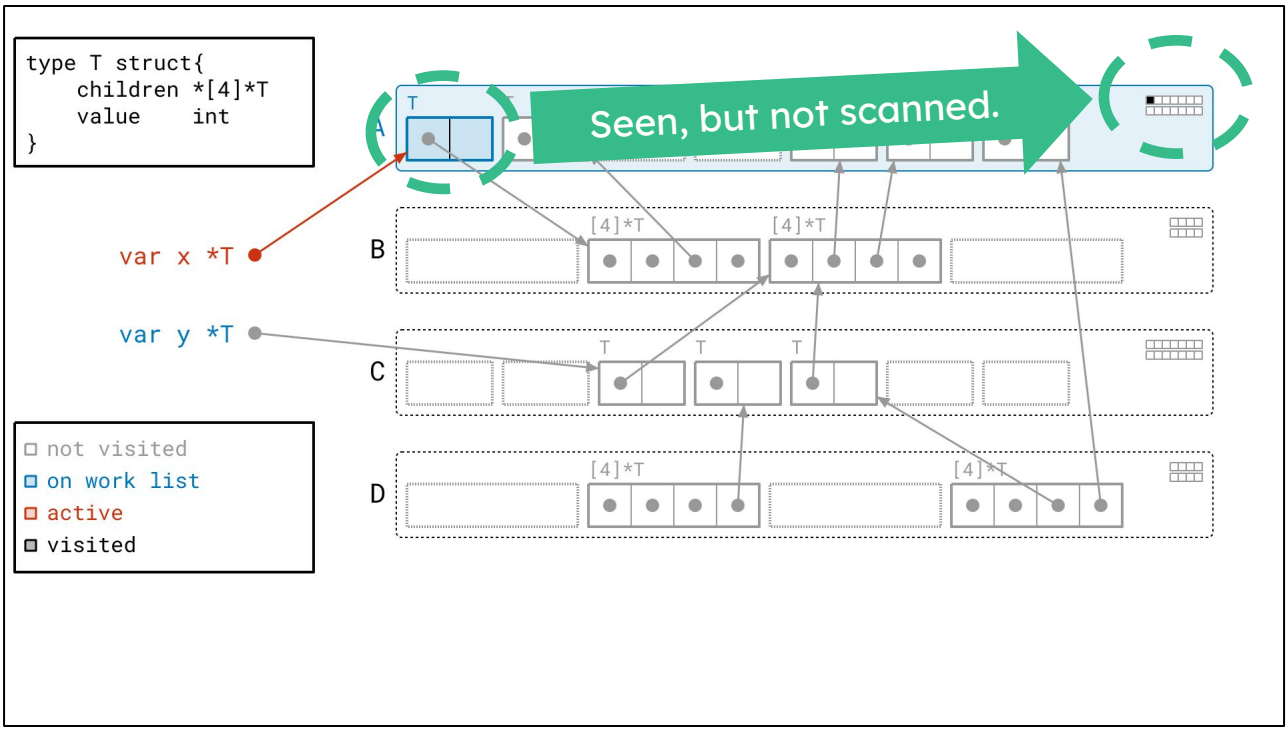
<next>



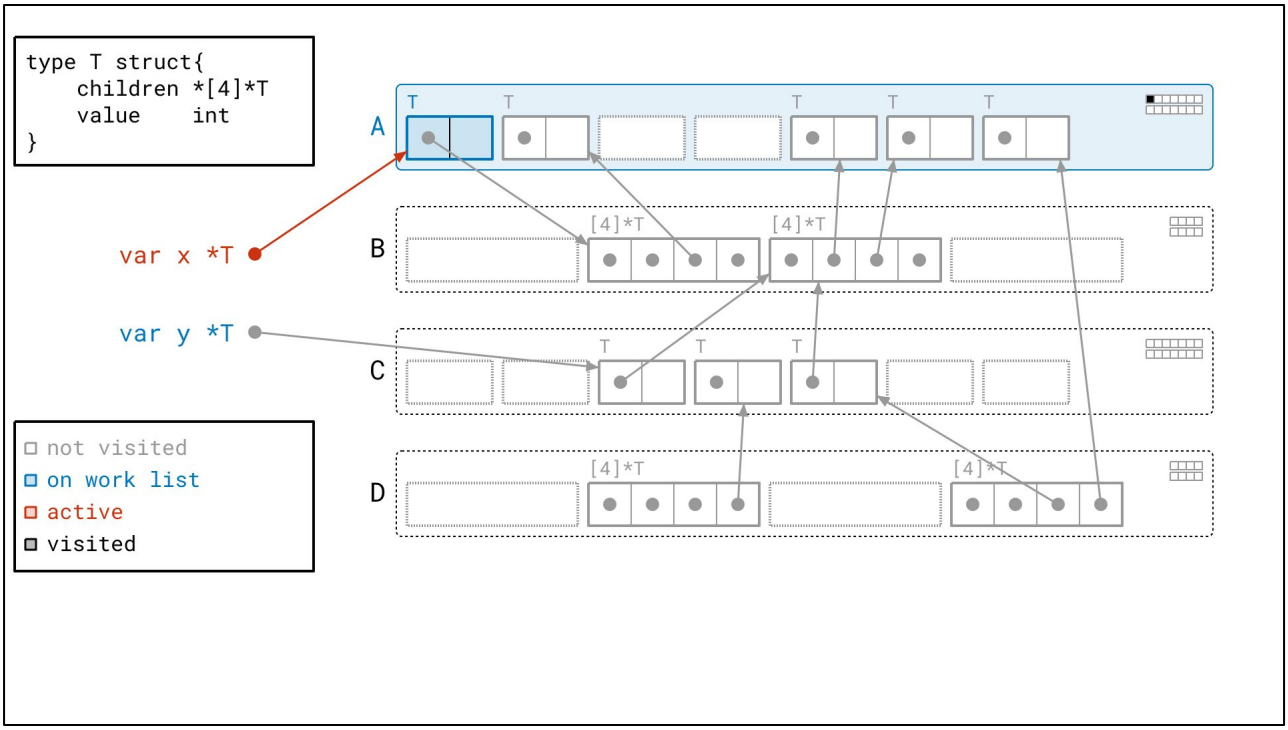
But this time, we put the *whole page* on our work list…



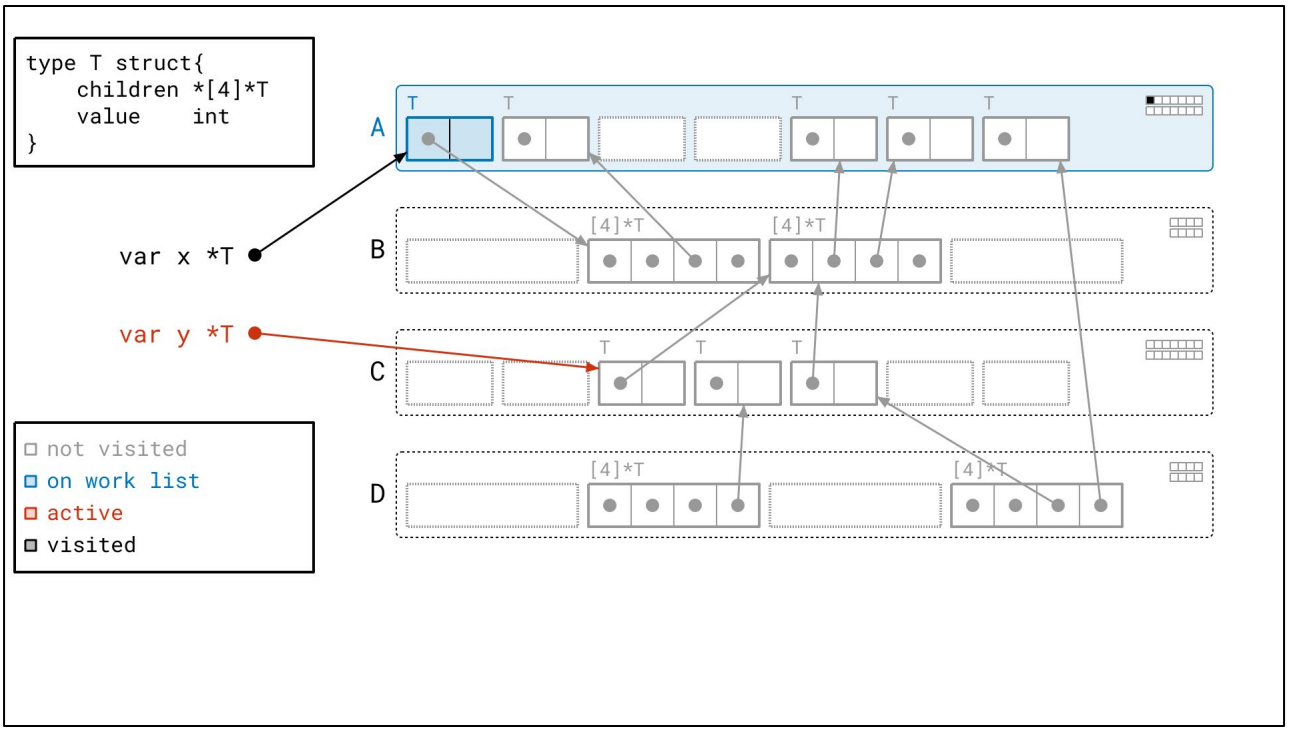
In this case, page A, hence shaded in blue.



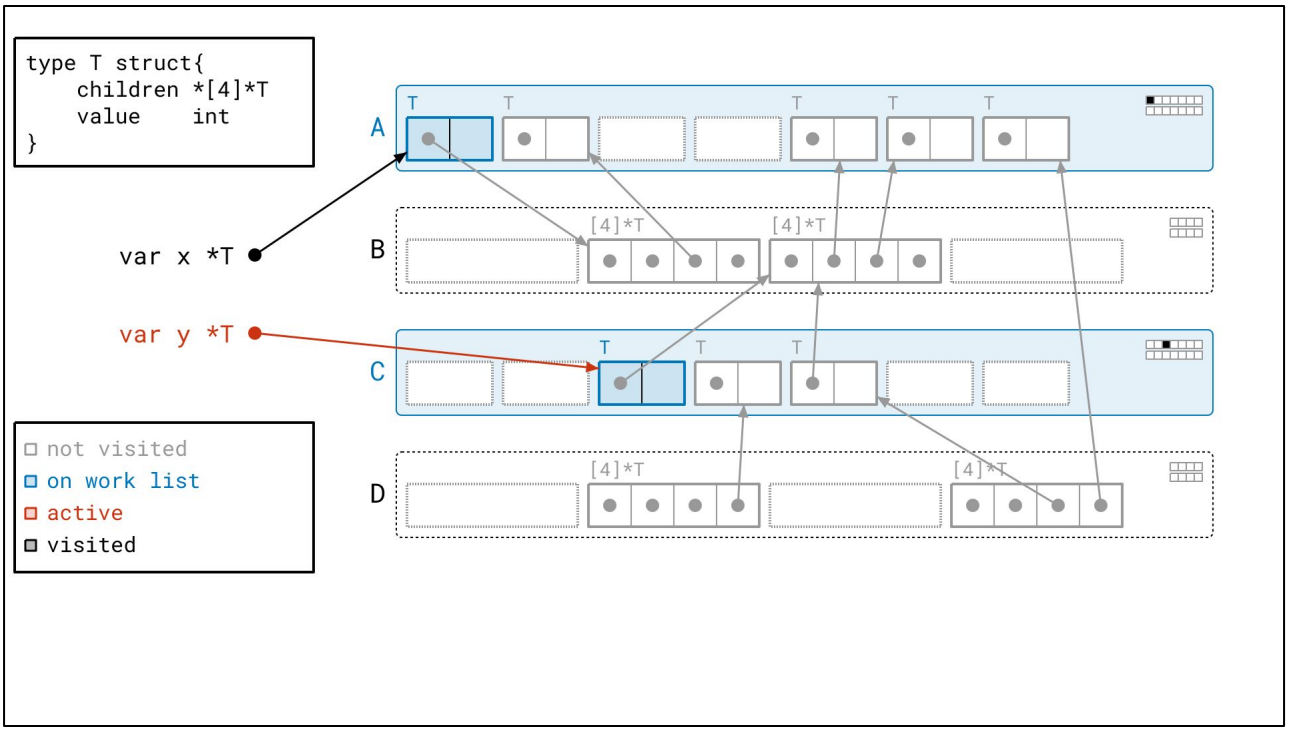
The object we found is *also* blue to indicate that when we do take this page off of the work list, we will need to look at that object. Note that the object's blue hue directly reflects the metadata in page A. Its corresponding *seen* bit is set, but it's *scanned* bit is not.



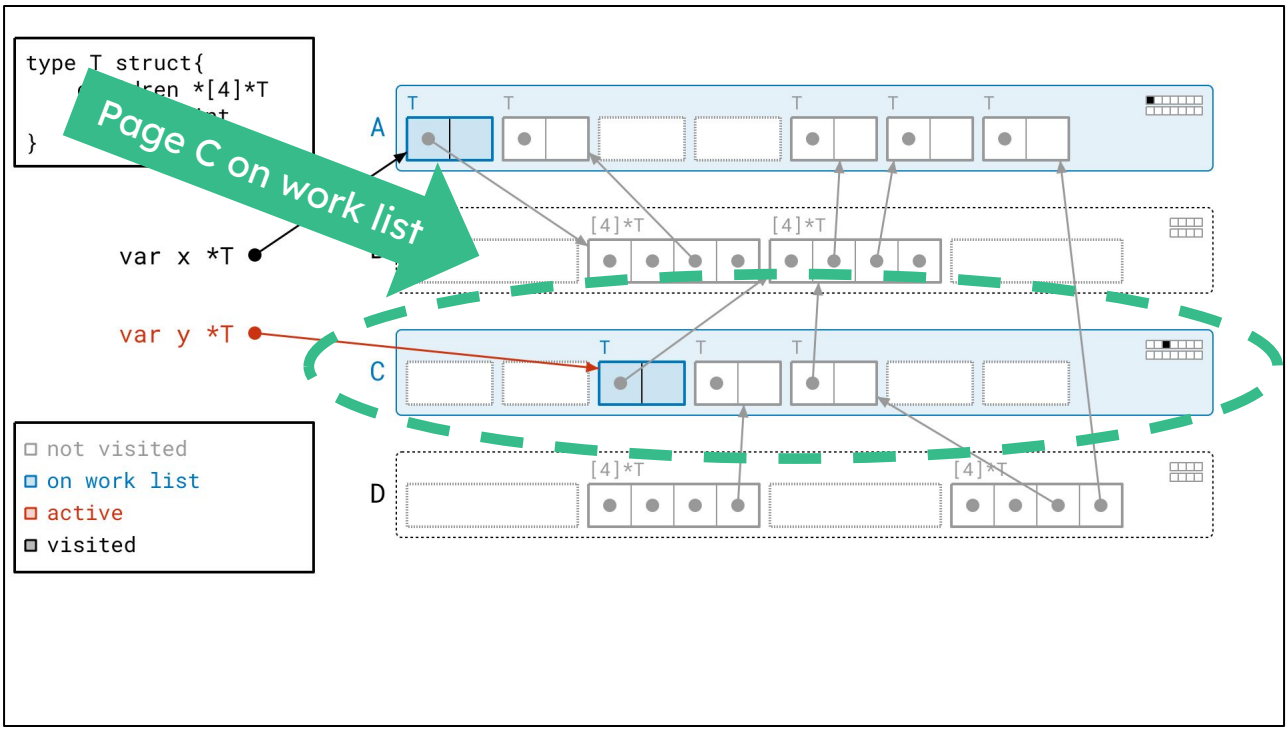
Let's continue.



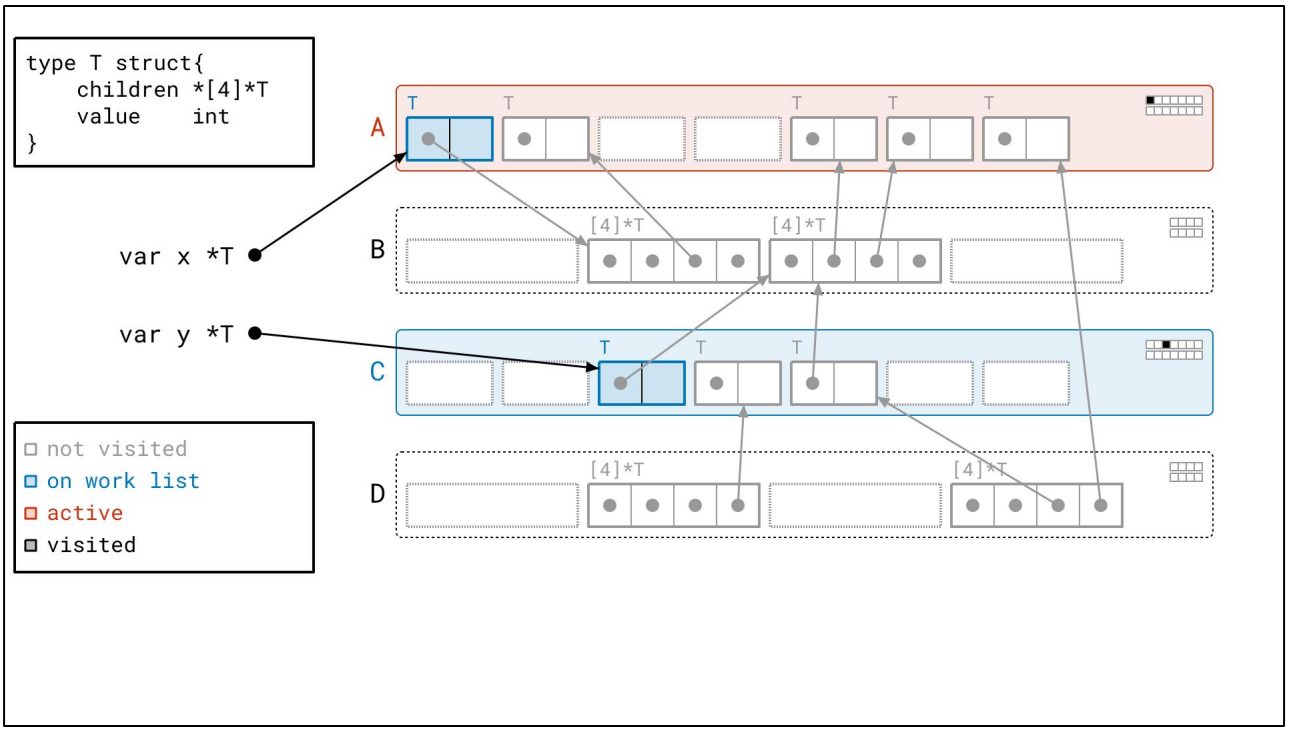
We follow the next root…



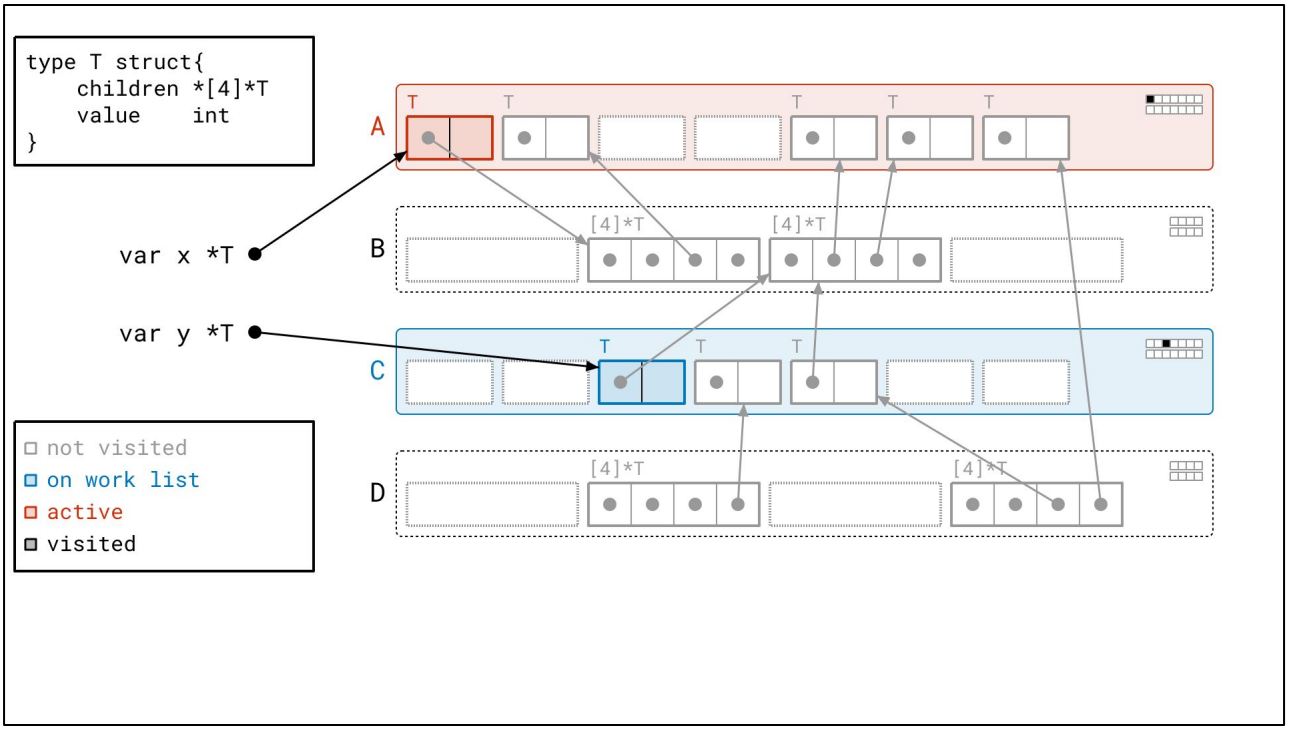
And find another object.



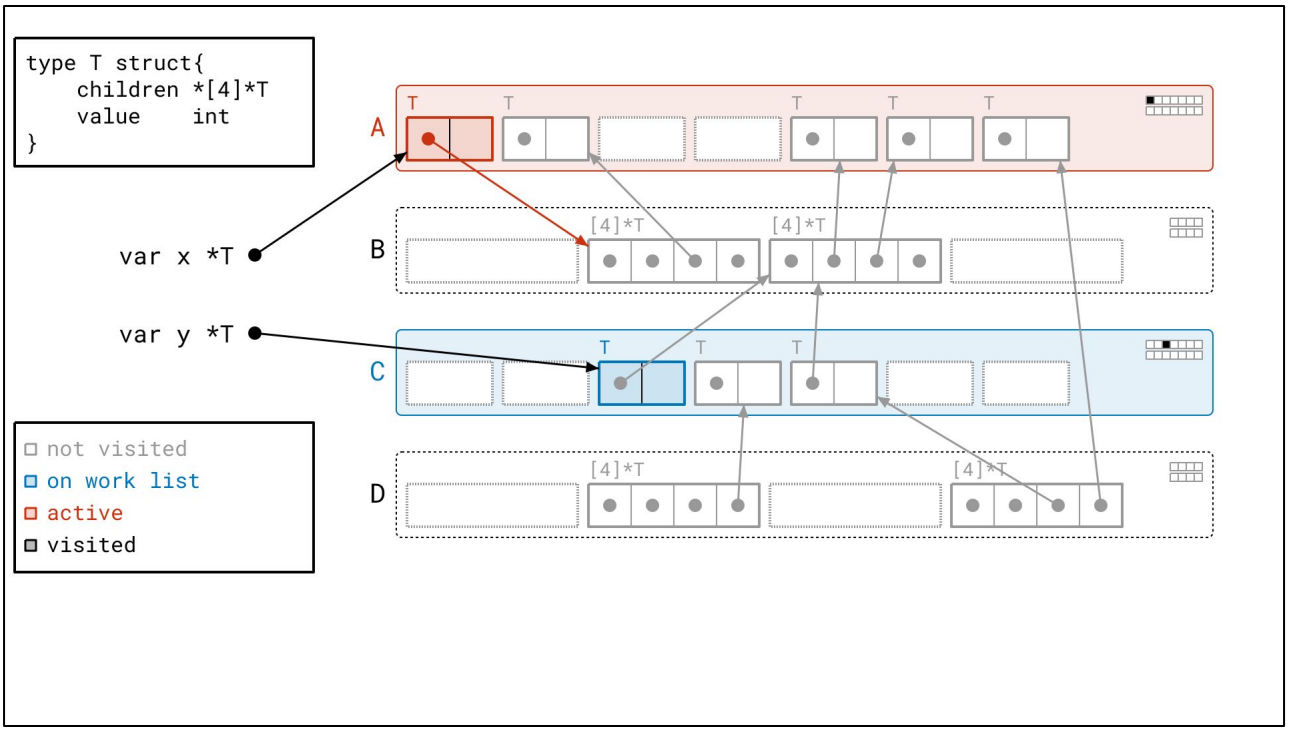
Once again, we put the whole page on the work list. Now page C is now shaded blue.



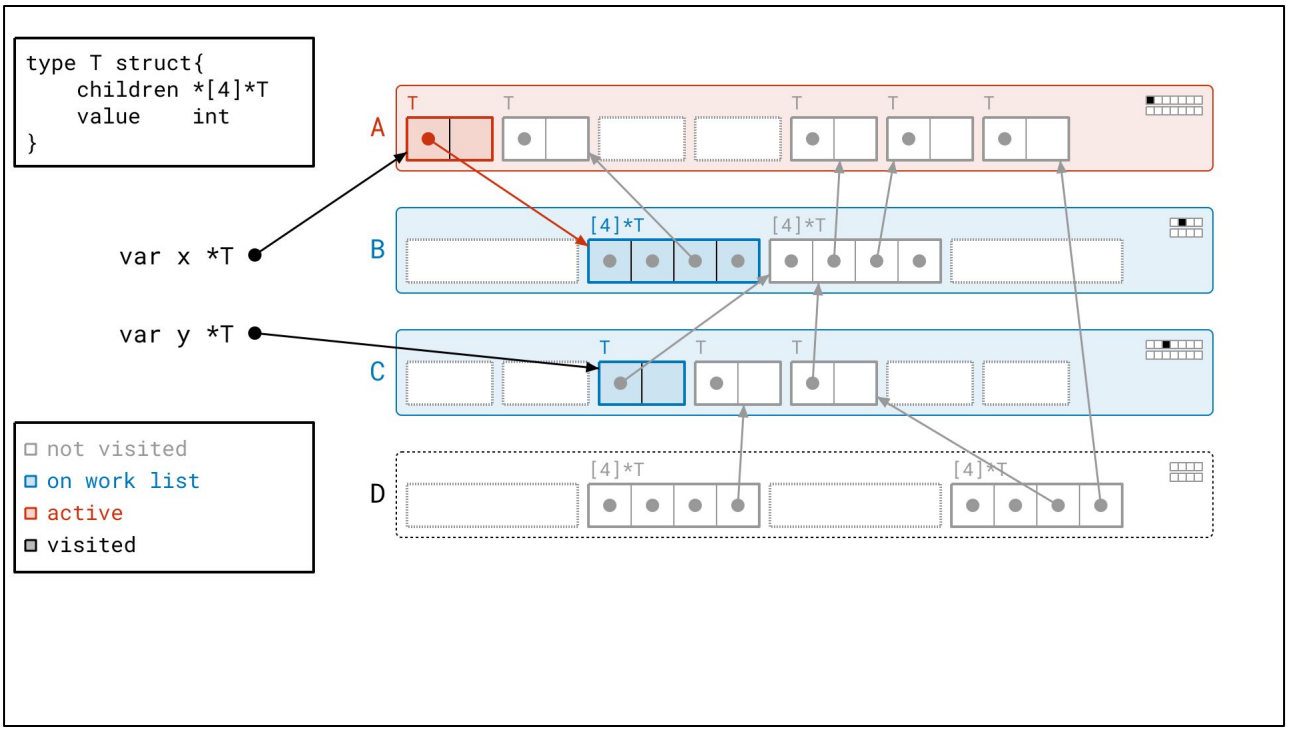
Next, we take page A off of the work list.



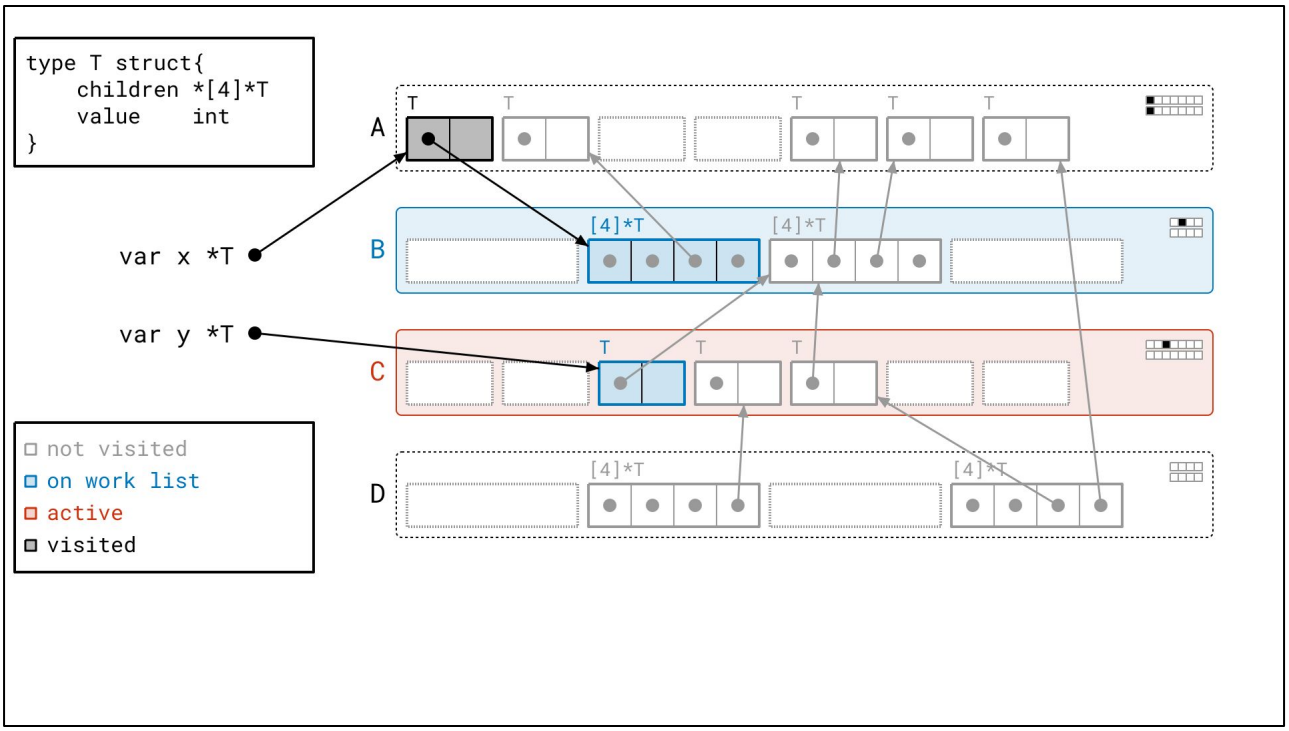
And we see that there's only one object to visit for page A. So, we look at its pointers.


type T struct{
children *[4]*T
value int
}
var x *T
var y *T
A
B
C
D
T
[4]*T
not visited
on work list
active
visited

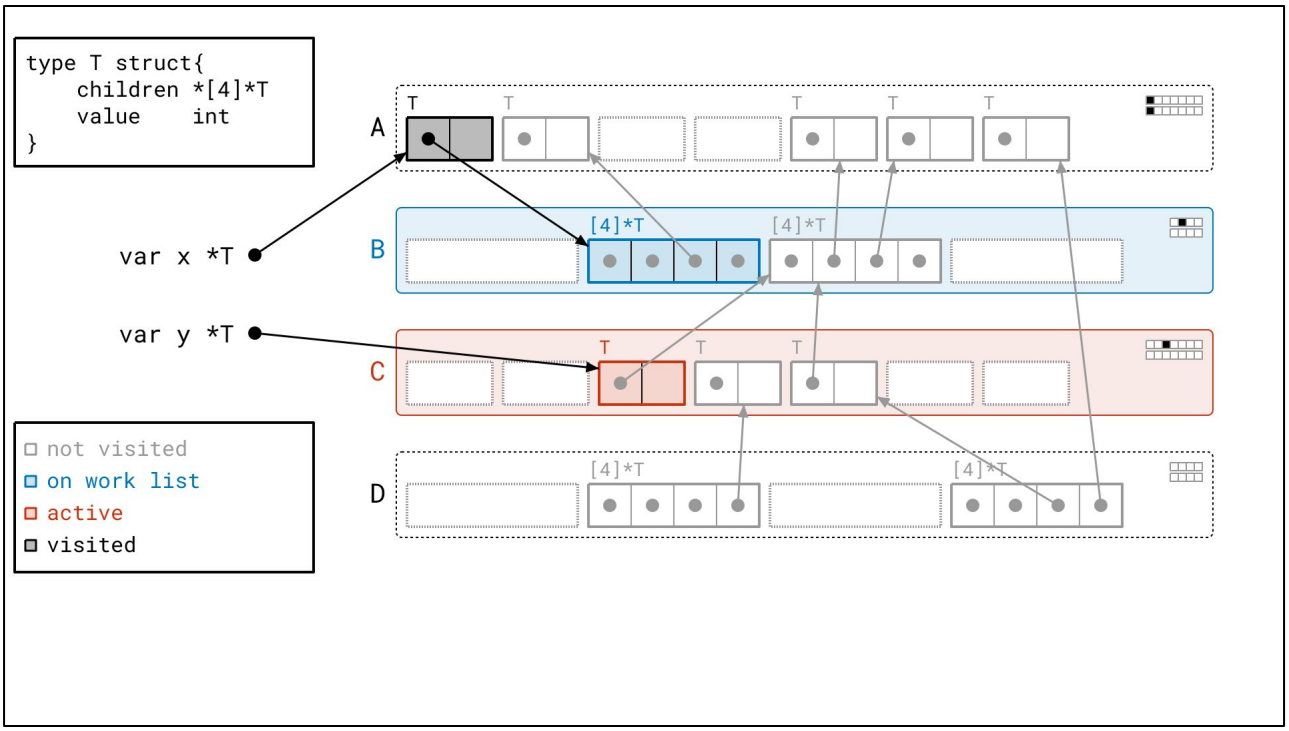
<next>



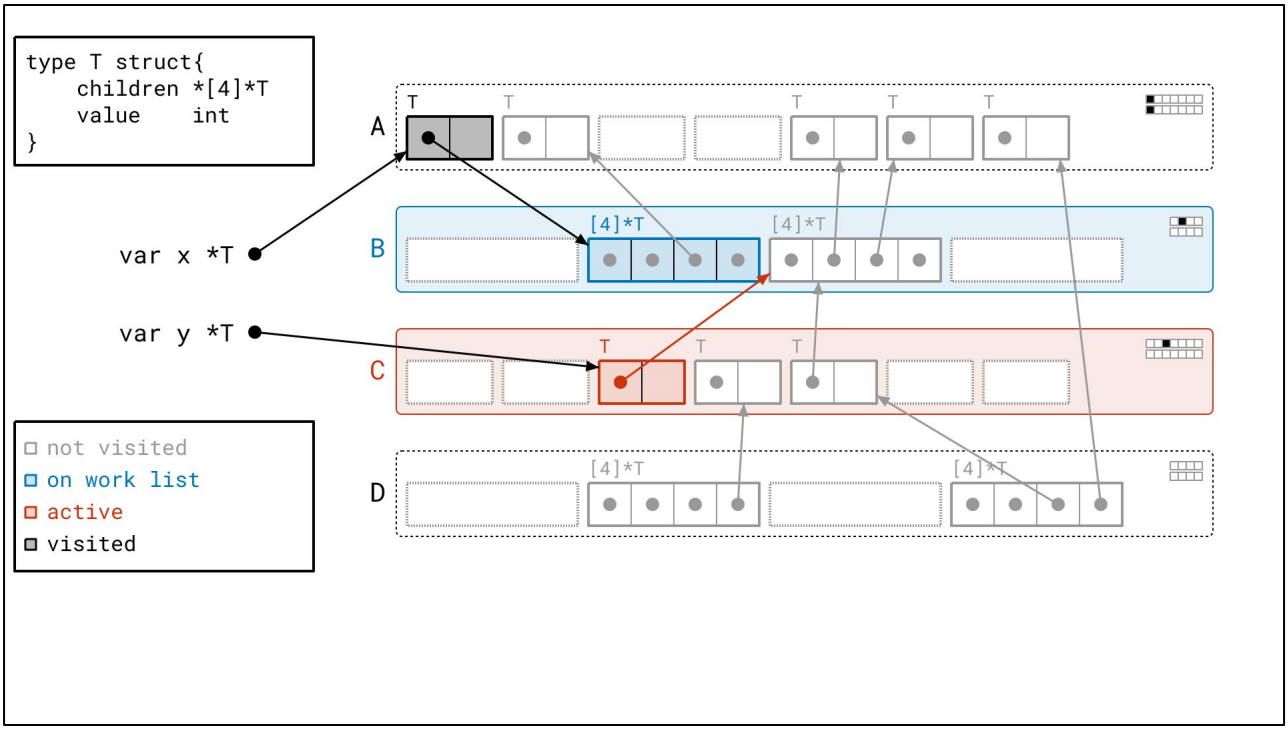
And as a result, we add page B to the work list, since the first object in page A points to an object in page B.



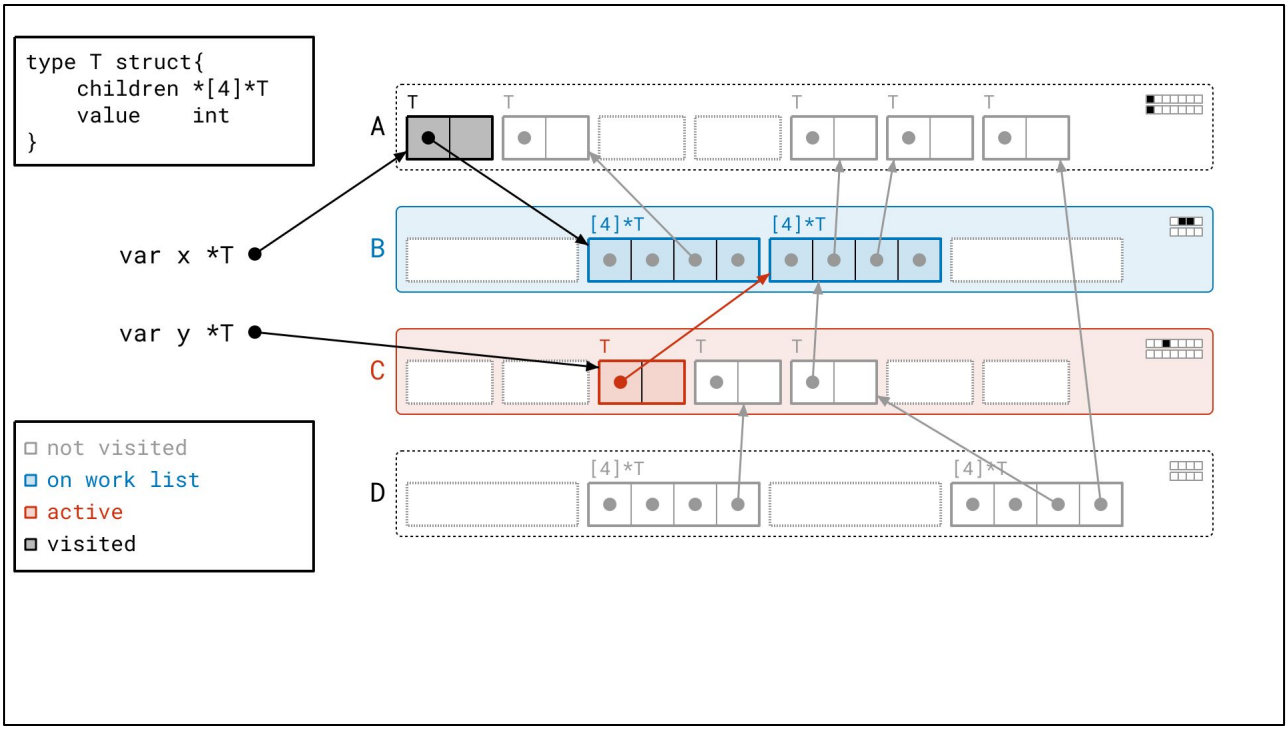
Next we take page C off the work list.



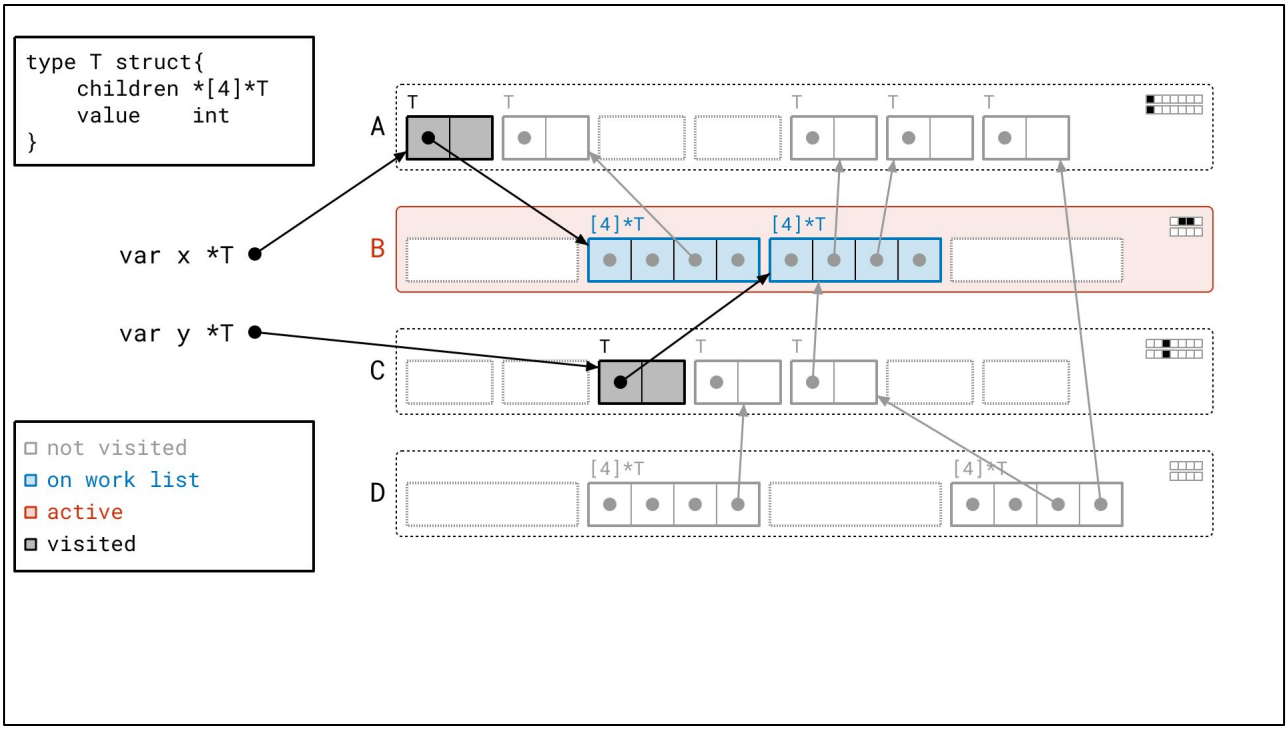
Let's process the object in page C now…



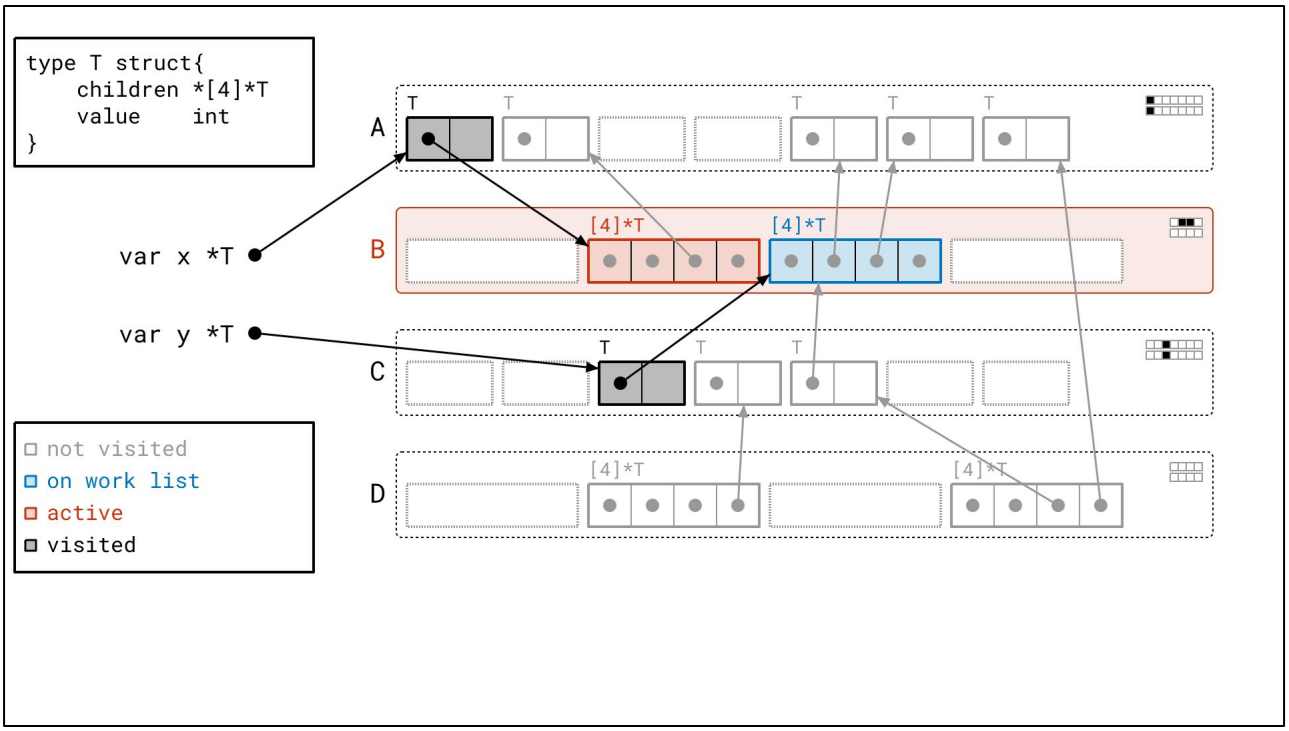
We found another object in page B, which is already on the work list, so no need to add it to the work list.



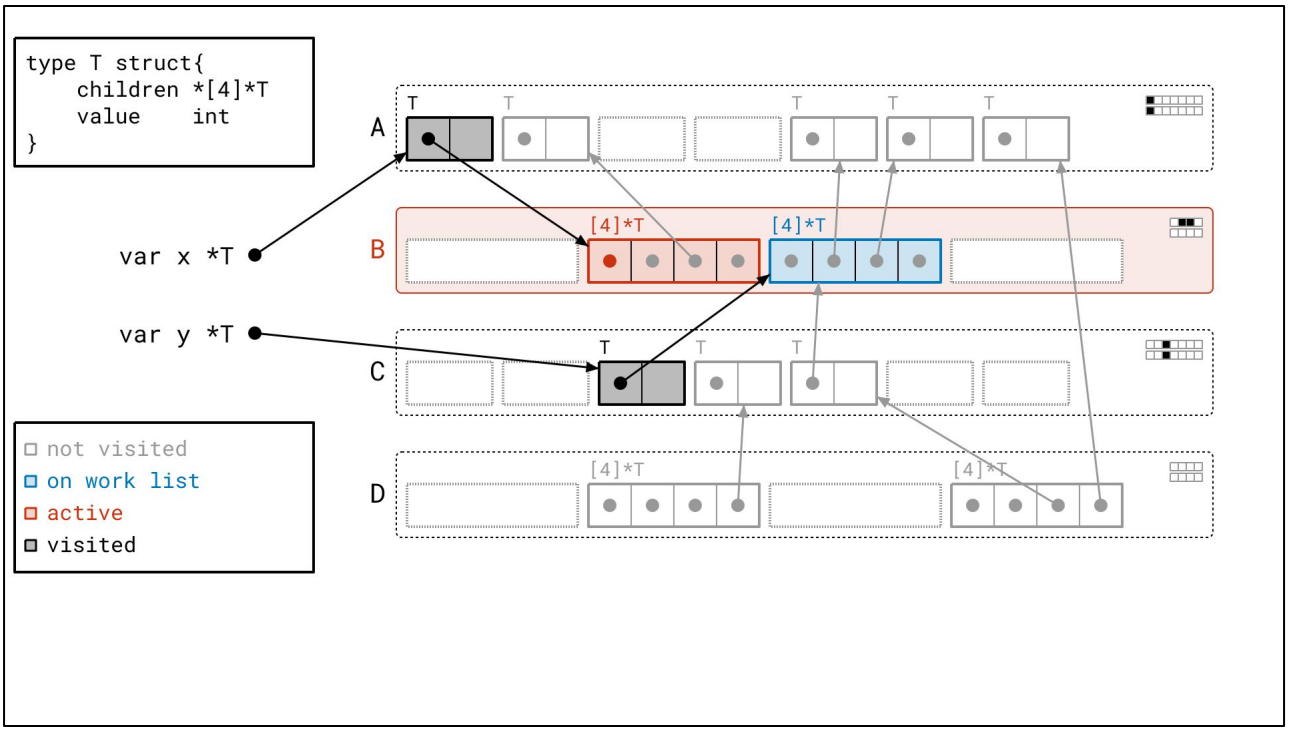
But we still mark the object we saw as *seen*, which means that we'll have to look at it later, when processing page B.



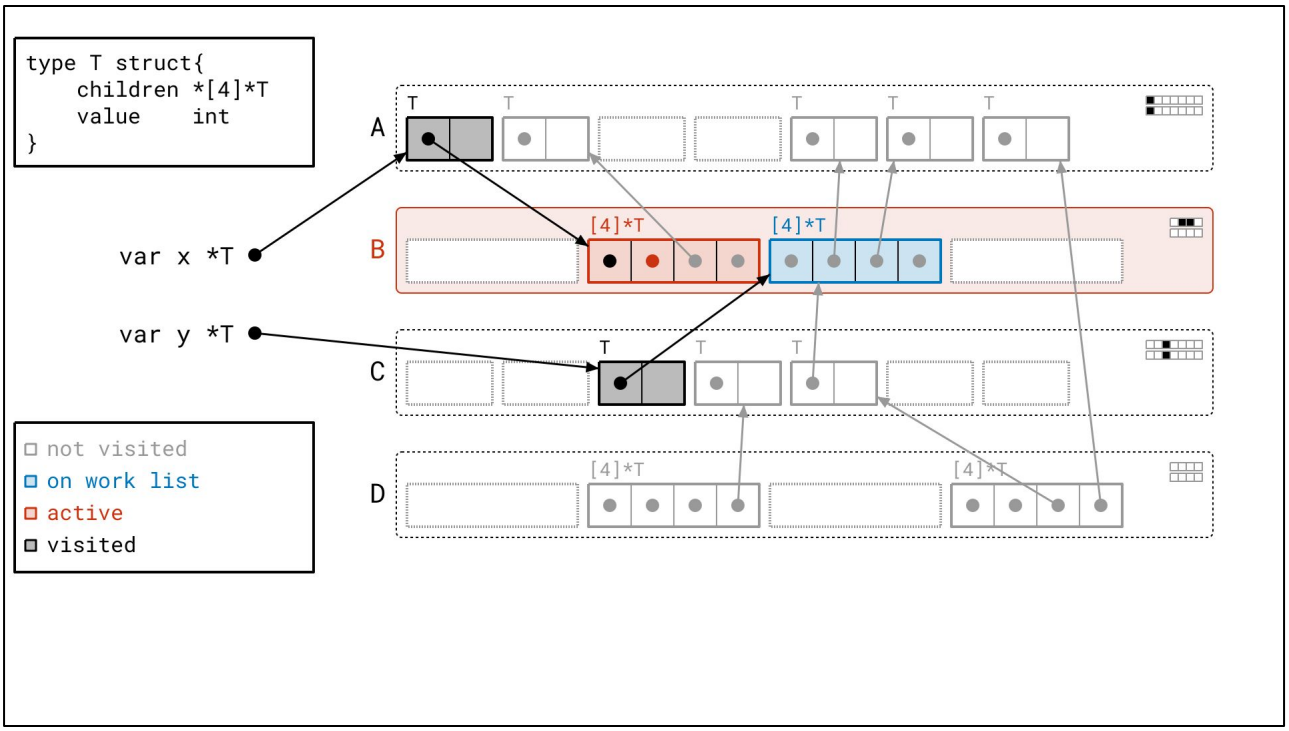
And now it's page B's turn. We've now accumulated *two* objects to scan on page B.



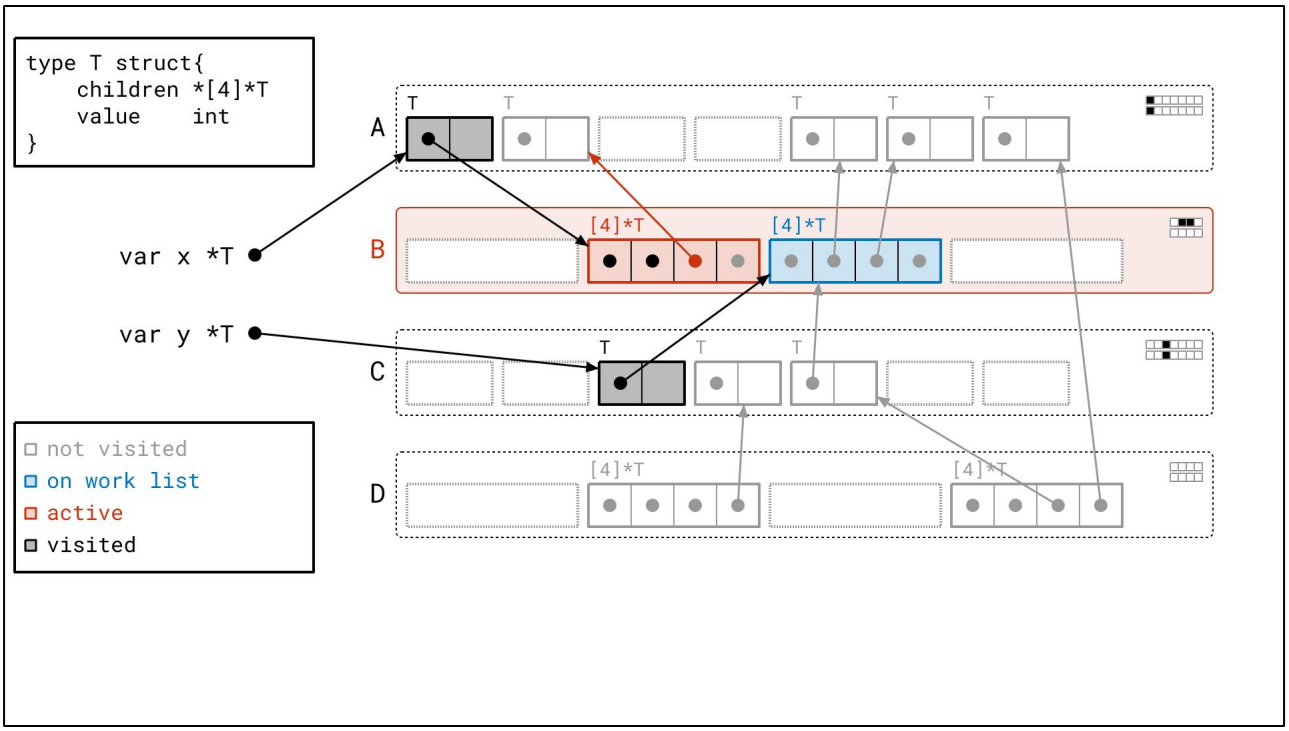
And we process both of these objects in a row, in memory order!



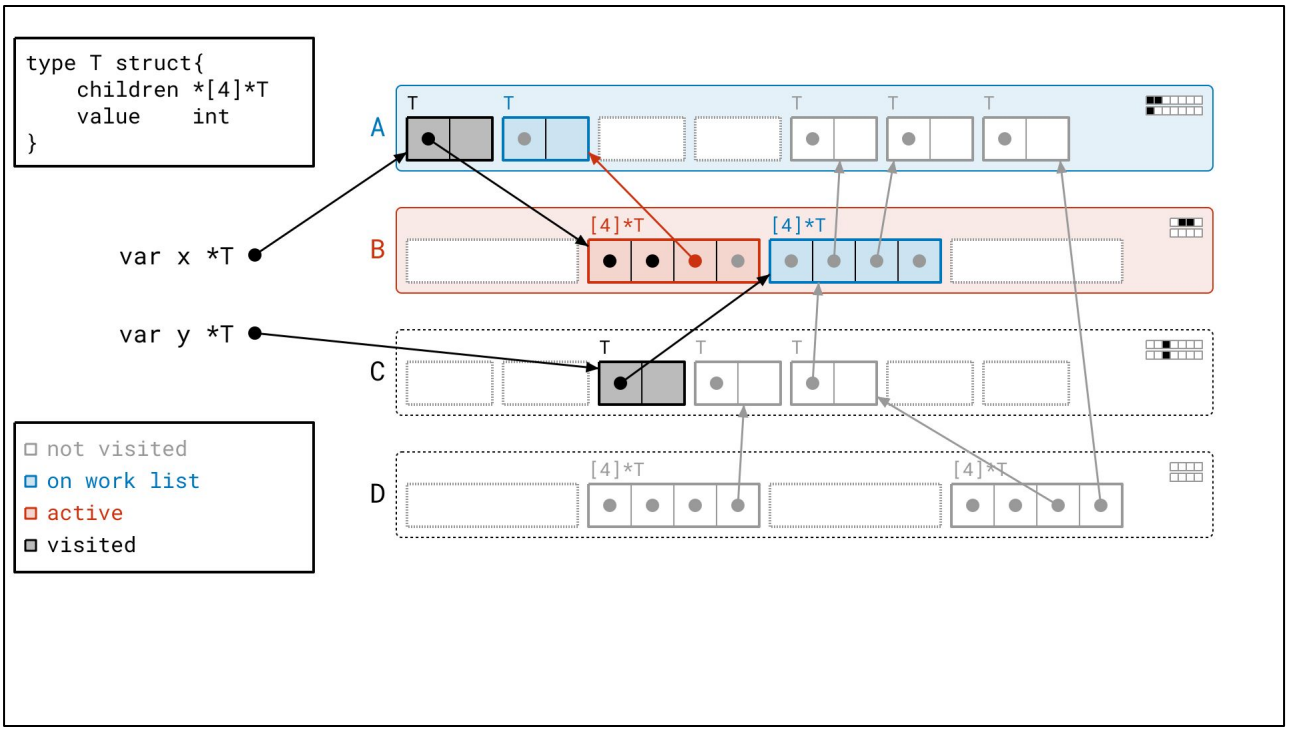
We walk the pointers of the first object…


type T struct{
children *[4]*T
value int
}
var x *T
var y *T
A
B
C
D
T
[4]*T
not visited
on work list
active
visited

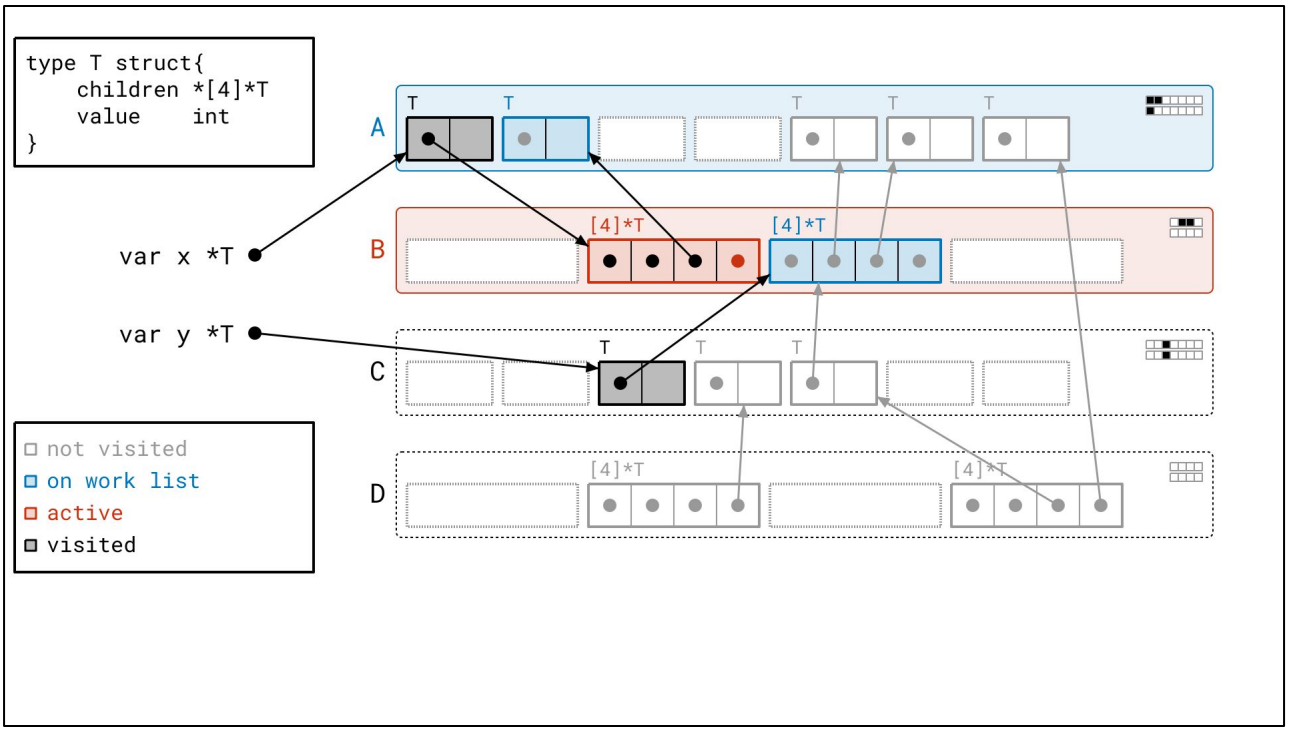
<next>


type T struct{
children *[4]*T
value int
}
var x *T
var y *T
A
B
C
D
T
[4]*T
not visited
on work list
active
visited

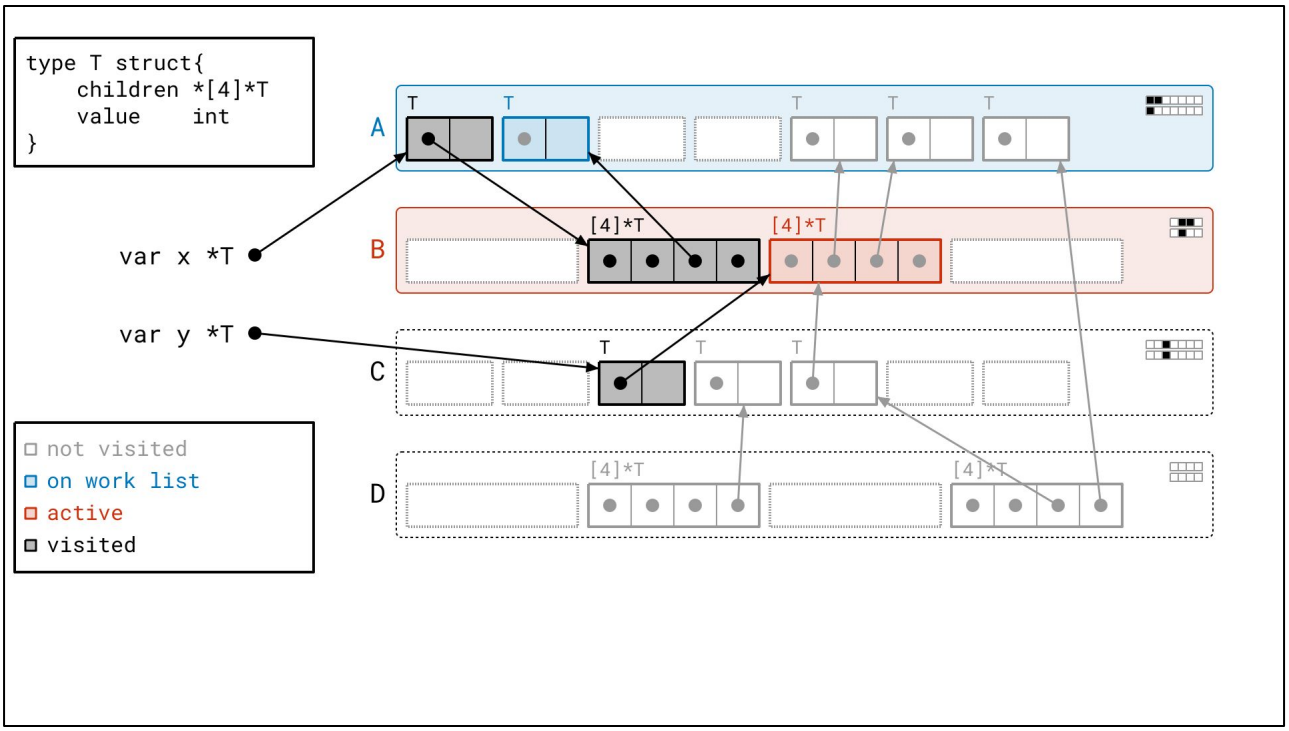
<next>



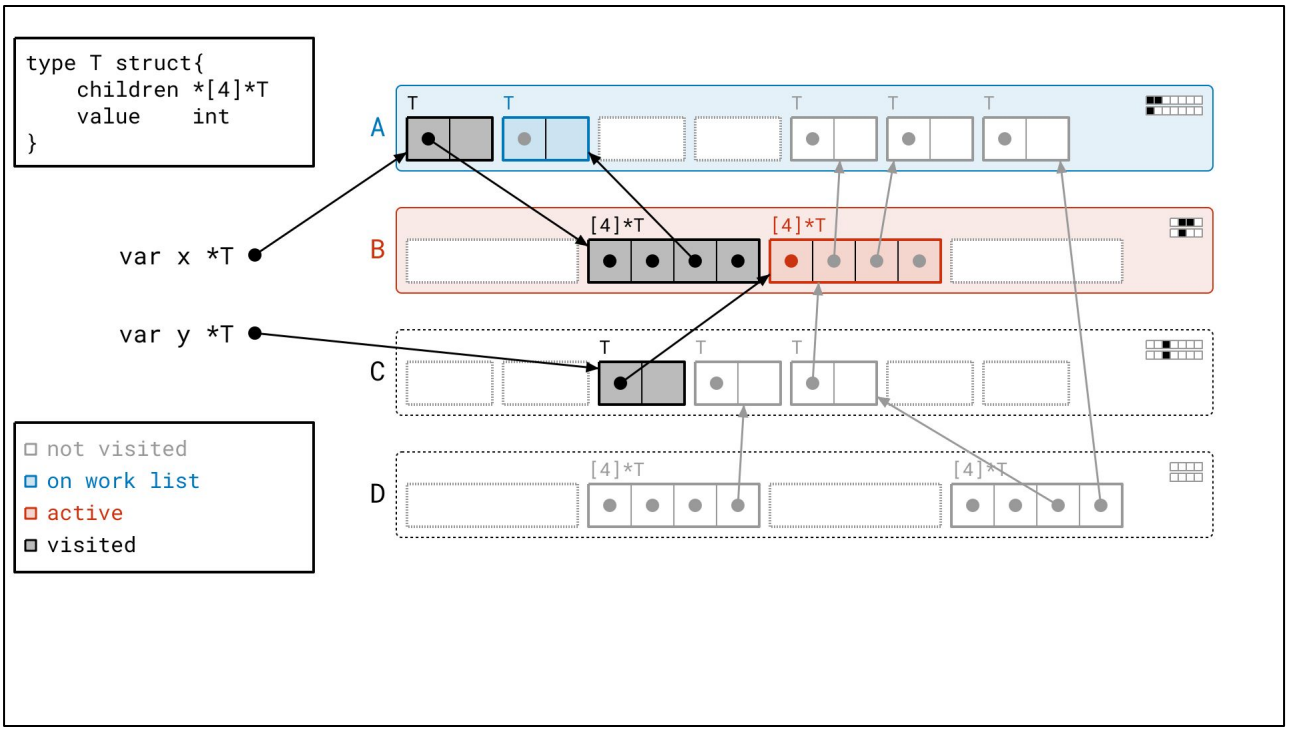
And, we're going to find a bunch of objects in page A, so we put page A on the work list.


type T struct{
children *[4]*T
value int
}
var x *T
var y *T
A
B
C
D
T
[4]*T
not visited
on work list
active
visited

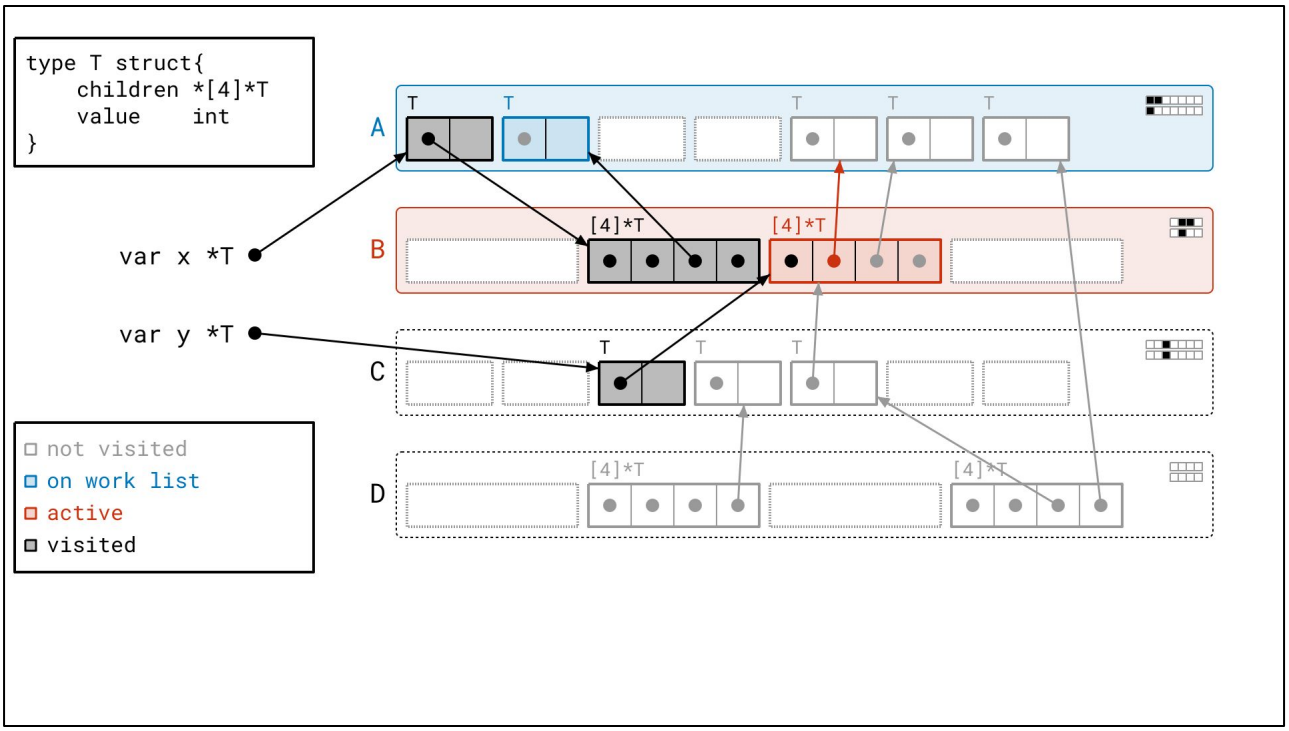
<next>



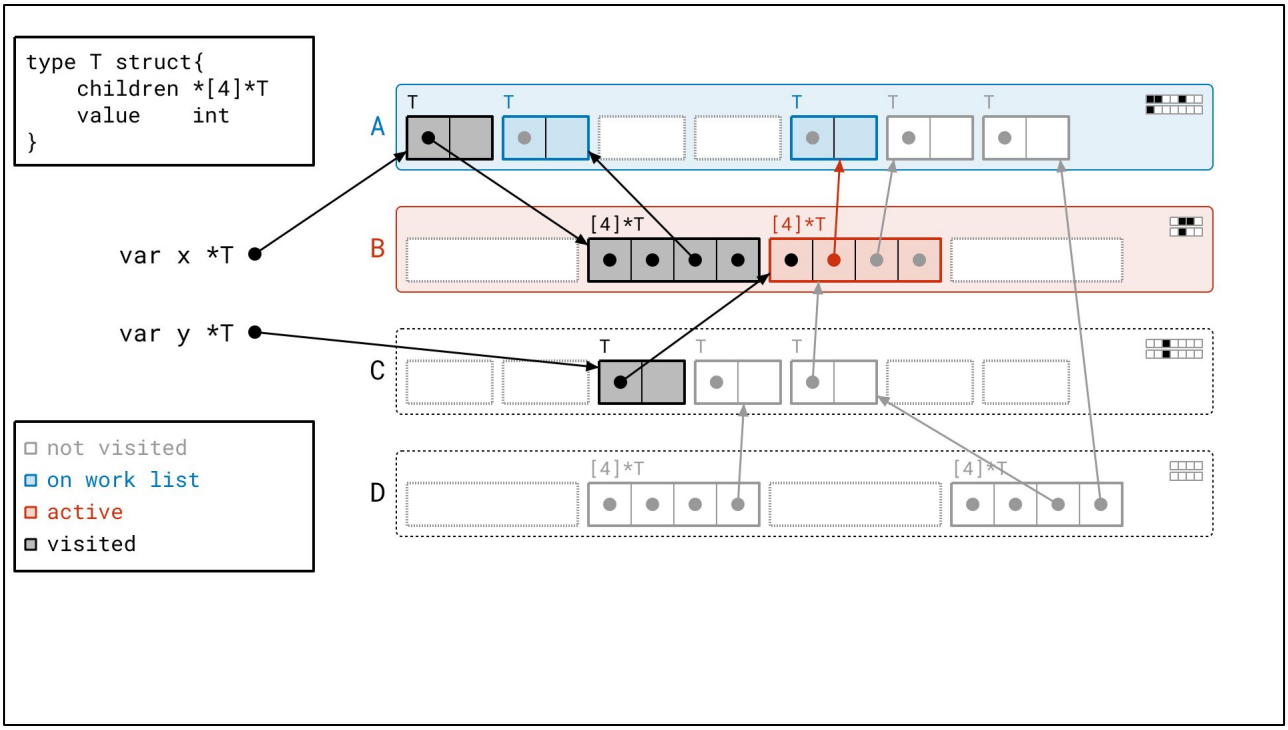
And we handle the second seen object in the page immediately after.


type T struct{
children *[4]*T
value int
}
var x *T
var y *T
A
B
C
D
T
[4]*T
not visited
on work list
active
visited

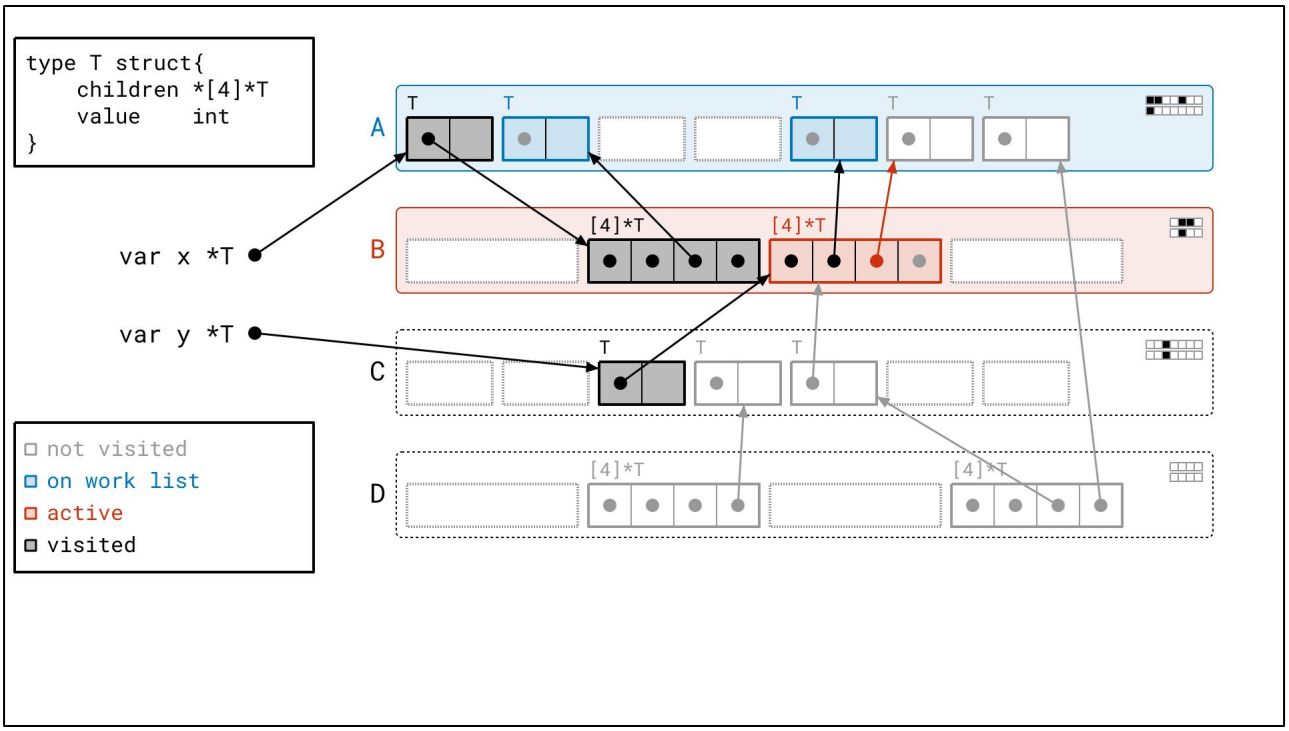
<next>


type T struct{
children *[4]*T
value int
}
var x *T
var y *T
A
B
C
D
T
[4]*T
not visited
on work list
active
visited

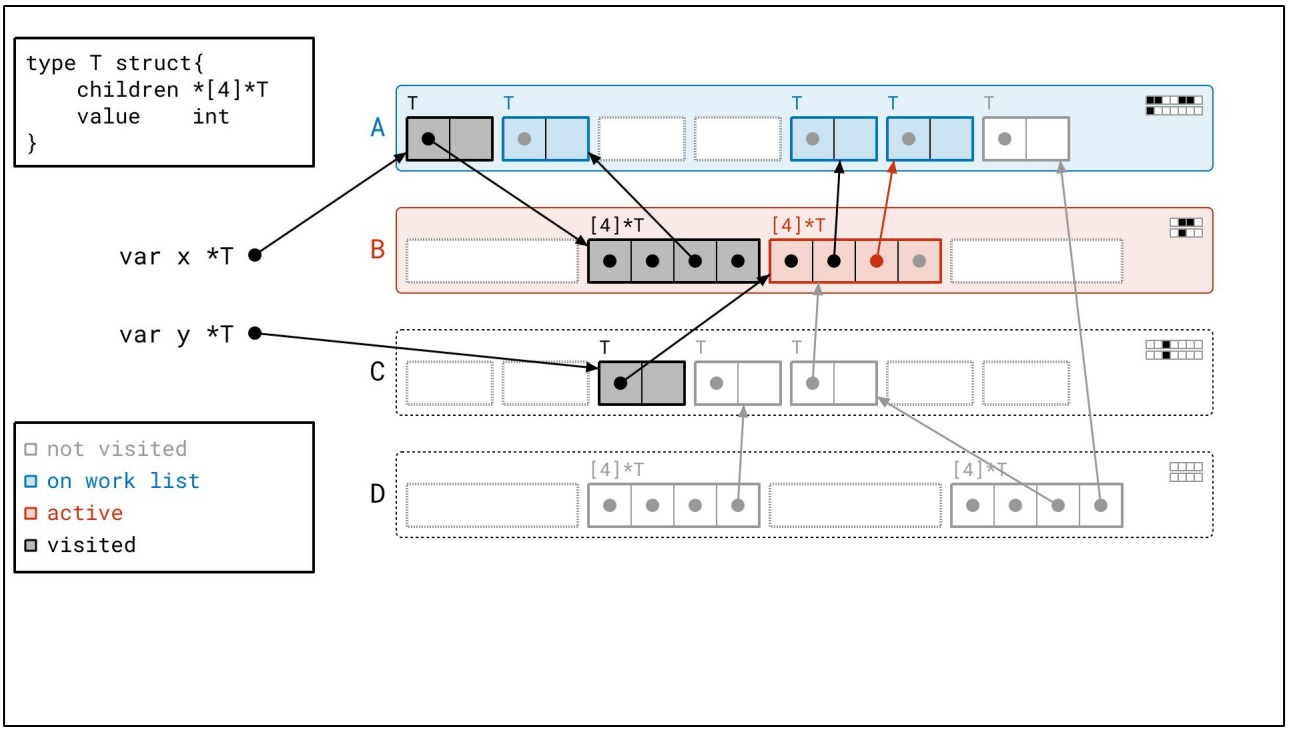
<next>



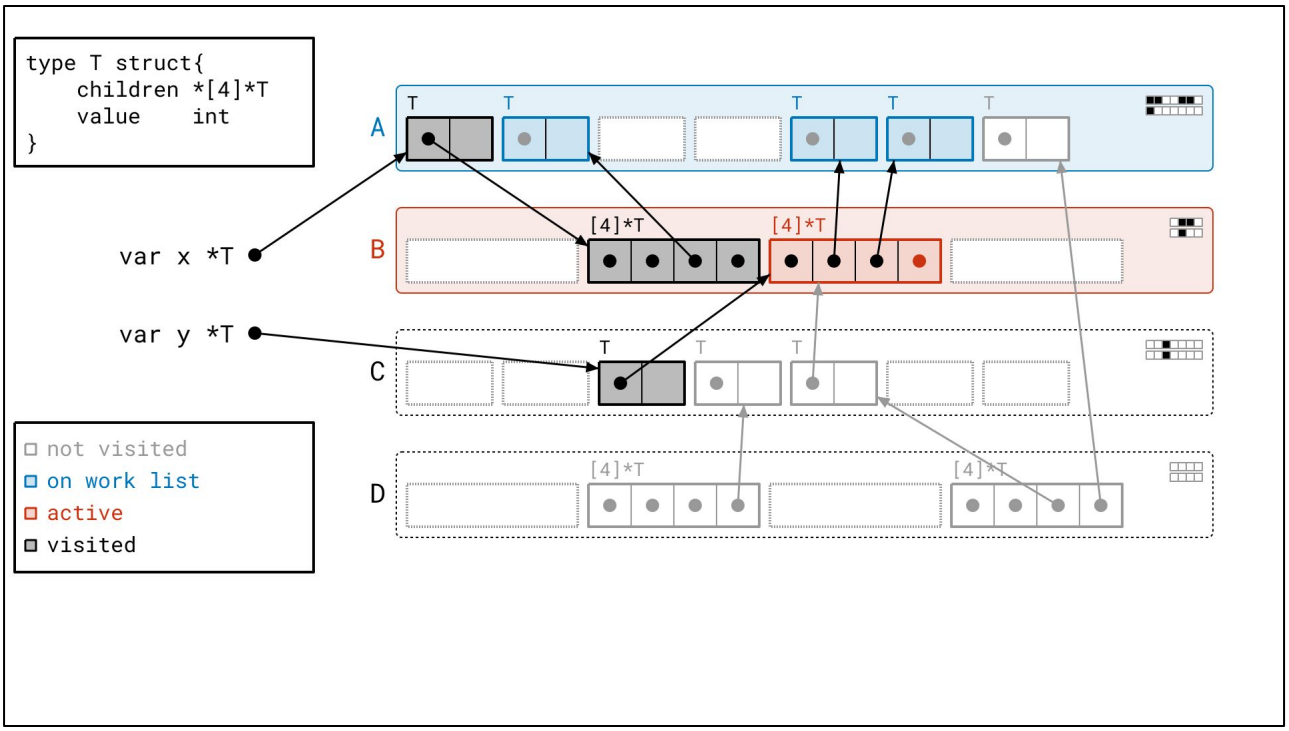
We find a few more objects in page A…


type T struct{
children *[4]*T
value int
}
var x *T
var y *T
A
B
C
D
T
[4]*T
not visited
on work list
active
visited

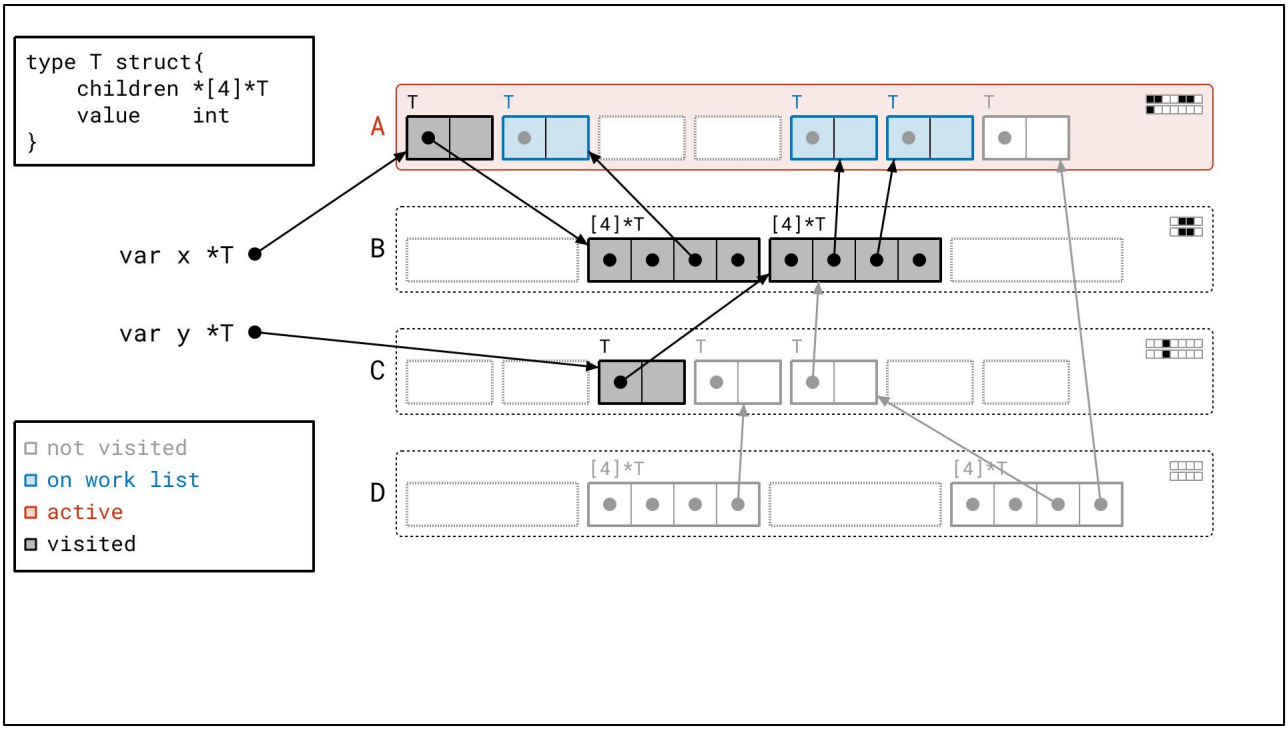
<next>


type T struct{
children *[4]*T
value int
}
var x *T
var y *T
A
B
C
D
T
[4]*T
not visited
on work list
active
visited

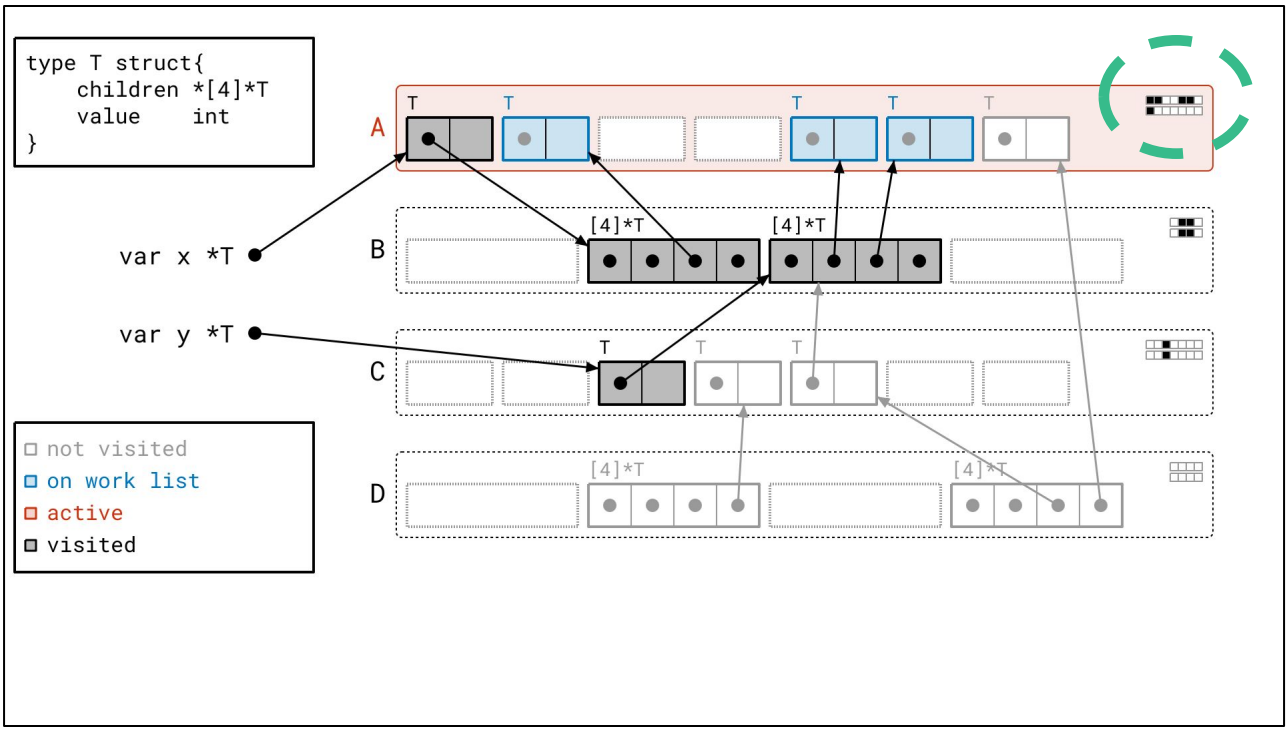
<next>


type T struct{
children *[4]*T
value int
}
var x *T
var y *T
A
B
C
D
T
[4]*T
not visited
on work list
active
visited

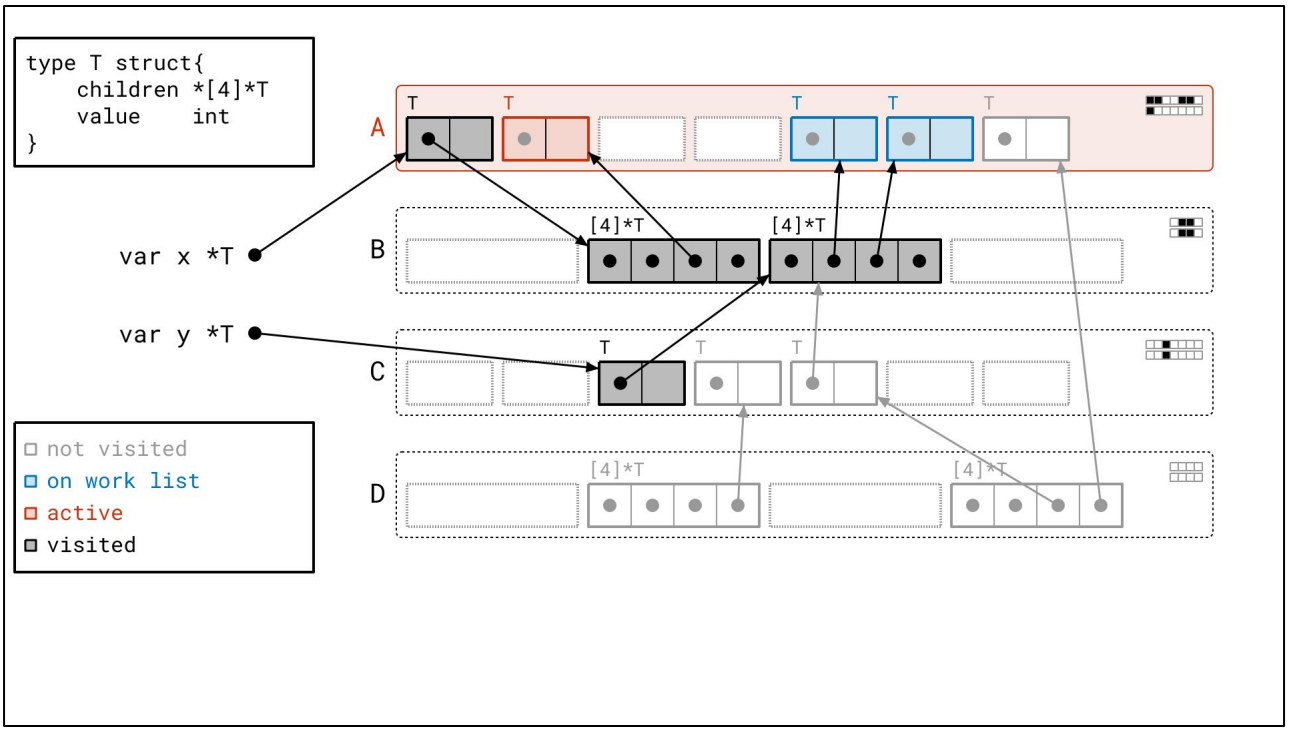
<next>



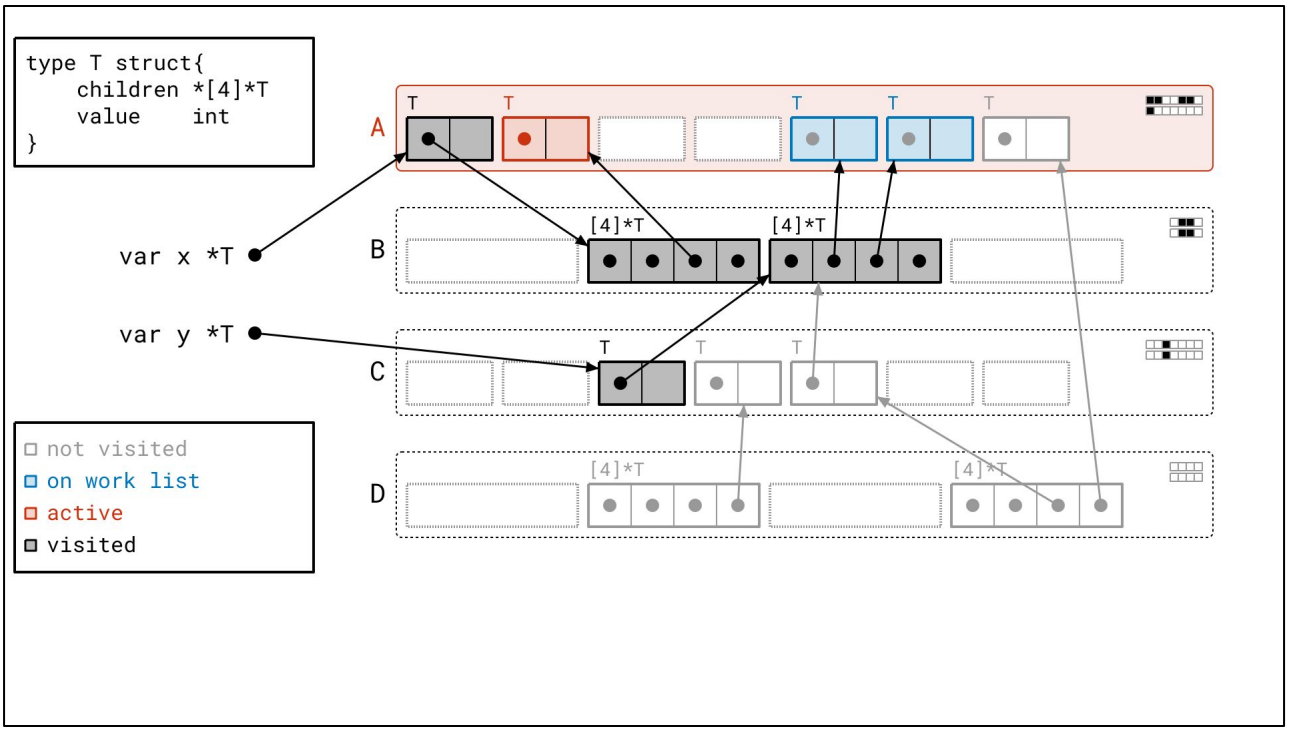
And now it's time to process page A. Notice how we only need to process three objects, not four. We don't need to process first one, since we already looked at it before. How do we know?



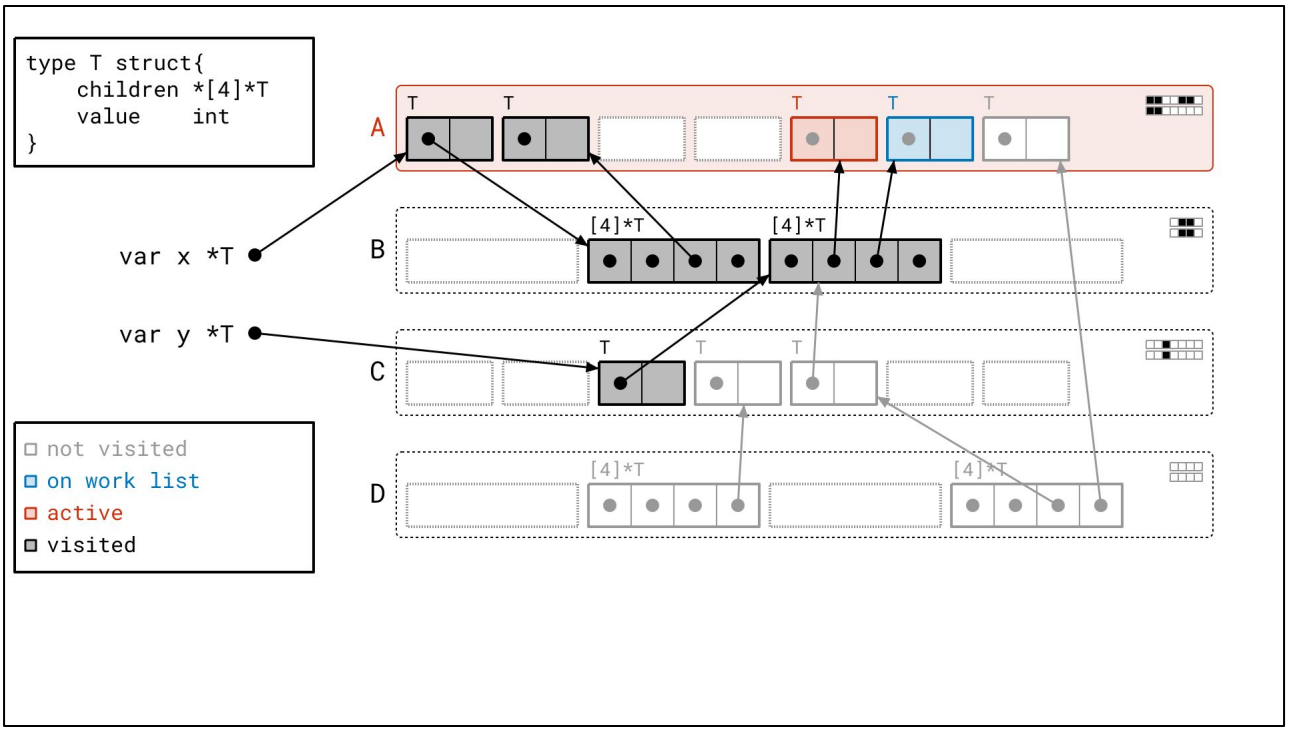
Again, it's difference between the "seen" and "scanned" bits that tells us what we need to do.



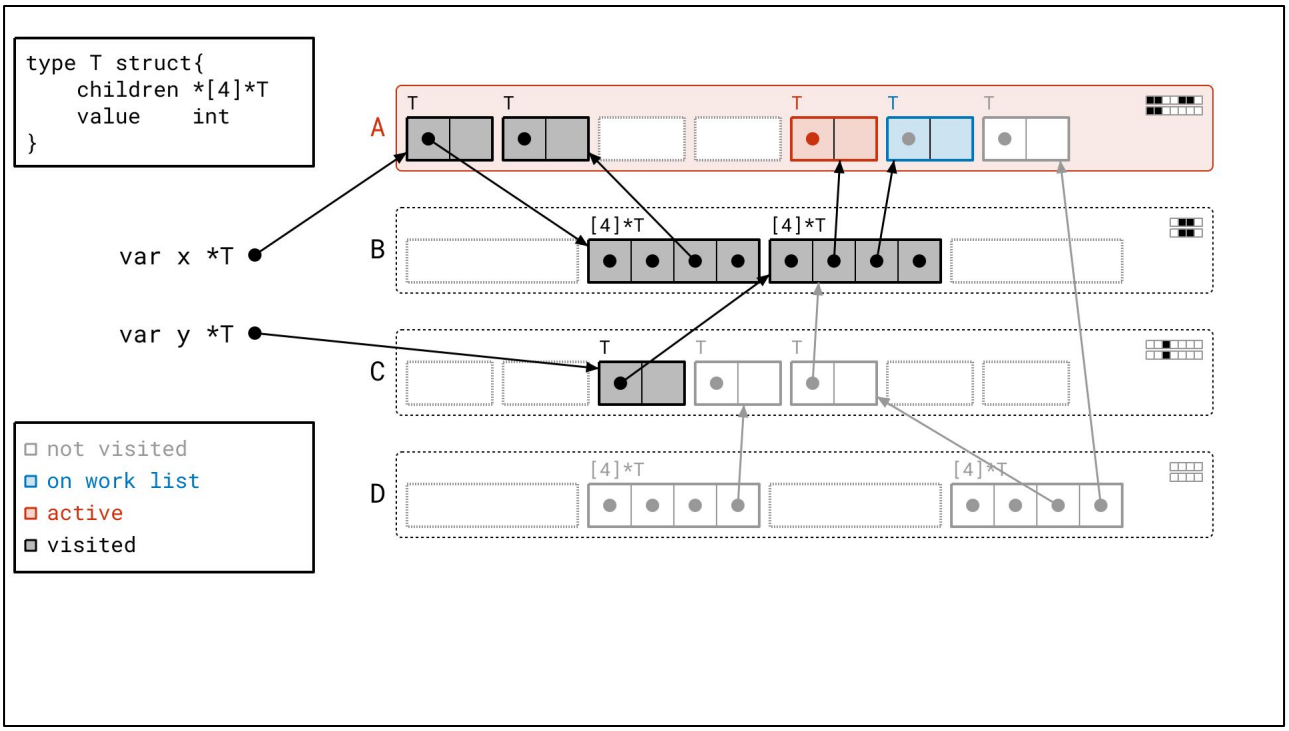
Let's just quickly process the rest of these…


type T struct{
children *[4]*T
value int
}
var x *T
var y *T
A
B
C
D
T
[4]*T
not visited
on work list
active
visited

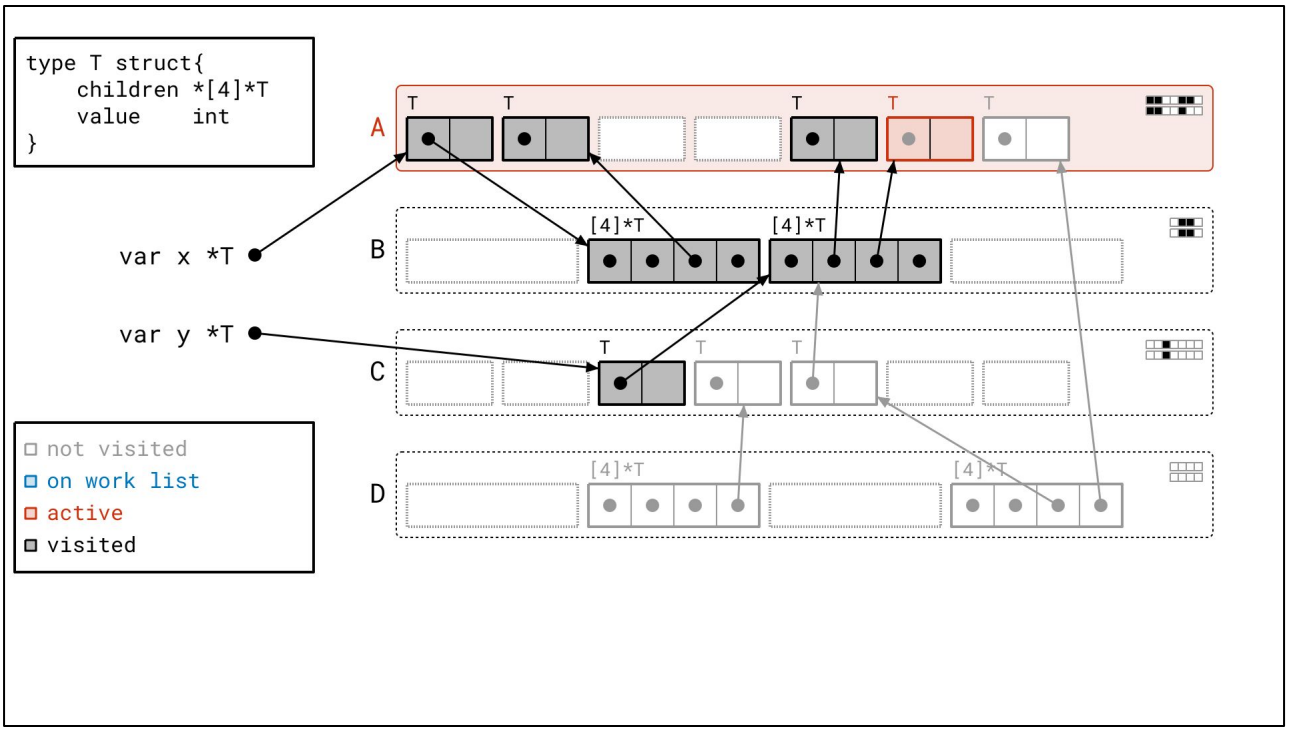
<next>


type T struct{
children *[4]*T
value int
}
var x *T
var y *T
A
B
C
D
T
[4]*T
not visited
on work list
active
visited

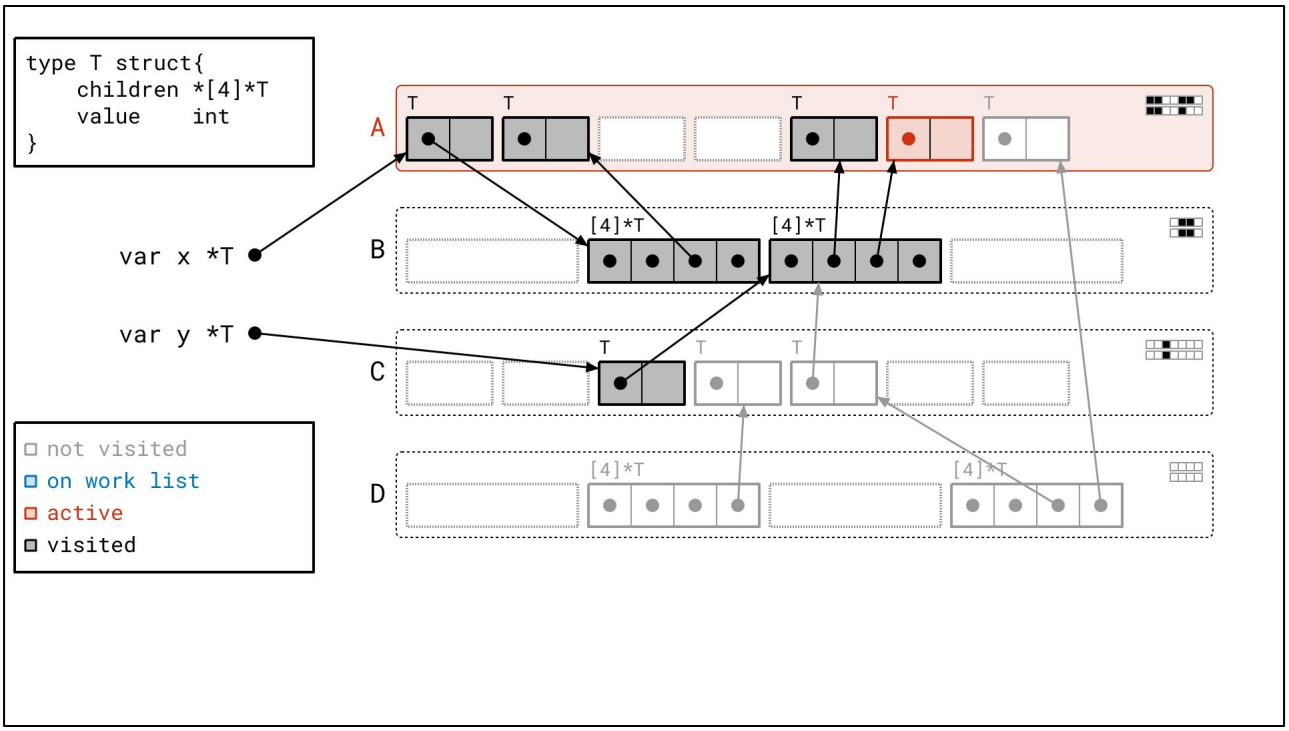
<next>


type T struct{
children *[4]*T
value int
}
var x *T
var y *T
A
B
C
D
T
[4]*T
not visited
on work list
active
visited

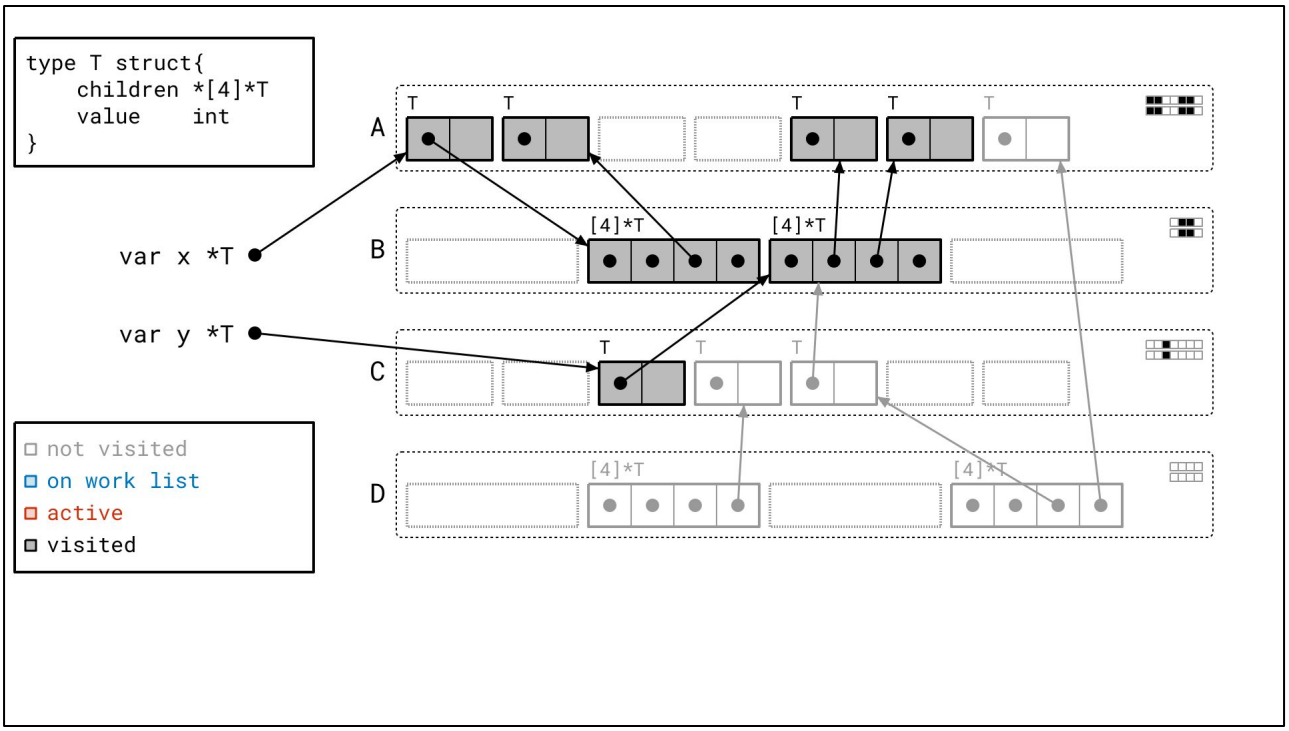
<next>


type T struct{
children *[4]*T
value int
}
var x *T
var y *T
A
B
C
D
T
[4]*T
not visited
on work list
active
visited

and…

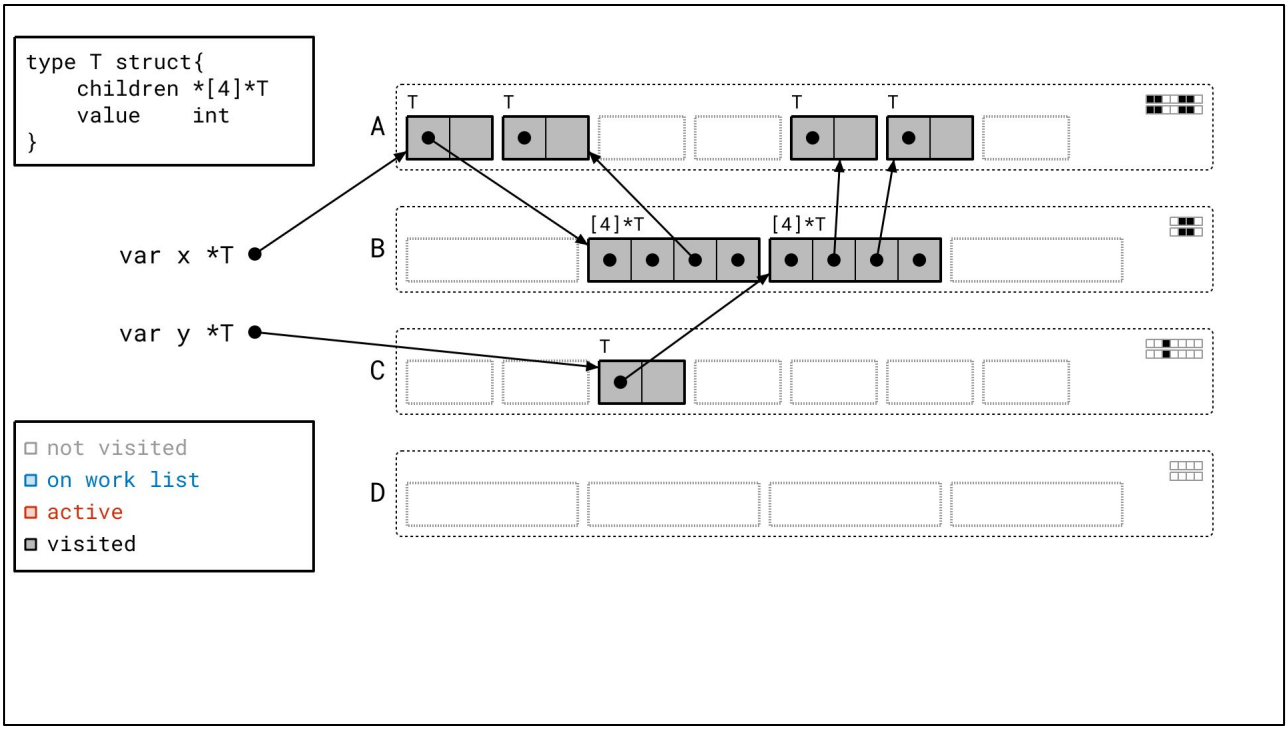

type T struct{
children *[4]*T
value int
}
var x *T
var y *T
A
B
C
D
T
[4]*T
not visited
on work list
active
visited

<next>



We're done! There are no more pages are on the work list and there's nothing we're actively looking at. Notice that the metadata now all lines up nicely, since all these objects were both *seen* and *scanned*.

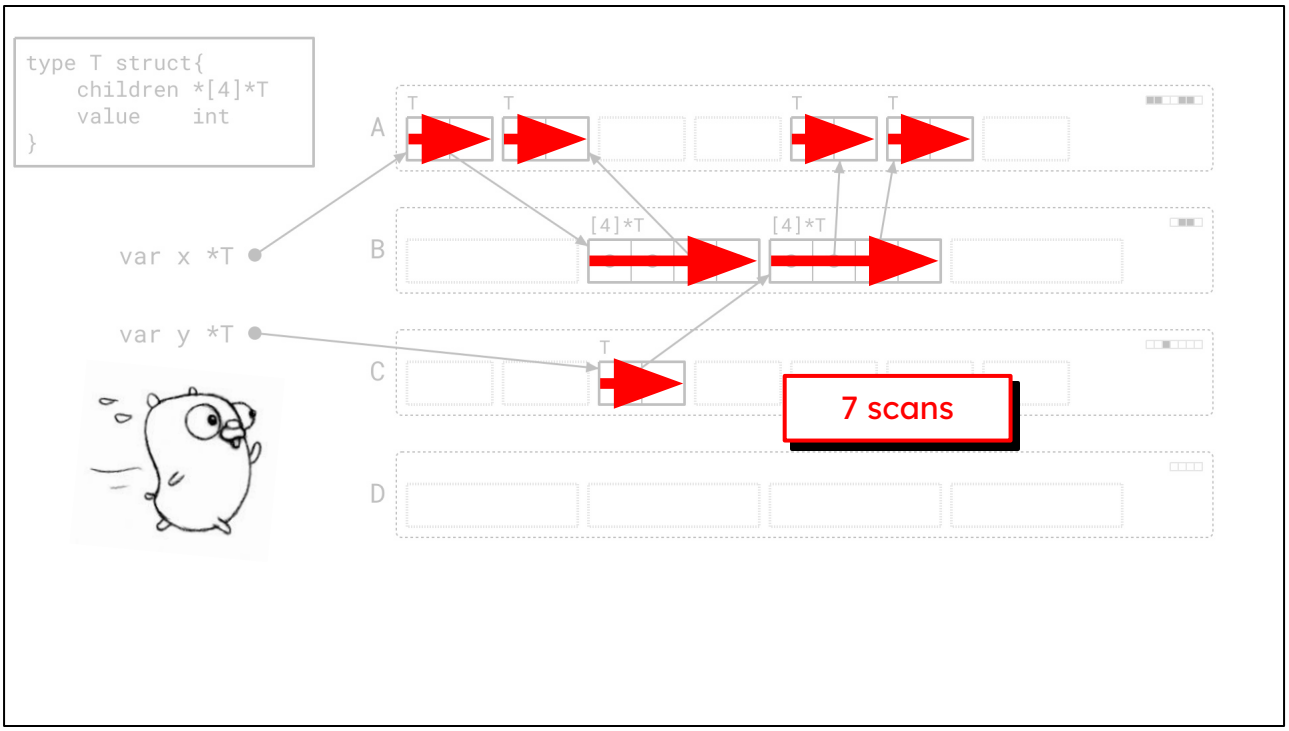
You may have also noticed during our traversal that the work list order is a little different from the graph flood too. Where the graph flood had a last-in-first-out, or stack-like, order, here we're using a first-in-first-out, or queue-like, order for the pages on our work list. This is intentional. We let seen objects accumulate on each page while the page sits on the queue, so we can process as many as we can at once. That's how we were able to hit so many objects on page A at once. Sometimes laziness is a virtue.



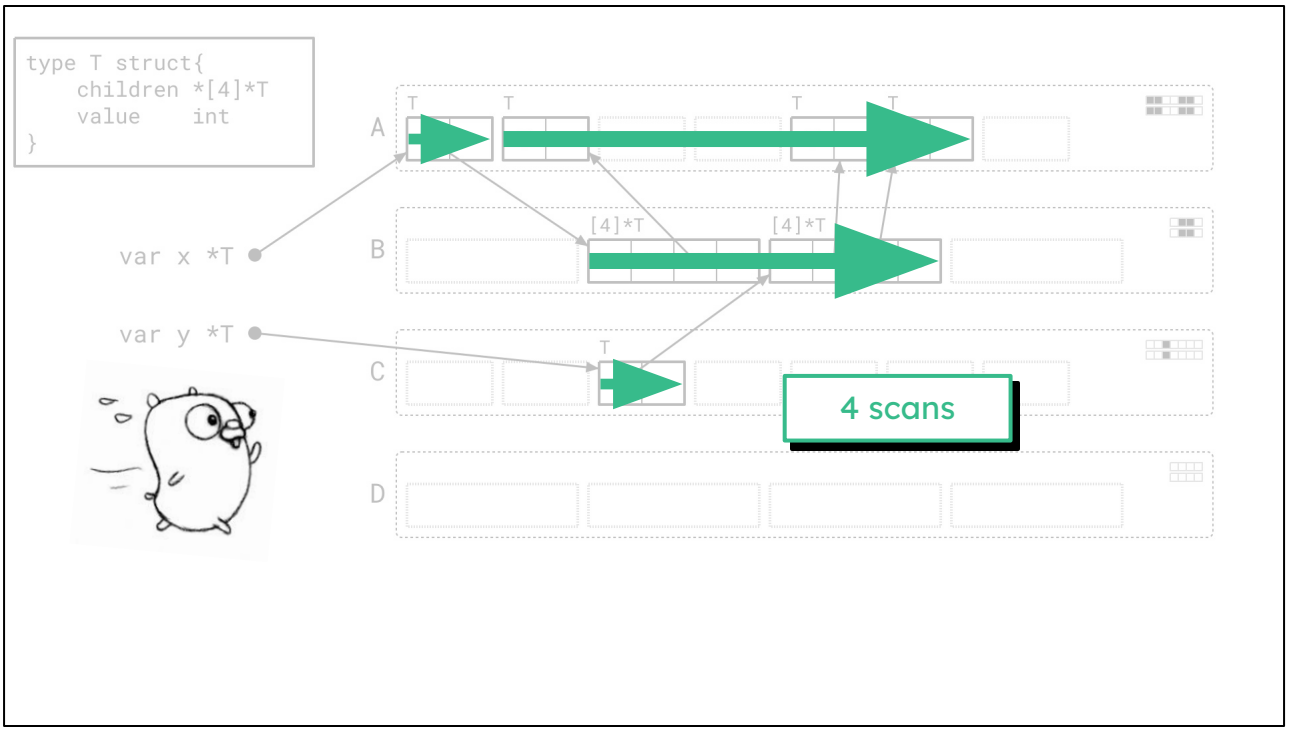
And finally we can sweep away the unvisited objects, as before.

Getting on the highway

Let's come back around to our driving analogy. Are we finally getting on the highway?

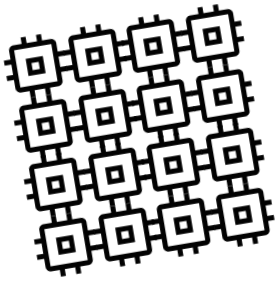


Let's recall our graph flood picture before. We jumped around a whole lot, doing little bits of work in different places.



And with Green Tea, it looks a lot better! What I want you to focus on in this picture in particular is the long, clean left-to-right arrows across pages A and B. The longer these arrows, the better, and with bigger heaps, this effect can be even stronger. *That's* the magic of this algorithm, and *that's* our opportunity to ride the highway.

A better fit for modern hardware



Scanning whole pages ***fits the microarchitecture better***:

*Much higher chance we **scan objects close together**.*

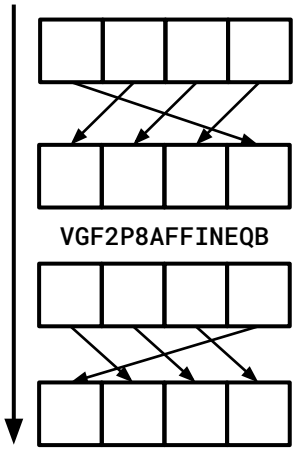
Fewer object metadata lookups.

*Tracking pages means **less pressure on work lists.***

*Opportunity for **vector acceleration**!*

More concretely, this all adds up to a better fit with the microarchitecture.

- We can clearly scan objects closer together with much higher probability, so there's a better chance we can make use of our caches and avoid main memory.
- We can access object metadata together with a much higher probability, so that should be in cache too.
- Tracking pages means work lists are smaller. Less pressure on work lists means less contention, and fewer stalls.
- And speaking of the highway, we can take our metaphorical engine into gears we've never been able to before, since now we can use vector hardware!



Exploiting vector hardware

Designed primarily for accelerating media applications.

But we don't really care, we want the ***64-byte registers*** ...

... and *powerful* ***bit-manipulation*** *and* ***permutation*** *ops.*

AMD Zen 4+ or Intel Ice Lake or later, only. (Maybe arm64 SVE.)

If you're only passively familiar with vector hardware, you might be confused as to how we can even do any of this stuff with it. Usually you hear about vector hardware for fancy math in AI or media applications.

But the truth is that what we don't really care about that. What we *do* care about is the fact that we get access to larger registers, like 512-bits wide instead of our usual 64-bits wide. This is actually enough for us to hold *all* the metadata for an entire page in just two registers, right on the CPU.

This wasn't even an option for the graph flood, where we'd be jumping between scanning objects that are all sorts of different sizes. Sometimes you needed two bits of metadata and sometimes you needed ten thousand. You wouldn't know until you got there.

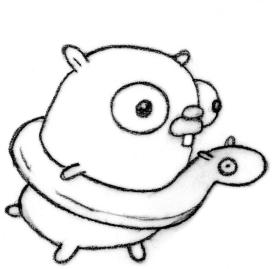
That's still not *quite* enough, since to make this actually work in practice, we need some of the powerful bit manipulation instructions that are only available in more recent hardware to make this work. I won't go into a lot of detail since it's somewhat complicated and outside the scope of this talk, but come talk to me after if you're curious what Galois fields have to do with garbage collection!

What you need to know is that (1) we've figured out how to do it, (2) it works well, and (3) we can only do this effectively on newer hardware. You need AMD Zen 4 or Intel Ice Lake, at the least, in terms of x86 hardware.

Evaluation (so far)

So that's it for how it works. How much does it actually help?

Early results



Today: anecdotally, approx. ***10% less GC CPU time.***

Some applications benefit way more, up to *-40%*!

Expect ***another -10%*** when ***vector acceleration*** is available.

(Not always possible.)

It can be quite a lot, actually! We're seeing reductions in garbage collection CPU costs between 10% and 40% in our benchmark suite. 10% reduction in garbage collection time seems to be the modal improvement, speaking anecdotally. It's quite hard to give a concrete average because it turns out it's quite workload-dependent.

However, we've just rolled this out to most of Google and we're seeing promising results.

Also, using vector hardware gets us another 10%, roughly, when it applies, which is great!

Notes on performance

Not all workloads benefit. Still ironing out all the edge cases.

For example, led to single-object page scan fast path.

But it's also **more applicable than we expected**.

Effective even at relatively low densities, e.g. 2% of page.

But it's not all perfect. Like I said, it's workload-dependent. And more specifically, we know that not all workloads benefit.

At the root of this work is the hypothesis that we even can accumulate multiple objects to scan on a single page. This is clearly the case if the heap has a very regular structure: objects of the same size at a similar depth in the graph. But there are some workloads that often require us to scan only a single object per page at a time. This is potentially *worse* than the graph flood because we might be doing more work than before, trying to accumulate objects on pages and failing. We worked around this early on by adding a fast path for pages which only have a single object to scan, which helped reduce and eliminate regressions.

On the bright side, we learned that page scanning is more applicable than we expected, and that even relatively low densities, like even only scanning 2% of the page at a time, can yield improvements, which is motivating.

Availability

I hope by this point you're all excited and buzzing in your seats, eager to get your hands on it.

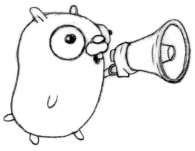
Go 1.25

Available as an experiment.

Set *GOEXPERIMENT=greenteagc* at build time.

Vector acceleration not available.

Green Tea is already available as an experiment in the recent Go 1.25. You can set the environment variable GOEXPERIMENT to greenteagc at build time to enable it.



Go 1.26

Planned as the default.

Set `GOEXPERIMENT=nogreenteagc` to opt out.

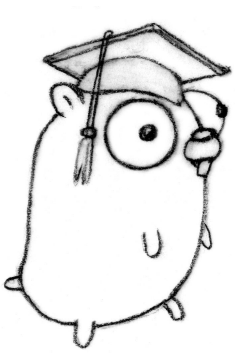
Vector acceleration available.

And we expect to make it the default in Go 1.26. This release will also add vector acceleration on newer x86 hardware.

The journey

And before I wrap up this talk, I've spent all this time talking about where we've ended up, but I want to take a moment to talk about the journey that got us here. The human element of the technology. And yes, I will tell you why it's called Green Tea.

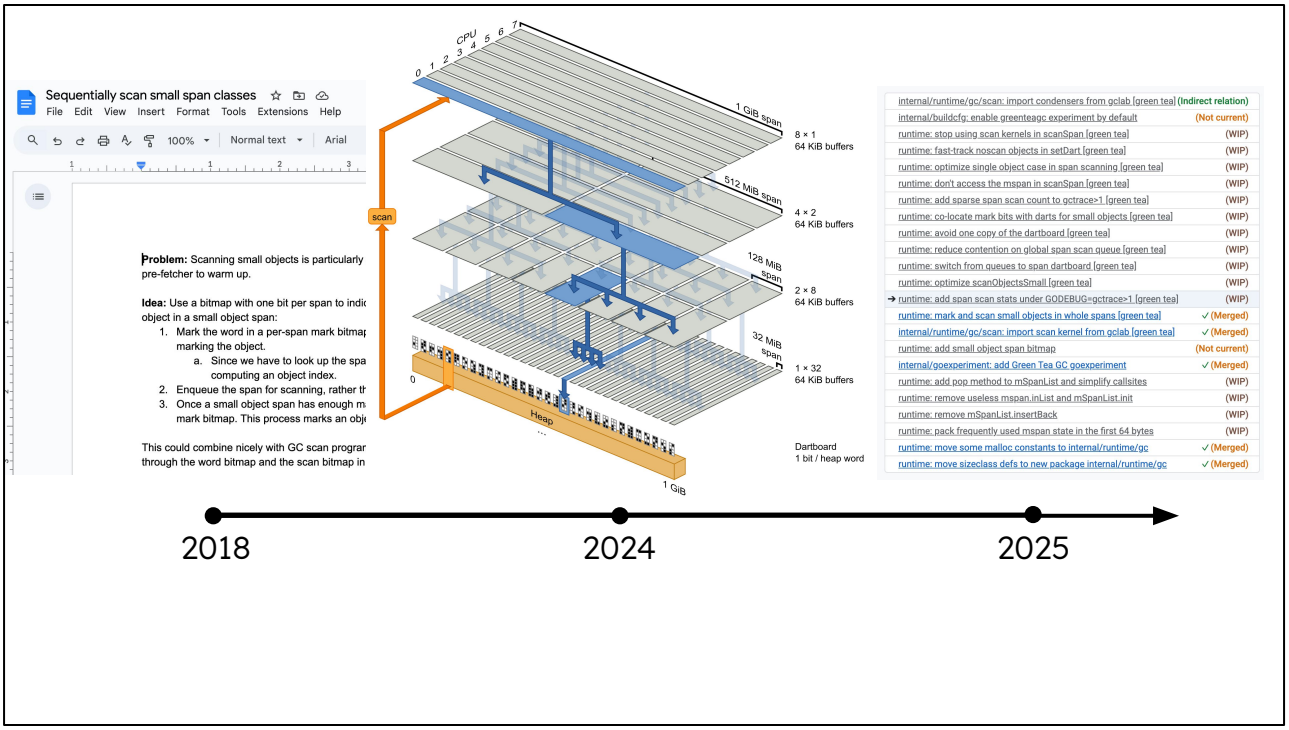
Good ideas take a village



Austin Clements *(Go team)*, Michael Pratt *(Go team)*, Cherry Mui *(Go team)*, David Chase *(Go team)*, Keith Randall *(Go team)*, Yves Vandriessche *(Intel)*, *... and more.*

When I introduced Green Tea earlier, it was with a single, simple idea. Like the spark of inspiration that just one single person had.

But that's not true at all. So much of the work we've done on this had input from many different people over the years. And it's been like, 7 years. There were a lot of ideas that didn't work, and there were a *lot* of details that needed figuring out. Just to make this single, simple idea viable.



The seeds of this idea go all the way back to 2018. What's funny is that everyone on the team thinks someone else thought of this initial idea.

I promised I'd tell you how Green Tea got its name. It got its name in 2024 when, Austin, Go tech lead, worked out a prototype of an earlier version of Green Tea while cafe crawling in Japan and drinking LOTS of matcha! This prototype showed that the core idea of Green Tea was viable. And from there we were off to the races.

Throughout 2025, as I implemented and productionized Green Tea, the ideas evolved and changed even further.

This took so much collaborative exploration because Green Tea is not just an algorithm, but an entire design space. One that I don't think any of us could've navigated alone. It's not enough to just have the idea, but you need to figure out the details and *prove it*. And now that we've done it, we can finally *iterate*. The future of Green Tea is bright.

Please try it out by setting GOEXPERIMENT=greenteagc and let us know how it goes! We're really excited about this work and want to hear from you!